



UNIVERSIDAD
DE MURCIA

UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA

MÁSTER EN INTELIGENCIA ARTIFICIAL

TRABAJO FIN DE MÁSTER

CURSO ACADÉMICO [2024-2025]

2 de julio de 2025

TÍTULO:

**ESTUDIO DE TÉCNICAS DE MONTE CARLO Y MONTE CARLO
TREE SEARCH APLICADAS AL APRENDIZAJE POR REFUERZO EN
JUEGOS DE ESTRATEGIA**

AUTORA:

ANA GIL MOLINA

TUTOR:

ALBERTO RUIZ GARCÍA

COTUTOR:

PEDRO ENRIQUE LÓPEZ DE TERUEL ALCOLEA

Resumen

Este Trabajo de Fin de Máster se centra en el estudio e implementación de algoritmos para la toma de decisiones en juegos de estrategia, con especial énfasis en métodos basados en Monte Carlo y Monte Carlo Tree Search (MCTS).

Se inicia con una introducción que expone brevemente los objetivos del trabajo. A continuación, se profundiza en los conceptos generales de la Teoría de Juegos, incluyendo definiciones y ejemplos, y se presentan las distintas representaciones de los juegos, en forma normal y en forma extensiva. Además, se detallan los algoritmos seleccionados para el análisis, así como diversas políticas de selección utilizadas en MCTS, como ϵ -greedy, UCB y Softmax.

Posteriormente, se realizan experimentos y simulaciones en cuatro juegos clásicos de estrategia: Nim, 4 en raya, Othello y Go. Para cada uno de ellos, se llevan a cabo simulaciones enfrentando agentes de Monte Carlo contra un jugador humano, con el fin de comprender la lógica del juego, y después se realizan comparativas entre los distintos agentes implementados, enfrentados entre sí por pares. Además, se desarrolla una herramienta de depuración basada en diagramas de árbol para MCTS, que facilita el análisis y la comprensión del comportamiento de los algoritmos, especialmente en juegos sencillos, más fáciles de visualizar.

Finalmente, se discuten los resultados obtenidos y se extraen conclusiones acerca de la eficacia y aplicabilidad de las técnicas estudiadas. En los anexos se incluyen detalles de la implementación en Python de los algoritmos, las políticas de selección, los juegos y la herramienta de depuración.

En definitiva, este trabajo profundiza en el ámbito de la inteligencia artificial aplicada a juegos de estrategia y aporta resultados relevantes en torno a su desarrollo y evaluación.

Palabras clave: juegos de estrategia, Monte Carlo, Monte Carlo Tree Search, políticas de selección, simulación

Extended Abstract

This Master's Thesis focuses on the study and implementation of decision-making algorithms in strategy games, with particular emphasis on methods based on Monte Carlo and Monte Carlo Tree Search (MCTS).

It begins with an introduction that briefly outlines the primary objectives and motivation behind the study. This is followed by a deeper exploration of general concepts from Game Theory, a formal framework for analyzing different kinds of situations involving multiple decision-makers whose interest may be conflicting. This section includes definitions of concepts such as rational choice theory, utility functions, or strategic interdependence, and provides some illustrative examples of the concepts presented. A classification of games based on various properties, such as zero-sum versus non-zero-sum, perfect versus imperfect information, deterministic versus stochastic environments, and simultaneous versus sequential actions, is also given. This is necessary for understanding the several types of strategic games.

Afterwards, the thesis delves into two forms of presenting games: the normal form and the extensive form. The normal form is introduced through matrix-based representations of strategic scenarios where decisions are made simultaneously. In addition, some relevant definitions such as strictly dominated strategies, best responses and Nash equilibrium, are presented, accompanied by examples used to demonstrate how the theoretical concepts can be used to identify rational outcomes.

Following the analysis of the normal form, the thesis introduces the extensive form of games, which is used to represent scenarios where interactions between players develop sequentially. Games in extensive form are usually represented using a game tree, in which nodes represent decision points and branches represent possible actions.

The extensive form is useful for studying games of perfect and imperfect information. In games of perfect information, all players are fully aware of all previous moves, while in games of imperfect information, players may have to make some decisions without complete knowledge of earlier actions. This distinction is made clear by the concept of information set. Additionally, the notion of subgame is also defined, leading to solution concepts such as Subgame Perfect Nash Equilibrium (SPNE), an extension of the concept of Nash Equilibrium previously defined. To solve games in extensive form, the method of backward induction is employed. This method consists of analyzing the game from its terminal nodes back to the root, determining the optimal strategy at each point by assuming rational play in future decisions.

Then the thesis transitions to the study of decision-making algorithms designed to address limitations of classical game theory in complex environments. These methods are necessary because traditional methods like Minimax and its variants (α - β pruning, $\max^n$, Expectimax) are impractical for strategic games which have a large scale and are too complex. To overcome these limitations, the work explores Monte Carlo and Monte Carlo Tree Search algorithms, which are based on simulations.

The basic form is known as flat Monte Carlo. This method samples available actions and runs numerous simulations for each, providing an estimation of expected rewards. However, this method may fail to efficiently allocate computational effort in complex scenarios. Because of this, the Monte Carlo Tree Search (MCTS) algorithm is introduced as a more sophisticated method.

MCTS builds a search tree by selectively expanding the most promising parts of the game space. In each iteration, four steps are carried out: selection, expansion, simulation and backpropagation. In the selection step, different selection policies are used to balance between exploration and exploitation of the game space. Several of these policies are presented, including ϵ -greedy, which stochastically selects between the best known action and a random alternative; UCB (Upper Confidence Bound), which uses a statistical bound to prioritize less explored actions which may potentially be rewarding; and Softmax, which introduces a probabilistic model for selecting actions based on expected rewards.

Subsequently, experiments and simulations are conducted on four classic strategy games: Nim, Connect 4, Othello, and Go. These games serve to analyze how the algorithms behave in environments with varying complexity, starting with the simplest one (Nim) and increasing in complexity until reaching a highly complex and strategically rich scenario (Go).

Nim is a strategic game where two players move in turns, removing sticks from a single pile until none remain. In the variant used here, a player can remove between 1 and k sticks per turn, and the objective is to take the last stick from the pile. A well known winning strategy based on modular arithmetic is presented, which allows to classify each position as winning or losing.

Connect 4 is a two-player game on a board typically consisting of 6 rows and 7 columns (although in this implementation, board sizes can vary from 4×4 to 10×10), where players take turns dropping their colored discs into columns. The discs occupy the lowest available space in a column, and the goal is to align four consecutive discs of the same color either horizontally, vertically or diagonally.

Othello is a strategic board game for two players typically played on a 8×8 grid (in

this implementation, board sizes can vary from 4×4 to 10×10). Players alternate placing discs with their color facing up. The goal is to capture the opponent's discs by trapping them between two of their color. When discs are captured, they are flipped to the capturing player's color. The game ends when there are no valid moves remaining, and the player with the majority of discs on the board wins.

Go is a complex strategic game played in a square grid with an odd number of intersections, varying from 5×5 to 19×19 . In each turn, players place stones of their color on empty intersections, seeking to control territory and capture opponent stones by removing their liberties. The game incorporates rules such as no-suicide and situational Superko to avoid repeating board states. The game ends when both players pass consecutively, and the player with the higher score wins. This score is calculated taking into account the stones on board, the captured stones, the surrounded territory, and a Komi compensation for the second player.

For each of these games, simulations are carried out where Monte Carlo agents compete against a human player to better understand the game logic. These are followed by comparative simulations between different implemented agents, playing against each other in pairs. Furthermore, a debugging tool based on tree diagrams is developed for MCTS, which facilitates the analysis and understanding of the algorithm's behavior, especially in simpler games that are easier to visualize.

Finally, the results obtained are discussed, and conclusions are drawn regarding the effectiveness and applicability of the studied techniques. The annexes include detailed Python implementations of the algorithms, selection policies, games, and the debugging tool.

Ultimately, this work delves into the field of artificial intelligence applied to strategy games and provides relevant findings for their development and evaluation.

Keywords: strategy games, Monte Carlo, Monte Carlo Tree Search, selection policies, simulation

Índice

1. Introducción	12
1.1. Contexto general y motivación sobre los juegos de estrategia	12
1.2. Objetivos del trabajo de fin de máster	13
1.3. Estructura del documento	13
1.4. Metodología	14
1.4.1. Plataforma de ejecución	16
1.4.2. Uso de herramientas de IA generativa	17
2. Marco teórico	18
2.1. Fundamentos de la teoría de juegos	18
2.1.1. Forma normal	20
2.1.2. Forma extensiva	23
2.2. Algoritmos de toma de decisiones en juegos	28
2.2.1. Monte Carlo	29
2.2.2. Monte Carlo Tree Search	30
2.2.2.1. Políticas de selección	31
2.2.2.1.1. ϵ -greedy	31
2.2.2.1.2. UCB	31
2.2.2.1.3. Softmax	33
3. Experimentos y resultados	33
3.1. Nim	33
3.1.1. Simulaciones humano vs Monte Carlo	35
3.1.2. Herramienta de depuración para Monte Carlo Tree Search	42
3.1.3. Simulaciones comparativas entre varios agentes	44
3.2. 4 en raya	46
3.2.1. Simulaciones humano vs Monte Carlo	47
3.2.2. Herramienta de depuración para Monte Carlo Tree Search	53

3.2.3. Simulaciones comparativas entre varios agentes	54
3.3. Othello	56
3.3.1. Simulaciones humano vs Monte Carlo	57
3.3.2. Simulaciones comparativas entre varios agentes	62
3.4. Go	63
3.4.1. Simulaciones humano vs Monte Carlo	65
3.4.2. Simulaciones comparativas entre varios agentes	70
4. Conclusiones	73
Referencias	76
A. Abreviaturas	77
B. Detalles de la implementación en Python	77
B.1. Implementación de los juegos	77
B.1.1. Nim	78
B.1.2. 4 en raya	79
B.1.3. Othello	83
B.1.4. Go	88
B.2. Implementación de los métodos para la toma de decisiones	96
B.2.1. Monte Carlo	96
B.2.2. Monte Carlo Tree Search	97
B.3. Implementación de las políticas de selección para MCTS	102
B.3.1. ϵ -greedy	103
B.3.2. UCB1	105
B.3.3. UCB2	107
B.3.4. Softmax	109
B.3.5. Softmax Adaptativo	111
B.4. Implementación de la herramienta de depuración para MCTS con <code>graphviz</code>	112

Índice de figuras

1.	Diagrama de árbol en el que los nodos están conectados por arcos que representan las acciones tomadas por los jugadores. Al final de cada camino (historia particular del juego) están los payoffs de los jugadores representados por una tupla, donde el primer elemento de la tupla representa el payoff del jugador 1 (E), y el segundo elemento representa el payoff del jugador 2 (M). Las flechas azules muestran el camino encontrado mediante inducción hacia atrás.	24
2.	Diagrama de árbol en el que se representa el Dilema del Prisionero, un juego de información imperfecta.	26
3.	Diagrama de árbol en el que se representa un juego de información perfecta. Las flechas azules señalan el camino encontrado mediante inducción hacia atrás, mientras que los triángulos morados rodean a cada uno de los subjuegos (3 en total) que, además, aparecen numerados en los recuadros morados.	26
4.	Estado inicial de la primera partida de Nim con Humano vs Monte Carlo. .	37
5.	Primer movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.	37
6.	Segundo movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.	37
7.	Tercer movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.	37
8.	Cuarto movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.	37
9.	Quinto movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.	38
10.	Sexto movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.	38
11.	Séptimo movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.	38
12.	Octavo movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.	38

13.	Último movimiento de la primera partida de Nim, llevado a cabo por el jugador humano, ganando la partida.	39
14.	Estado inicial de la segunda partida de Nim con Humano vs Monte Carlo.	39
15.	Primer movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.	39
16.	Segundo movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.	40
17.	Tercer movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.	40
18.	Cuarto movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.	40
19.	Quinto movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.	41
20.	Sexto movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.	41
21.	Séptimo movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.	41
22.	Octavo movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.	42
23.	Último movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo, ganando la partida.	42
24.	Diagrama de árbol de profundidad 2 de la partida de Nim inicializada con 5 palos y permitiendo retirar un máximo de 3 palos por turno, con MCTS con método de selección ϵ -greedy con $\epsilon = 0.2$	43
25.	Estado del nodo con ID <code>nodo3</code>	44
26.	Resultado de enfrentar al agente de Monte Carlo frente al agente de MCTS con ϵ -greedy en 100 partidas de Nim	45
27.	Estado inicial de la partida de 4 en raya con Humano vs Monte Carlo. . . .	48
28.	Movimientos del 1º al 4º de la partida de 4 en raya con Humano vs Monte Carlo.	48
29.	Movimientos del 5º al 8º de la partida de 4 en raya con Humano vs Monte Carlo.	49

30.	Movimientos del 9 ^o al 12 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	50
31.	Movimientos del 13 ^o al 16 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	50
32.	Movimientos del 17 ^o al 20 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	51
33.	Movimientos del 21 ^o al 24 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	51
34.	Movimientos del 25 ^o al 28 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	52
35.	Movimientos del 29 ^o al 32 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	52
36.	Diagrama de árbol de profundidad 1 de la partida de 4 en raya inicializada con un tablero vacío de 6×7 , con MCTS con método de selección UCB1 con $c = 1$	54
37.	Estado del nodo con ID <code>nodo4</code>	54
38.	Estado inicial de la partida de Othello con Humano vs Monte Carlo.	58
39.	Movimientos del 1 ^o al 4 ^o de la partida de Othello con Humano vs Monte Carlo.	59
40.	Movimientos del 5 ^o al 8 ^o de la partida de Othello con Humano vs Monte Carlo.	59
41.	Movimientos del 9 ^o al 12 ^o de la partida de Othello con Humano vs Monte Carlo.	60
42.	Movimientos del 13 ^o al 16 ^o de la partida de Othello con Humano vs Monte Carlo.	60
43.	Movimientos del 17 ^o al 20 ^o de la partida de Othello con Humano vs Monte Carlo.	61
44.	Movimientos del 21 ^o al 24 ^o de la partida de Othello con Humano vs Monte Carlo.	61
45.	Movimientos del 25 ^o al 28 ^o de la partida de 4 en raya con Humano vs Monte Carlo.	61

46.	Movimientos del 29 ^o al 32 ^o de la partida de Othello con Humano vs Monte Carlo.	62
47.	Estado inicial de la partida de Go con Humano vs Monte Carlo.	66
48.	Movimientos del 1 ^o al 4 ^o de la partida de Go con Humano vs Monte Carlo.	66
49.	Movimientos del 5 ^o al 8 ^o de la partida de Go con Humano vs Monte Carlo.	67
50.	Movimientos del 9 ^o al 12 ^o de la partida de Go con Humano vs Monte Carlo.	67
51.	Movimientos del 13 ^o al 16 ^o de la partida de Go con Humano vs Monte Carlo.	68
52.	Movimientos del 17 ^o al 20 ^o de la partida de Go con Humano vs Monte Carlo.	68
53.	Movimientos del 21 ^o al 24 ^o de la partida de Go con Humano vs Monte Carlo.	69
54.	Movimientos del 25 ^o al 28 ^o de la partida de Go con Humano vs Monte Carlo.	69
55.	Movimientos del 29 ^o al 32 ^o de la partida de Go con Humano vs Monte Carlo.	69
56.	Movimientos del 33 ^o al 36 ^o de la partida de Go con Humano vs Monte Carlo.	70

Índice de cuadros

1.	Resultados de enfrentamientos por pares entre agentes en el juego Nim (100 partidas por par).	46
2.	Resultados de enfrentamientos por pares entre agentes en el juego 4 en raya (100 partidas por par).	56
3.	Resultados de enfrentamientos por pares entre agentes en el juego Othello (100 partidas por par).	63
4.	Resultados de enfrentamientos por pares entre agentes en el juego Go (10 partidas por par).	71

1. Introducción

1.1. Contexto general y motivación sobre los juegos de estrategia

Los juegos de estrategia han sido estudiados durante décadas, tanto en el ámbito de la teoría de juegos como en la inteligencia artificial. Tradicionalmente, estos juegos se han analizado mediante métodos matemáticos clásicos, usando una representación formal de las decisiones de los jugadores y buscando soluciones óptimas usando los equilibrios de Nash o algoritmos como Minimax y sus variantes. Estos enfoques proporcionan una base sólida para entender cómo actuar racionalmente en situaciones estratégicas competitivas.

Sin embargo, a medida que los juegos se vuelven más complejos, los métodos clásicos presentan limitaciones significativas, especialmente en términos de viabilidad computacional. La exploración exhaustiva del árbol de decisiones se vuelve prácticamente impracticable, lo cual dificulta encontrar soluciones óptimas en un tiempo razonable.

Es debido a estas limitaciones por lo que se han desarrollado métodos más sofisticados, que combinan simulaciones y métodos heurísticos de búsqueda para manejar la elevada complejidad de los juegos. Entre ellos, los algoritmos de Monte Carlo y Monte Carlo Tree Search (MCTS) han ganado relevancia en los últimos años. Particularmente, MCTS permite explorar selectivamente las partes más prometedoras del espacio de juego, equilibrando exploración y explotación. El estudio de este equilibrio es un dilema clave en la inteligencia artificial, y ha permitido grandes avances en la eficiencia para la resolución de juegos de estrategia complejos, facilitando avances en áreas como la inteligencia artificial aplicada a juegos.

Además, métodos como MCTS no solo se limitan al ámbito de los juegos, sino que presentan una gran aplicabilidad en diversos ámbitos más generales. Por ejemplo, algunos de estos métodos tienen un importante potencial en problemas reales de planificación, robótica, optimización... es decir, en cualquier ámbito donde la toma de decisiones en entornos inciertos y con grandes espacios de estado sea necesaria.

No obstante, este trabajo se centra en los juegos de estrategia, puesto que estos constituyen un entorno controlado para realizar pruebas, ideal para comprobar las potencialidades y las limitaciones de los distintos métodos como Monte Carlo y MCTS. Incluir variantes de juegos con diferente nivel de dificultad, complejidad y tamaño permite realizar un análisis más preciso de la robustez y el rendimiento de los algoritmos, pudiendo comprender mejor sus puntos fuertes y débiles, y orientando futuras mejoras o el desarrollo de nuevos algoritmos.

Finalmente, este trabajo también busca sentar las bases para comprender y desarrollar sistemas de inteligencia artificial y aprendizaje por refuerzo más avanzados, como aquellos basados en aprendizaje por refuerzo profundo (*deep reinforcement learning*). Un ejemplo de esto es el programa *AlphaGo*, desarrollado por Google DeepMind para jugar al juego de mesa Go, que está basado en redes neuronales profundas y aprendizaje por refuerzo.

1.2. Objetivos del trabajo de fin de máster

Los objetivos generales del trabajo son el estudio, implementación y experimentación con algoritmos de toma de decisiones para juegos de estrategia. En particular, se estudian los métodos basados en Monte Carlo y Monte Carlo Tree Search (MCTS). Se pretende analizar las ventajas y limitaciones de estos métodos, creados con la intención de superar las limitaciones de los métodos clásicos de teoría de juegos, especialmente en entornos con una gran complejidad computacional y estratégica.

Por otro lado, también se pretende evaluar el desempeño de dichos algoritmos en distintos juegos de estrategia con diferentes niveles de dificultad, desde juegos con espacios de estados pequeños y simples, como Nim, hasta juegos con estructuras mucho más complejas, como Go.

1.3. Estructura del documento

El desarrollo del trabajo comienza en la sección 2, donde se exponen los fundamentos teóricos y los conceptos generales que servirán como base para la posterior implementación de los algoritmos, así como para la experimentación y el análisis de resultados. En primer lugar, en la sección 2.1 se realiza una revisión de los orígenes y principios fundamentales de la Teoría de Juegos, disciplina clave para entender el comportamiento estratégico de los jugadores en entornos competitivos. Esta sección proporciona un contexto acerca de los métodos clásicos de toma de decisiones y resalta sus limitaciones en escenarios de elevada complejidad, lo que motiva la exploración de técnicas más avanzadas como Monte Carlo y Monte Carlo Tree Search.

Así, se da paso a la sección 2.2, donde los algoritmos más avanzados, desarrollados para superar las limitaciones de la teoría clásica, son expuestos teóricamente. La implementación de los mismos, así como de los juegos utilizados en este trabajo, se puede encontrar en el anexo B.

A continuación, en la sección 3 se llevan a cabo los experimentos y se muestran los

resultados para 4 juegos estratégicos de creciente dificultad: Nim, 4 en raya, Othello y Go. Cada subsección de esta sección está dedicada a un juego, comenzando con la explicación de las reglas del juego, para luego dar paso a los experimentos y simulaciones.

Finalmente, en la sección 4 se presentan las conclusiones finales del trabajo, destacando los principales hallazgos y las limitaciones observadas en el desempeño de los algoritmos en cada uno de los juegos analizados. Además, se discuten posibles mejoras que podrían implementarse para superar dichas limitaciones.

1.4. Metodología

En este apartado, se explica cuál ha sido la metodología utilizada en el desarrollo del trabajo. Todos los algoritmos, juegos y herramientas han sido implementados en Python. Las principales librerías de Python utilizadas son:

- **sys**: proporciona funciones y variables para interactuar con el intérprete de Python. Se utiliza para modificar la variable de entorno `sys.path`, permitiendo agregar directorios al path de búsqueda de módulos y así facilitar la importación de módulos personalizados.
- **os**: permite la interacción con el sistema operativo, manejo de archivos, rutas y procesos.
- **pandas**: librería para manipulación y análisis de datos estructurados, en particular tablas.
- **random**: genera números aleatorios y realiza operaciones relacionadas con aleatoriedad.
- **numpy**: librería fundamental para cálculo numérico y manejo eficiente de arrays multidimensionales.
- **copy**: permite realizar copias superficiales y profundas de objetos en Python.
- **graphviz**: interfaz en Python para la creación y visualización de grafos mediante el motor de Graphviz.
- **abc**: soporte para clases abstractas y definición de interfaces en Python.

A continuación, se muestra cómo se ha organizado dicha implementación, mediante diferentes directorios conteniendo el código necesario para los experimentos. De forma general, se tienen las siguientes carpetas:

- **games/**: Contiene las implementaciones de los juegos, sobre los que se prueban los algoritmos. Incluye los archivos:
 - **nim.py**: Implementación del juego Nim.
 - **connect4.py**: Implementación del juego 4 en raya.
 - **othello.py**: Implementación del juego Othello.
 - **go.py**: Implementación del juego Go.
- **methods/**: Contiene los métodos implementados para la toma de decisiones, explicados en la sección 2.2. Incluye los archivos:
 - **monte_carlo.py**: Implementación del método de Monte Carlo, explicado en la sección 2.2.1.
 - **monte_carlo_tree_search.py**: Implementación del método de Monte Carlo Tree Search, explicado en la sección 2.2.2.
- **selection/**: Contiene las diferentes políticas de selección explicadas en la sección 2.2.2.1, que se utilizan dentro del algoritmo MCTS. Incluye:
 - **algorithm.py**: Clase genérica para los algoritmos de selección.
 - **epsilon_greedy.py**: Implementación de la política de selección ϵ -greedy, explicada en la sección 2.2.2.1.1.
 - **ucb1.py**: Implementación de la política de selección UCB1, explicada en la sección 2.2.2.1.2.
 - **ucb2.py**: Implementación de la política de selección UCB2, explicada en la sección 2.2.2.1.2.
 - **softmax.py**: Implementación de la política de selección Softmax, explicada en la sección 2.2.2.1.3.
 - **adaptive_softmax.py**: Implementación de la política de selección Softmax adaptativo, explicada en la sección 2.2.2.1.3.

- **graphviz/**: Con el fin de facilitar el análisis del comportamiento del árbol de búsqueda generado por MCTS, se ha desarrollado una herramienta de depuración que permite visualizar gráficamente dicho árbol mediante la biblioteca **graphviz**. Esta herramienta se encuentra en el archivo:
 - **MCTS_graphviz.py**: Implementación de la herramienta de depuración para MCTS.
- **notebooks**: Finalmente, se han desarrollado varios notebooks de Jupyter en los que se realizan simulaciones, visualizando los resultados y comparando el rendimiento de los distintos métodos en cada juego. Se encuentra un notebook por cada juego:
 - **nim_simulations.ipynb**: Notebook con los resultados para el juego Nim.
 - **connect4_simulations.ipynb**: Notebook con los resultados para el juego 4 en raya.
 - **othello_simulations.ipynb**: Notebook con los resultados para el juego Othello.
 - **go_simulations.ipynb**: Notebook con los resultados para el juego Go.

Los detalles específicos de la implementación se encuentran en el anexo B.

1.4.1. Plataforma de ejecución

Todos los experimentos y cálculos se realizaron en un entorno local, utilizando un equipo portátil con las siguientes características:

- **Fabricante del dispositivo**: HP
- **Sistema operativo**: Windows 11 Home, versión 24H2 (build 26100.4484)
- **Procesador**: Intel Core i3-8130U CPU @ 2.20GHz (2 núcleos, 4 hilos)
- **Memoria RAM**: 8 GB
- **Arquitectura del sistema**: 64 bits (x64)
- **Entorno de ejecución**: Python 3.8.5 ejecutado localmente
- **Entorno de desarrollo**: Jupyter Notebook y ejecución directa desde scripts locales

■ **Versiones de las librerías utilizadas:**

- numpy: 1.23.5
- pandas: 2.0.3
- graphviz: 0.20.3

No se ha hecho uso de GPU ni de paralelización explícita en los experimentos. Todos los cálculos se llevaron a cabo en un único núcleo del procesador en ejecución secuencial.

1.4.2. Uso de herramientas de IA generativa

A lo largo de este trabajo, se han usado herramientas de IA generativa, particularmente ChatGPT, como ayuda en la revisión y corrección de errores del código implementado.

El código ha sido implementado manualmente, basado en la teoría presentada en la sección 2.2 sobre los algoritmos, y en la sección 3 sobre los juegos. Sin embargo, se han preguntado sugerencias generales a ChatGPT sobre la organización del mismo, y dudas sobre elementos en la implementación que pudieran estar dando errores o funcionando incorrectamente. Además, una vez finalizado el código, se ha utilizado ChatGPT para la revisión y corrección de posibles errores pasados por alto.

2. Marco teórico

2.1. Fundamentos de la teoría de juegos

En cualquier sociedad, las personas interactúan constantemente. Estas interacciones pueden ser cooperativas, como cuando dos compañeros de negocios colaboran en un proyecto, o competitivas, como cuando dos empresas luchan por hacerse con el mercado. En cualquier caso, se da una situación de **interdependencia**, donde el comportamiento de una persona afecta al bienestar de otra, ya sea positiva o negativamente. Las situaciones de interdependencia se llaman **escenarios estratégicos** porque, para que una persona elija cómo actuar, debe tener en cuenta como las demás personas a su alrededor escogen sus acciones.

La **teoría de juegos** es una disciplina que estudia situaciones estratégicas en las que los jugadores deben tomar decisiones considerando sus propias posibles acciones, pero también las de los demás jugadores. Este campo de estudio se ha vuelto especialmente relevante en diversas disciplinas, como la economía, las ciencias políticas, la biología o el derecho (Watson, 2013). Además, proporciona una base teórica fundamental para comprender y diseñar juegos de estrategia.

Un juego, en el sentido cotidiano, es una actividad en la que los jugadores compiten entre sí de acuerdo con un conjunto de reglas. Antes de definir formalmente qué es un juego en el contexto de la teoría de juegos, es importante entender algunos aspectos acerca de los jugadores y de la toma de decisiones.

La teoría de juegos se basa en la **teoría de la elección racional**, que consiste en que cada jugador elige la mejor acción de acuerdo con sus preferencias, de entre todas las acciones que tiene disponibles. Más específicamente, esta teoría supone que cada jugador:

- Conoce el conjunto de acciones posibles que puede tomar en una jugada.
- Tiene preferencias bien definidas y consistentes entre las distintas acciones.
- Elige racionalmente la acción de mayor preferencia.

Según Osborne (2004), una forma de representar las preferencias de los jugadores es mediante una **función de utilidad** o **payoff**, que asocia un número a cada acción, de forma que se prefieren aquellas acciones con un valor mayor. Además, existe la posibilidad de que varias acciones tengan el mismo nivel de preferencia. Por lo tanto, de manera

formal, la teoría de la elección racional establece que: *La acción escogida por un jugador es al menos tan buena, según sus preferencias, como cualquier otra acción disponible.*

Tal como se presenta en Watson (2013), cualquier representación de un juego debe contener los siguientes elementos:

1. Una lista de jugadores.
2. Una descripción completa de las posibles acciones de los jugadores.
3. Una descripción de lo que los jugadores saben cuando actúan.
4. Una especificación de cómo las acciones de los jugadores llevan a los resultados.
5. Una especificación de las preferencias de los jugadores sobre los resultados.

Ejemplo 2.1 *Para el juego del ajedrez, los elementos anteriores se pueden especificar de la siguiente manera:*

1. *Hay dos jugadores.*
2. *Los jugadores se alternan para mover sus piezas en el tablero, sujetos a reglas sobre qué movimientos pueden hacer en una configuración determinada del tablero.*
3. *Los jugadores observan los movimientos del otro, por lo que cada uno conoce el historial de jugadas completo a medida que avanza el juego.*
4. *Un jugador que captura al rey del otro gana la partida, y en caso contrario, se da un empate.*
5. *Aunque no está implícito en las reglas del juego, se puede asumir que los jugadores prefieren ganar a empatar, y empatar a perder.*

Además de los elementos básicos que definen un juego, estos se suelen clasificar en función de diferentes propiedades, entre las que se encuentran:

- **Suma cero:** un juego es de suma cero si la ganancia de un jugador implica una pérdida equivalente para otro. En el caso de dos jugadores, esto significa competencia estricta.
- **Información:** puede ser perfecta (los jugadores conocen todo el historial de jugadas) o imperfecta (existe incertidumbre sobre las acciones previas o el estado del juego).

- **Determinismo:** un juego es determinista si no intervienen factores aleatorios, es decir, si no hay incertidumbre sobre las recompensas.
- **Secuencialidad:** los juegos pueden ser secuenciales (los jugadores actúan por turnos) o simultáneos (actúan al mismo tiempo).
- **Discreción:** se refiere a si las acciones son discretas (conjunto finito de elecciones) o continuas (en tiempo real).

Los juegos con dos jugadores que son de suma cero, con información perfecta, deterministas, secuenciales y discretos se describen como **juegos combinatorios**. Esto incluye juegos como el Go, el ajedrez y el tres en raya, entre muchos otros (Browne et al., 2012).

2.1.1. Forma normal

Definición 2.2 *Considérese la siguiente notación:*

- Sea un conjunto de jugadores $N = \{1, 2, \dots, n\}$. Sea un jugador $i \in N$.
- Se denota s_i una estrategia particular del jugador i , S_i el conjunto de todas las posibles estrategias del jugador i , $s = (s_1, \dots, s_n)$ un perfil de estrategias, y $s_{-i} = (s_1, \dots, s_{i-1}, s_{i+1}, \dots, s_n)$ un perfil de estrategias con las estrategias particulares de todos los jugadores excepto la del jugador i . Entonces $s = (s_i, s_{-i})$. Además, $S_{-i} = \prod_{j \neq i} S_j$, y $S = \prod_{i \in N} S_i$ es el conjunto de todos los posibles perfiles de estrategias.
- Sea la función de utilidad $u_i : S \rightarrow \mathbb{R}$ tal que $u_i(s) = u_i(s_1, \dots, s_n) = u_i(s_i, s_{-i})$ (en ocasiones se usa la función de beneficio $\pi_i : S \rightarrow \mathbb{R}$ tal que $\pi_i(s) = \pi_i(s_1, \dots, s_n) = \pi_i(s_i, s_{-i})$).

Entonces $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ representa un **juego en forma normal**. Un juego en forma normal con dos jugadores y espacio de estrategias finito se puede describir mediante matrices, por lo que en este caso estos juegos también se llaman **juegos matriciales**.

Definición 2.3 Una estrategia $s_i \in S_i$ domina estrictamente a la estrategia $s'_i \in S_i$, $s_i \neq s'_i$ para el jugador $i \in N$ si $u_i(s_i, s_{-i}) > u_i(s'_i, s_{-i}) \forall s_{-i} \in S_{-i}$.

La estrategia $s'_i \in S_i$ está dominada estrictamente si existe $s_i \in S_i$ que domina estrictamente a s'_i .

La estrategia $s_i \in S_i$ es dominante estrictamente para $i \in N$ si s_i domina estrictamente a todas las estrategias $s'_i \neq s_i$, $s'_i \in S_i$.

La estrategia $s'_i \in S_i$ está dominada débilmente por la estrategia $s_i \in S_i$ si $u_i(s_i, s_{-i}) \geq u_i(s'_i, s_{-i}) \forall s_{-i} \in S_{-i}$ y existe $s_{-i} \in S_{-i}$ tal que $u_i(s_i, s_{-i}) > u_i(s'_i, s_{-i})$.

Ejemplo 2.4 Sea el juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ dado por:

- Jugadores: $N = \{1, 2\}$
- Conjuntos de estrategias:
 - $S_1 = \{T, B\}$
 - $S_2 = \{L, M, R\}$
- Payoffs:
 - $u_1(T, L) = 4, u_1(T, M) = 2, u_1(T, R) = 0, u_1(B, L) = 5, u_1(B, M) = 5, u_1(B, R) = -4$
 - $u_2(T, L) = 3, u_2(T, M) = 7, u_2(T, R) = 4, u_2(B, L) = 5, u_2(B, M) = -1, u_2(B, R) = -2$

Entonces el juego $\mathcal{G}$ se puede representar matricialmente como:

		Jugador 2		
		L	M	R
Jugador 1	T	4, 3	2, 7	0, 4
	B	5, 5	5, -1	-4, -2

Ahora, para determinar el perfil de estrategias que serán elegidas racionalmente, se puede aplicar eliminación iterada de estrategias estrictamente dominadas:

- Para el jugador 2, la estrategia M domina estrictamente a R , ya que:
 - $u_2(T, M) = 7 > u_2(T, R) = 4$
 - $u_2(B, M) = -1 > u_2(B, R) = -2$

De esta forma, el jugador 2 no va a jugar R , por lo que se puede eliminar la columna asociada, quedando:

		Jugador 2	
		L	M
Jugador 1	T	4, 3	2, 7
	B	5, 5	5, -1

- Para el jugador 1, la estrategia B domina estrictamente a T , ya que:

- $u_1(B, L) = 5 > u_1(T, L) = 4$
- $u_1(B, M) = 5 > u_1(T, M) = 2$

Así, el jugador 1 no va a jugar T , por lo que se puede eliminar la fila asociada, quedando:

		Jugador 2	
		L	M
Jugador 1	B	5, 5	5, -1

- Para el jugador 2, la estrategia L domina estrictamente a M , ya que:

- $u_2(B, L) = 5 > u_2(B, M) = -1$

Así, el jugador 2 no va a jugar M , por lo que se puede eliminar la columna asociada, quedando:

		Jugador 2
		L
Jugador 1	B	5, 5

Finalmente, el perfil de estrategias (B, L) sobrevive a la eliminación iterada de estrategias estrictamente dominadas, y está compuesto por las estrategias que serán escogidas por los jugadores de forma racional.

Definición 2.5 En un juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$, una estrategia $s_i^* \in S_i$ del jugador $i \in N$ es una **mejor respuesta (best response)** para el jugador i frente a un perfil de estrategias $s_{-i} \in S_{-i}$ si $s_i^* \in BR_i(s_{-i}) = \{s_i \in S_i : u_i(s_i, s_{-i}) \geq u_i(s'_i, s_{-i}) \forall s'_i \in S_i\}$, es decir, si $u_i(s_i^*, s_{-i}) \geq u_i(s'_i, s_{-i}) \forall s'_i \in S_i$, o equivalentemente, si $s_i^* \in \arg \max_{s_i \in S_i} u_i(s_i, s_{-i})$.

Definición 2.6 Un **equilibrio de Nash** de un juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ es un perfil de estrategias $s^* = (s_1^*, \dots, s_n^*) \in S$ tal que para cada jugador $i \in N$, $u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$, $\forall s_i \in S_i$. Alternativamente, $s^* \in S$ es un equilibrio de Nash si y solo si $s_i^* \in BR_i(s_{-i}^*)$ para todos los jugadores $i \in N$.

Ejemplo 2.7 Sea el juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$, donde $N = \{1, 2\}$, $S_1 = \{U, M, D\}$ y $S_2 = \{l, c, r\}$, representado matricialmente como:

		Jugador 2		
		l	c	r
Jugador 1	U	0, 4	4, 0	5, 3
	M	4, 0	0, 4	5, 3
	D	3, 5	3, 5	6, 6

Entonces:

$$\begin{aligned}
 \blacksquare BR_1(l) &= \{M\} & \blacksquare BR_2(U) &= \{l\} \\
 \blacksquare BR_1(c) &= \{U\} & \blacksquare BR_2(M) &= \{c\} \\
 \blacksquare BR_1(r) &= \{D\} & \blacksquare BR_2(D) &= \{r\}
 \end{aligned}$$

Por lo que (D, r) es un equilibrio de Nash, ya que $BR_1(r) = \{D\}$ y $BR_2(D) = \{r\}$.

Ejemplo 2.8 Sea el juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$, donde $N = \{1, 2\}$, $S_1 = \{T, M, D\}$ y $S_2 = \{l, c, r\}$, representado matricialmente como:

		Jugador 2		
		l	c	r
Jugador 1	U	1, 1	0, 1	3, 1
	M	1, 0	2, 2	1, 3
	D	1, 3	3, 1	2, 2

Entonces:

$$\begin{aligned}
 \blacksquare BR_1(l) &= \{T, M, D\} & \blacksquare BR_2(T) &= \{l, c, r\} \\
 \blacksquare BR_1(c) &= \{D\} & \blacksquare BR_2(M) &= \{r\} \\
 \blacksquare BR_1(r) &= \{T\} & \blacksquare BR_2(D) &= \{l\}
 \end{aligned}$$

Por lo que (T, l) , (D, l) y (T, r) son equilibrios de Nash.

Definición 2.9 La **extensión mixta** de un juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$ es un juego Γ , donde:

1. $\Gamma = \langle N, \{\Delta(S_i)\}_{i \in N}, \{\mathbb{E}u_i\}_{i \in N} \rangle$
2. $\Delta(S_i)$ es el conjunto de distribuciones de probabilidad sobre S_i y $\Delta(S) = \prod_{i \in N} \Delta(S_i)$
3. $\mathbb{E}u_i : \Delta(S) \rightarrow \mathbb{R}$ es la función de utilidad de Neumann-Morgenstein que asigna a cada $\sigma \in \Delta(S)$ el valor esperado $\mathbb{E}u_i$ de la lotería sobre S inducido por σ .

Una estrategia $\sigma_i \in \Delta(S_i)$ se llama **estrategia mixta**. Además, una estrategia $\sigma_i \in \Delta(S_i)$ que asigna probabilidad 1 a exactamente un elemento del conjunto S_i se llama **estrategia pura**. Se denota $\sigma_{-i} \in \Delta(S_{-i}) = \prod_{j \neq i} \Delta(S_j)$.

2.1.2. Forma extensiva

Hasta ahora, se ha explicado la forma normal de un juego, en el que los movimientos se dan de forma simultánea. Sin embargo, también existen juegos donde los movimientos se

dan de forma secuencial, es decir, juegos donde los jugadores mueven por turnos. En estos juegos secuenciales, los jugadores pueden observar los movimientos del resto de jugadores antes de elegir un movimiento.

Para representar la interacción estratégica entre los jugadores en un juego secuencial se suele usar un diagrama de árbol, compuesto por nodos y arcos. Los nodos marcan los momentos en los que los jugadores deben tomar una decisión, mientras que los arcos indican las distintas acciones que los jugadores pueden elegir a partir de cada nodo. Un árbol de este tipo se llama **representación en forma extensiva** del juego.

Una forma sencilla de resolver estos juegos es mediante **inducción hacia atrás** (*backward induction*), un procedimiento que analiza el juego desde el final hasta el principio para identificar la mejor acción en cada nodo. Comienza por el último jugador en los nodos terminales, donde el jugador elige la acción que maximiza su ganancia. Luego se va avanzando hacia los nodos anteriores, donde el jugador debe tomar una decisión teniendo en cuenta cómo actuarán los jugadores siguientes, cuyas decisiones ya fueron evaluadas. Ese análisis continúa hacia atrás hasta llegar al principio del juego, determinando de esta forma una estrategia óptima para todos los jugadores (Watson, 2013).

Ejemplo 2.10 *Considérese el siguiente juego secuencial:*

- Hay un mercado dominado por un monopolista (M).
- Una nueva compañía entrante (E) decide si mantenerse fuera (F) o ingresar (I) al mercado del monopolista.
- Si la entrante ingresa, el monopolista puede acomodarse (A) o luchar (L).

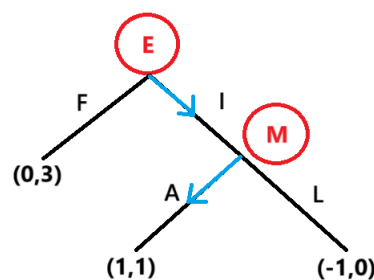


Figura 1: Diagrama de árbol en el que los nodos están conectados por arcos que representan las acciones tomadas por los jugadores. Al final de cada camino (historia particular del juego) están los payoffs de los jugadores representados por una tupla, donde el primer elemento de la tupla representa el payoff del jugador 1 (E), y el segundo elemento representa el payoff del jugador 2 (M). Las flechas azules muestran el camino encontrado mediante inducción hacia atrás.

Los payoffs de los jugadores vienen dados en el diagrama de árbol representado en la Figura 1. Para aplicar inducción hacia atrás, se empieza en el último conjunto de nodos. El último en jugar es el monopolista (M), que debe elegir entre acomodarse (A) con un payoff de 1, o luchar (L) con un payoff de 0, por lo que elige A .

A continuación se avanza hacia el anterior nodo, donde la compañía entrante (E) debe escoger entre mantenerse fuera (F), con un payoff de 0 o ingresar (I) al mercado, con un payoff de 1 (suponiendo que M elige A), por lo que elige I . De esta forma, el camino encontrado mediante inducción hacia atrás viene dado por (I, A) .

Definición 2.11 Se definen los siguientes conceptos:

- Un **conjunto de información** del jugador i es una colección de nodos entre los cuales no puede distinguir.
- En un **juego de información perfecta** cada conjunto de información de cada jugador contiene solo 1 nodo. En caso contrario, es un **juego de información imperfecta**.
- Una estrategia pura del jugador i es un plan completo de acciones. Especifica lo que el jugador i hace en cada nodo de su conjunto de información.
- Un **subjuego** es una parte del juego que parece un juego dentro del árbol. Satisface que:
 1. Empieza desde un único nodo.
 2. Contiene a todos los sucesores de ese nodo.
 3. No rompe ningún conjunto de información.

Ejemplo 2.12 El Dilema del Prisionero es un juego $\mathcal{G} = \langle N, \{S_i\}_{i \in N}, \{u_i\}_{i \in N} \rangle$, donde $N = \{1, 2\}$, $S_1 = \{C, D\}$ y $S_2 = \{c, d\}$, representado matricialmente como:

		Jugador 2	
		c	d
Jugador 1	C	2, 2	-1, 3
	D	3, -1	0, 0

Este juego se puede representar en forma de árbol como se muestra en la Figura 2.

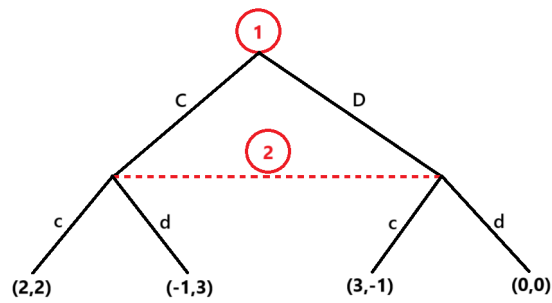


Figura 2: Diagrama de árbol en el que se representa el Dilema del Prisionero, un juego de información imperfecta.

El Dilema del Prisionero es un ejemplo de juego de información imperfecta. En él, el jugador 2 no puede elegir acciones diferentes por separado en sus nodos. O bien juega c en ambos nodos, o bien juega d en ambos.

Definición 2.13 Un equilibrio de Nash $(s_1^*, \dots, s_N^*)$ es un **equilibrio perfecto en subjuegos (SPE o SPNE)** si induce un equilibrio de Nash en cada subjuego del juego.

Ejemplo 2.14 Sea el juego de información perfecta representado en la Figura 3.

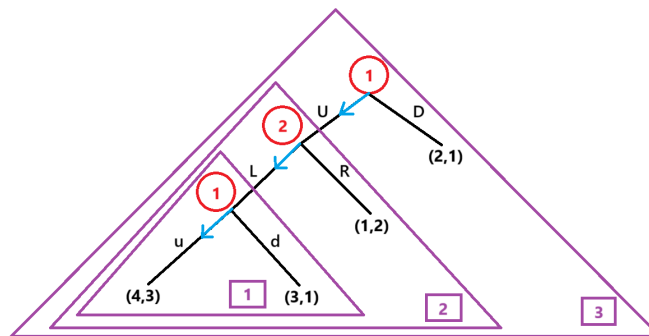


Figura 3: Diagrama de árbol en el que se representa un juego de información perfecta. Las flechas azules señalan el camino encontrado mediante inducción hacia atrás, mientras que los triángulos morados rodean a cada uno de los subjuegos (3 en total) que, además, aparecen numerados en los recuadros morados.

Los conjuntos de estrategias de los dos jugadores del juego vienen dados por:

- $S_1 = \{Uu, Ud, Du, Dd\}$
- $S_2 = \{L, R\}$

Para aplicar inducción hacia atrás, se empieza en el último conjunto de nodos. El último en jugar es el jugador 1, que debe elegir entre las acciones u , con un payoff de 4, o d , con un payoff de 3, por lo que elige u .

A continuación se avanza hacia el nodo anterior, donde el jugador 2 debe elegir entre las acciones L , con un payoff de 3 (suponiendo que en el nodo posterior, el jugador 1 elige u) o R , con un payoff de 2, por lo que elige L .

Por último, se avanza hasta el primer nodo, donde el jugador 1 debe elegir entre las acciones U , con un payoff de 4 (suponiendo que en los nodos siguientes, el jugador 2 elige L , y después el 1 elige u), o D , con un payoff de 2, por lo que elige U . Así, el perfil de estrategias encontrado mediante inducción hacia atrás viene dado por (Uu, L) .

Ahora, para encontrar los equilibrios de Nash del juego en su totalidad, representamos el juego de forma matricial como:

		Jugador 2	
		L	R
Jugador 1	Uu	4, 3	1, 2
	Ud	3, 1	1, 2
	Du	2, 1	2, 1
	Dd	2, 1	2, 1

Entonces:

- $BR_1(L) = \{Uu\}$
- $BR_1(R) = \{Du, Dd\}$
- $BR_2(Uu) = \{L\}$
- $BR_2(Ud) = \{R\}$
- $BR_2(Du) = \{L, R\}$
- $BR_2(Dd) = \{L, R\}$

Por lo que (Uu, L) , (Du, R) y (Dd, R) son equilibrios de Nash.

Ahora, para encontrar los equilibrios perfectos en subjuegos, debemos considerar cada uno de los 3 subjuegos que tiene este juego:

1. El subjuego 1 se puede representar matricialmente como:

		Jugador 2
Jugador 1	u	4, 3
	d	3, 1

Entonces, u es un equilibrio de Nash del subjuego 1.

2. El subjuego 2 se puede representar matricialmente como:

		Jugador 2	
		L	R
Jugador 1	u	4, 3	1, 2
	d	3, 1	1, 2

Entonces, en este subjuego:

- $BR_1(L) = \{u\}$
- $BR_2(u) = \{L\}$
- $BR_1(R) = \{u, d\}$
- $BR_2(d) = \{R\}$

Por lo que (u, L) y (d, R) son equilibrios de Nash del subjuego 2.

3. El subjuego 3 se corresponde con el juego completo, y ya hemos visto que (Uu, L) , (Du, R) y (Dd, R) eran sus equilibrios de Nash.

Finalmente, se tiene que:

- (Uu, L) : induce el equilibrio de Nash u en el subjuego 1, el (u, L) en el subjuego 2 y el (Uu, L) en el subjuego 3.
- (Du, R) : induce el equilibrio de Nash u en el subjuego 1, pero no induce ninguno en el subjuego 2.
- (Dd, R) : no induce ningún equilibrio de Nash en el subjuego 1.

En conclusión, el único equilibrio de Nash del juego que induce un equilibrio de Nash en cada subjuego es (Uu, L) , por lo que este es el único equilibrio perfecto en subjuegos (SPE). Es interesante observar que este coincide con el perfil de estrategias obtenido mediante inducción hacia atrás. De hecho, esto siempre ocurre.

2.2. Algoritmos de toma de decisiones en juegos

La teoría de juegos clásica proporciona una base matemática para analizar decisiones estratégicas entre varios jugadores. Sin embargo, su aplicabilidad práctica en entornos de gran complejidad es limitada. Por ello, en los últimos años se han desarrollado nuevos algoritmos computacionales que permiten abordar problemas estratégicos mediante aproximaciones, que pueden ser especialmente útiles, por ejemplo, en juegos de tablero.

En particular, los juegos combinatorios son especialmente útiles para hacer pruebas con inteligencia artificial, ya que son entornos controlados definidos por reglas simples, pero

que presentan un juego profundo y complejo, lo que plantea desafíos importantes en la investigación. En estos contextos, los juegos se suelen modelar como árboles de decisión.

Algunos métodos tradicionales de búsqueda, como el algoritmo *Minimax*, intentan minimizar la ganancia máxima del oponente en cada estado, deteniéndose prematuramente y utilizando funciones de evaluación para estimar el resultado del juego, además de heurísticas como la heurística $\alpha - \beta$ para reducir el espacio de búsqueda podando el árbol. Variantes como $\max^n$ y *Expectimax* extienden este enfoque a juegos que no son de suma cero o con elementos de azar, respectivamente, pero el tamaño y la complejidad mayores del árbol dificultan su aplicación en juegos de gran escala (Browne et al., 2012).

En este apartado se explican en profundidad algunos algoritmos más avanzados para la toma de decisiones, particularmente útiles en juegos de tablero estratégicos más complejos.

2.2.1. Monte Carlo

Los métodos de Monte Carlo tienen sus raíces en la física estadística, donde se utilizaban para aproximar soluciones a problemas complejos, como algunas integrales intratables. En el contexto de los juegos, estos métodos permiten estimar el valor de un movimiento a través de múltiples simulaciones.

En el enfoque más simple, denominado **Monte Carlo plano (flat Monte Carlo)**, las acciones disponibles desde un estado se eligen de forma uniforme, y para cada una se realizan numerosas simulaciones hasta el final de la partida. Sea $Q(s, a)$ el valor estimado de tomar la acción a en el estado s , esto es, la recompensa esperada de dicha acción. Entonces:

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{N(s)} \mathbb{I}_i(s, a) z_i \quad (1)$$

donde:

- $N(s, a)$ es el número de veces que la acción a ha sido seleccionada desde el estado s .
- $N(s)$ es el número de simulaciones que han pasado por s .
- z_i es el resultado de la i -ésima simulación jugada desde s .
- $\mathbb{I}_i(s, a)$ es una función que vale 1 si en la simulación i se tomó la acción a desde s , y 0 en caso contrario.

Este método ha demostrado ser efectivo en ciertos juegos de información imperfecta como Bridge o Scrabble, donde se ha utilizado logrando un rendimiento competitivo. Sin embargo, debido a su simplicidad, también muestra algunas limitaciones. Debido a que la elección de acciones no se guía por experiencia previa (todas se exploran por igual), puede malgastar recursos en jugadas poco prometedoras y fallar en juegos con estructuras complejas o con oponentes fuertes (Browne et al., 2012).

La implementación del algoritmo de Monte Carlo utilizada en este trabajo se puede encontrar en el anexo B.2.1.

2.2.2. Monte Carlo Tree Search

Debido a las limitaciones de Monte Carlo plano, se han desarrollado algoritmos más avanzados como **Monte Carlo Tree Search (MCTS)**, que combina la idea de la evaluación mediante simulaciones con una estructura de árbol y una política de selección que centra los recursos computacionales en las partes importantes del árbol. Esta política equilibra la **exploración** de los estados que han tenido pocas jugadas, permitiendo mejorar las estimaciones de dichos estados, y la **explotación** de los estados que han dado buenos resultados en jugadas anteriores. Para ello, en cada iteración se hace crecer el árbol en cuatro pasos (Russell and Norvig, 2022):

1. **Selección:** Comenzando en el nodo raíz, se elige un movimiento según la política de selección, alcanzando un nodo hijo, y se repite el proceso sucesivamente, recorriendo el árbol hasta alcanzar un nodo hoja, esto es, un nodo que tiene al menos un hijo potencial al que no se le ha aplicado ninguna simulación todavía.
2. **Expansión:** A partir del nodo hoja seleccionado, crear un nodo hijo, es decir, un movimiento válido desde el nodo hoja, haciendo crecer el árbol.
3. **Simulación:** Se completa una jugada desde el nodo hijo generado, eligiendo movimientos para ambos jugadores de acuerdo a una política. Estos movimientos no se almacenan en el árbol.
4. **Retropropagación:** Se usan los resultados de la simulación para actualizar información en los nodos, recorriéndolos desde el nodo expandido hasta el nodo raíz.

La implementación del algoritmo de Monte Carlo Tree Search utilizada en este trabajo se encuentra en el anexo B.2.2.

2.2.2.1 Políticas de selección

La política de selección es un componente fundamental en el algoritmo MCTS, ya que determina cómo se equilibra la exploración de movimientos poco visitados y la explotación de aquellos que han mostrado buenos resultados en simulaciones anteriores. Existen diversas estrategias para este fin, y en esta sección se explican desde un punto de vista teórico las políticas implementadas en este trabajo.

2.2.2.1.1 ϵ -greedy

El método ϵ -greedy (Sutton and Barto, 2018) es un método simple y efectivo para balancear entre exploración y explotación. Esta estrategia se basa en lo siguiente:

- Con probabilidad $1 - \epsilon$ se selecciona la acción **greedy**, es decir, la acción con mayor recompensa estimada, que es la que hasta el momento parece la mejor acción según las estimaciones (explotación).
- Con probabilidad ϵ se selecciona una acción aleatoria (exploración).

La implementación de la política de selección ϵ -greedy utilizada en este trabajo se puede encontrar en el anexo B.3.1.

2.2.2.1.2 UCB

El método de **límites superiores de confianza** o **Upper Confidence Bound (UCB)** mide el potencial de cada acción con un índice que está formado por la suma de dos términos:

$$ucb(a) = Q(a) + u(a) \quad (2)$$

donde:

- $Q(a)$ es un término de explotación.
- $u(a)$ es un término de exploración (límite superior), una medida de la incertidumbre asociada con la estimación de la recompensa promedio de la acción a . Debe ser más pequeño conforme aumente el número de veces que se selecciona la acción a .

Este método siempre selecciona la acción más codiciosa para maximizar el límite superior de confianza:

$$a^* = \arg \max_{a \in A} uc b(a) \quad (3)$$

Existen varias versiones del algoritmo UCB, entre las cuales destacan las dos siguientes, propuestas por Auer et al. (2002):

- **UCB1:** cuyo valor viene dado por:

$$ucb1(a) = Q(a) + c \cdot \sqrt{\frac{\ln t}{N(a)}} \quad (4)$$

donde:

- t es el número total de simulaciones hasta ahora.
- $N(a)$ es el número de veces que la acción a ha sido seleccionada hasta el momento.
- c es un parámetro de ajuste que controla el grado de exploración.

La implementación de la política de selección UCB1 se encuentra en el anexo B.3.2.

- **UCB2:** cuyo valor viene dado por:

$$ucb2(a) = Q(a) + \sqrt{\frac{(1 + \alpha) \ln \left(\frac{e \cdot t}{\tau(k_a)} \right)}{2\tau(k_a)}} \quad (5)$$

donde:

- t es el número total de simulaciones hasta ahora.
- $0 < \alpha < 1$ es un parámetro de ajuste para el balance entre exploración y explotación.
- k_a es el número de épocas de la acción a .
- $\tau(k_a) = \lceil (1 + \alpha)^{k_a} \rceil$.

A diferencia de UCB1, en UCB2 no se selecciona libremente una acción en cada paso, sino que la selección se estructura por **épocas**: cuando una acción a es seleccionada, esta se ejecuta el número de simulaciones consecutivas dado por $\lceil \tau(k_a + 1) - \tau(k_a) \rceil$. Al terminar dicha época, entonces se vuelve a evaluar cuál es la siguiente acción a seleccionar.

La implementación de la política de selección UCB2 se encuentra en el anexo B.3.3.

2.2.2.1.3 Softmax

El método **Softmax** asigna una probabilidad a cada acción proporcional al valor de la recompensa esperada actual, según la función de Gibbs (softmax), dada por:

$$\pi_t(a) = P(A_t = a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^k e^{Q_t(b)/\tau}} \quad (6)$$

Por lo tanto, las acciones son seleccionadas de acuerdo a la distribución de probabilidad dada por 6. Además, existe una versión adaptativa del método Softmax que ajusta el parámetro τ dinámicamente en cada iteración.

La implementación de la política de selección Softmax se encuentra en el anexo B.3.4, mientras que la implementación de la versión adaptativa del método Softmax se encuentra en el anexo B.3.5.

3. Experimentos y resultados

En esta sección se muestran los experimentos y simulaciones realizados para evaluar el desempeño de los diferentes algoritmos implementados. El código completo que se ha utilizado en este trabajo se encuentra en el anexo B.

Los experimentos se han llevado a cabo utilizando cuatro juegos diferentes, que varían en complejidad y estrategias: Nim, 4 en raya, Othello y Go. Así, en esta sección se analiza el comportamiento de los algoritmos en distintos entornos, desde juegos sencillos con espacio de estados pequeño (Nim), hasta juegos con un espacio de búsqueda mucho más amplio y complejidad estratégica elevada (Go).

A continuación, se muestran los resultados obtenidos para cada juego, discutiendo el rendimiento de los métodos y las particularidades observadas en cada uno de ellos.

3.1. Nim

Nim de una pila es un juego estratégico en el que dos jugadores se turnan para retirar palos de un montón hasta que no quede ninguno.

Definición 3.1 Sean $a_1 < a_2 < \dots < a_k \in \mathbb{N}$ y $n \in \mathbb{N}$. Entonces $NIM(a_1, \dots, a_k; n)$ es el siguiente juego:

- Se comienza con un número n de palos y dos jugadores, Jugador 1 y Jugador 2.

- Los jugadores se alternan, siendo el Jugador 1 quien hace el primer movimiento.
- En cada turno, el jugador correspondiente retira $a \in \{a_1, a_2, \dots, a_k\}$ palos.
- Gana quien tome el último palo (o equivalentemente, pierde el jugador que no pueda mover).

Ejemplo 3.2 *Existen diversos ejemplos de patrones conocidos de victoria y derrota para los juegos de NIM, presentados en Gasarch and Purtilo (2015). Para simplificar la notación en estos ejemplos, se usa la expresión "Jugador I gana" para referirse a que el Jugador I tiene una estrategia ganadora que funcionará sin importar lo que haga el Jugador II. Lo mismo aplica para "Jugador II gana".*

1. Sea $L \geq 1$. El Jugador II gana $NIM(1, \dots, L; n) \iff n \equiv 0 \pmod{L+1}$.
2. Sea $L \geq 2$ y par. El Jugador II gana $NIM(1, L; n) \iff n \equiv 0, 2, 4, \dots, L-2 \pmod{L+1}$.
3. Sea $L \geq 3$. El Jugador II gana $NIM(1, L-1, L; n) \iff n \equiv 0, 2, 4, \dots, L-3 \pmod{2L-2}$.
4. Sea $L \geq 4$. El Jugador II gana $NIM(1, L-1, L; n) \iff n \equiv 0, 2, 4, \dots, L-2 \pmod{2L-1}$.
5. Sea $L \geq 3$. El Jugador II gana $NIM(2, 3, \dots, L; n) \iff n \equiv 0, 1 \pmod{L+2}$.
6. Sea $L \geq 3$. El Jugador II gana $NIM(L-2, L-1, L; n) \iff n \equiv 0, 1, \dots, L-3 \pmod{2L-2}$.

En particular, en este trabajo se usa una versión en la que los números $a_1 < a_2 < \dots < a_k \in \mathbb{N}$ son consecutivos y $a_1 = 1$. Para esta versión del juego, que llamaremos $NIM(1, 2, \dots, k; n)$, con $k, n \in \mathbb{N}$, existe una estrategia ganadora que consiste en dejarle al oponente una cantidad que sea un múltiplo de $k+1$. Esto es, en una pila con $N \equiv 0 \pmod{k+1}$, el jugador que tenga el turno estará en una posición perdedora, mientras que desde una posición tal que $N \not\equiv 0 \pmod{k+1}$, el jugador que tenga el turno podrá jugar de manera que el resultado quede $N \equiv 0 \pmod{k+1}$, dejando una posición perdedora al oponente. En resumen:

- Las posiciones perdedoras son todas aquellas donde $N \equiv 0 \pmod{k+1}$.
- El objetivo es llevar al oponente a una de esas posiciones.

- Si empiezas en una posición perdedora, el oponente puede mantener el control con la misma técnica.

La implementación del juego Nim se encuentra en el anexo B.1.1. En dicho anexo se muestra la clase `NimGame`, encargada de modelar el juego.

Las importaciones necesarias para el código utilizado en esta sección son las siguientes:

```
import sys
import os
import pandas as pd
import random
import numpy as np

# Fijar la semilla
SEED = 123
random.seed(SEED)
np.random.seed(SEED)

# Agregar los directorios al path para poder importar los módulos
sys.path.append(os.path.abspath("../games"))
sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))
sys.path.append(os.path.abspath("../graphviz"))

# Importar la clase NimGame y las funciones de los métodos
from nim import NimGame
from monte_carlo import MCPlay, MCAgent
from monte_carlo_tree_search import MCTS, MCTSPlay, Node, MCTSAgent
from epsilon_greedy import EpsilonGreedy
from softmax import Softmax
from adaptive_softmax import AdaptiveSoftmax
from ucb1 import UCB1
from ucb2 import UCB2
from gradiente_preferencias import GradienteDePreferencias
from MCTS_graphviz import generate_tree_MCTS
```

3.1.1. Simulaciones humano vs Monte Carlo

En este apartado, se muestran diversas simulaciones en las que el jugador humano se enfrenta a un agente de Monte Carlo. El objetivo principal de estas simulaciones es observar el desempeño del frente a un jugador que intenta aplicar la estrategia teóricamente ganadora explicada para $NIM(1, 2, \dots, k; n)$, con $k, n \in \mathbb{N}$. Además, se podrá observar hasta qué punto el agente de Monte Carlo es capaz de replicar este patrón estratégico, y cómo se comporta frente a un jugador humano que los aplica conscientemente.

Para llevar a cabo las simulaciones entre el jugador humano y el agente basado en Monte Carlo, se ha implementado un entorno interactivo en el que el jugador realiza sus

movimientos manualmente a través de la consola, mientras que el agente utiliza simulaciones aleatorias para tomar decisiones.

En cada turno del agente, se ejecutan un número determinado de simulaciones (en este caso, 1000) para estimar cuál de las jugadas disponibles tiene mayor probabilidad de llevar a una victoria. El juego se desarrolla en la versión $NIM(1, 2, 3, 4; 21)$, es decir, una única pila con 21 palos y un conjunto de movimientos posibles que permite retirar entre 1 y 4 palos en cada turno. El jugador humano comienza la partida. En el siguiente fragmento de código se muestra la lógica utilizada en estas simulaciones:

```
# Crear una instancia del juego Nim
game = NimGame(initial_sticks=21, max_sticks_per_turn=4, starting_player=1)

# Mostrar el estado inicial del juego
game.draw()

# Definir el número de simulaciones por movimiento
num_simulations = 1000

# Bucle del juego hasta que termine
while not game.terminal():
    if game.turn == 1: # Turno del jugador humano
        print("\nTu turno...")
        jugada = int(input("Cuántos palos quieres retirar? "))

        if jugada in game.valid_actions():
            game = game.action(jugada)
        else:
            print("Jugada inválida. Inténtalo de nuevo.")
            continue # Volver a pedir jugada

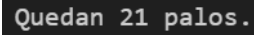
    else: # Turno de la IA (Monte-Carlo)
        print("\nTurno de la computadora...")
        game = MCPlay(game, num_simulations)

    game.draw() # Mostrar el estado después del turno

# Anunciar el ganador
if game.winner() == 1:
    print("\nGanas la partida!")
else:
    print("\nLa computadora gana. Inténtalo de nuevo!")
```

En primer lugar, se presentan los resultados de una partida en la que el jugador humano lleva a cabo la estrategia consistente en llevar al oponente (el agente de Monte Carlo) a posiciones perdedoras, es decir, aquellas en las que $N \equiv 0 \pmod{k+1}$. En este caso particular, serán aquellas en las que $N \equiv 0 \pmod{5}$, donde $N \in \mathbb{N}$ es el número de palos que quedan en la pila. Equivalentemente, en esta versión de Nim, donde $k = 4$, la condición para una posición perdedora es que el número de palos en la pila sea múltiplo

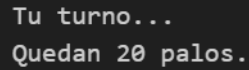
de $k + 1 = 5$.



```
Quedan 21 palos.
```

Figura 4: Estado inicial de la primera partida de Nim con Humano vs Monte Carlo.

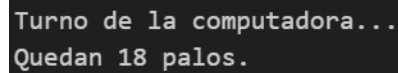
La partida comienza inicialmente con 21 palos, tal como se muestra en la Figura 4.



```
Tu turno...
Quedan 20 palos.
```

Figura 5: Primer movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.

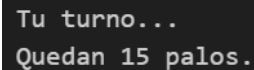
En la Figura 5, se muestra como el jugador humano retira 1 palo de la pila, dejando 20 palos en la misma, que es múltiplo de 5. De esta forma, está aplicando la estrategia explicada anteriormente.



```
Turno de la computadora...
Quedan 18 palos.
```

Figura 6: Segundo movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.

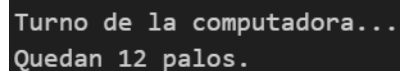
A continuación, el agente de Monte Carlo retira 2 palos, quedando 18 en la pila, tal como se muestra en la Figura 6.



```
Tu turno...
Quedan 15 palos.
```

Figura 7: Tercer movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.

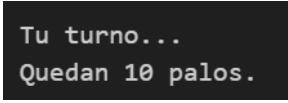
En la Figura 7, se muestra como el jugador humano retira 3 palos, quedando 15 palos que, de nuevo, es múltiplo de 5, aplicando la estrategia explicada por segunda vez.



```
Turno de la computadora...
Quedan 12 palos.
```

Figura 8: Cuarto movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.

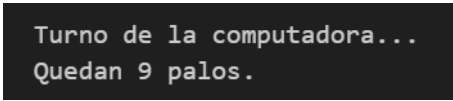
A continuación, el agente de Monte Carlo retira 3 palos, quedando 12 palos en la pila, como se aprecia en la Figura 8.



```
Tu turno...  
Quedan 10 palos.
```

Figura 9: Quinto movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.

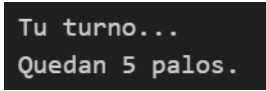
De nuevo, en la Figura 9, el jugador humano vuelve a aplicar la estrategia retirando 2 palos, y quedando 10 palos en la pila, que es múltiplo de 5.



```
Turno de la computadora...  
Quedan 9 palos.
```

Figura 10: Sexto movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.

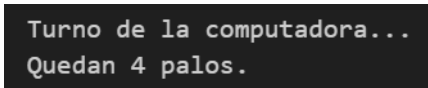
El agente de Monte Carlo retira 1 palo, quedando 9 en la pila, como se observa en la Figura 10.



```
Tu turno...  
Quedan 5 palos.
```

Figura 11: Séptimo movimiento de la primera partida de Nim, llevado a cabo por el jugador humano.

El jugador humano nuevamente aplica la estrategia de dejar un número de palos en la pila que sea múltiplo de 5. En la Figura 11 se observa que retira 4 palos, dejando 5 en la pila.

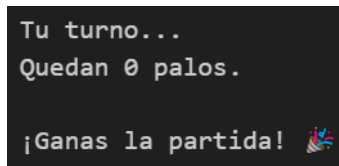


```
Turno de la computadora...  
Quedan 4 palos.
```

Figura 12: Octavo movimiento de la primera partida de Nim, llevado a cabo por el agente de Monte Carlo.

En la Figura 12 se muestra el penúltimo movimiento de la partida. En la pila quedan 5 palos, y le toca el turno a la computadora, que solo puede retirar 1, 2, 3 o 4 palos.

Cualquiera de los movimientos que haga va a dejar al jugador humano en una posición de victoria asegurada, gracias a la estrategia utilizada por el mismo: si el agente de Monte Carlo retira 1 palo, dejaría 4 palos en la pila; si retira 2, dejaría 3 en la pila; si retira 3, dejaría 2 en la pila; y si retira 4, dejaría 1 palo en la pila. Y cualquiera de esos estados es un movimiento posible para el jugador humano, permitiéndole dejar la pila con 0 palos. En particular, el agente de Monte Carlo retira 1 palo, quedando 4 palos en la pila para el siguiente turno.



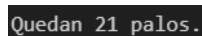
```
Tu turno...
Quedan 0 palos.
¡Ganas la partida! 🎉
```

Figura 13: Último movimiento de la primera partida de Nim, llevado a cabo por el jugador humano, ganando la partida.

Finalmente, en la Figura 13 se muestra el último movimiento de la partida, realizado por el jugador humano. Este jugador retira los 4 palos que quedan en la pila y gana la partida, pues ya no quedan más palos en la misma. La estrategia explicada en la introducción de este juego le ha asegurado la victoria al jugador humano.

A continuación, se muestra una segunda simulación, en la que el jugador humano no aplica la estrategia para ganar. De esta forma, se puede analizar cómo juega el agente de Monte Carlo en estas condiciones, y si es capaz de encontrar dicha estrategia por sí mismo y aplicarla para ganar la partida. Además, en este caso, el agente de Monte Carlo va a ser el jugador que comience la partida. Para ello, al crear la instancia del juego, se configura con `starting_player=-1`, es decir, se usa:

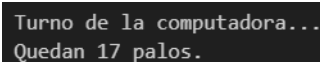
```
# Crear una instancia del juego Nim
game = NimGame(initial_sticks=21, max_sticks_per_turn=4, starting_player=-1)
```



```
Quedan 21 palos.
```

Figura 14: Estado inicial de la segunda partida de Nim con Humano vs Monte Carlo.

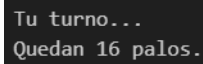
La partida comienza inicialmente con 21 palos, tal como se muestra en la Figura 14.



```
Turno de la computadora...
Quedan 17 palos.
```

Figura 15: Primer movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.

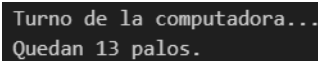
En la Figura 15, se muestra el primer movimiento, realizado por el agente de Monte Carlo, que retira 4 palos de la pila, dejando 17 palos en la misma, que no es múltiplo de 5. Esto es, desde su primer turno, el agente no aplica la estrategia óptima de llevar al oponente a una posición perdedora (aquella donde $N \equiv 0 \pmod{5}$). Esto deja abierta la posibilidad de que el jugador humano aproveche la situación en el siguiente turno y comience a aplicar dicha estrategia.



```
Tu turno...  
Quedan 16 palos.
```

Figura 16: Segundo movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.

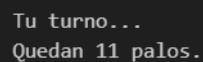
A continuación, tiene el turno el jugador humano. Como hay 17 palos en la pila, podría retirar 2 palos para que queden 15, dejando al agente de Monte Carlo en una posición perdedora. Sin embargo, como se pretende estudiar el funcionamiento de dicho agente cuando su oponente no utiliza la estrategia óptima, el jugador humano no usa la estrategia en este turno. En su lugar, retira 1 palo, dejando 16 en la pila, tal como se puede observar en la Figura 16.



```
Turno de la computadora...  
Quedan 13 palos.
```

Figura 17: Tercer movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.

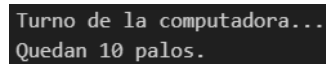
En la Figura 17, se muestra como el agente de Monte Carlo retira 3 palos, quedando 13 palos que, de nuevo, no es múltiplo de 5. Es decir, en su segundo turno tampoco realiza la estrategia para ganar la partida, permitiendo que su oponente pueda comenzar a realizarla en el siguiente turno. Esto puede deberse a que el número de simulaciones (1000 en este caso) es demasiado pequeño como para recorrer todo el espacio de búsqueda cuando todavía quedan demasiados palos en la pila. Esto dificulta que el agente identifique correctamente la jugada óptima.



```
Tu turno...  
Quedan 11 palos.
```

Figura 18: Cuarto movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.

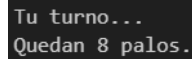
A continuación, el jugador humano vuelve a tener la oportunidad de comenzar a aplicar la estrategia para dejar a su oponente en una posición perdedora. Pero para poder estudiar correctamente el funcionamiento del agente de Monte Carlo, vuelve a no hacerlo. De esta forma, retira 2 palos, quedando 11 palos en la pila, como se aprecia en la Figura 18.



```
Turno de la computadora...  
Quedan 10 palos.
```

Figura 19: Quinto movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.

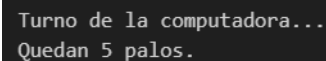
En la Figura 19 se muestra el tercer movimiento del agente de Monte Carlo, en el cual comienza finalmente a aplicar la estrategia óptima. En este turno, retira 1 palo, dejando 10 palos en la pila, una cantidad que es múltiplo de 5, y por tanto una posición perdedora para el oponente. Esto sugiere que, aunque el agente de Monte Carlo no identifica la estrategia ganadora en los primeros movimientos de la partida, sí logra descubrirla conforme el juego avanza y se reduce el espacio de búsqueda. A medida que el número de palos en la pila disminuye, las simulaciones reflejan de una forma más precisa la estructura estratégica del juego, permitiendo que el agente tome decisiones más acertadas en los turnos finales.



```
Tu turno...  
Quedan 8 palos.
```

Figura 20: Sexto movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.

Dado que se encuentra en una posición perdedora, el jugador humano ya no puede aplicar la estrategia óptima en este turno. En la Figura 20 se muestra como el jugador humano retira 2 palos, dejando 8 en la pila.

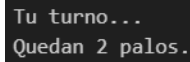


```
Turno de la computadora...  
Quedan 5 palos.
```

Figura 21: Séptimo movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo.

El agente de Monte Carlo nuevamente aplica la estrategia ganadora en su cuarto turno. En la Figura 21 se observa que retira 3 palos, dejando 5 en la pila. Esto refuerza la idea de que, conforme se acerca el final de la partida, las simulaciones reflejan de forma más

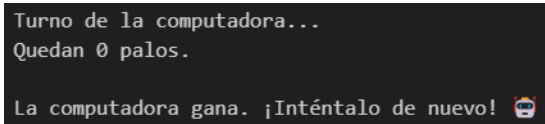
correcta las mejores jugadas que puede realizar el agente, al haber un espacio de búsqueda más pequeño.



```
Tu turno...  
Quedan 2 palos.
```

Figura 22: Octavo movimiento de la segunda partida de Nim, llevado a cabo por el jugador humano.

En la Figura 22 se muestra el penúltimo movimiento de la partida. En la pila quedan 5 palos, y le toca el turno al jugador humano, que solo puede retirar 1, 2, 3 o 4 palos. Cualquiera de los movimientos que haga va a dejar al agente de Monte Carlo en una posición de victoria asegurada, gracias a la estrategia utilizada por el mismo en sus turnos anteriores. En particular, el jugador humano retira 3 palos, quedando 2 palos en la pila para el siguiente turno.



```
Turno de la computadora...  
Quedan 0 palos.  
  
La computadora gana. ¡Inténtalo de nuevo! 🤖
```

Figura 23: Último movimiento de la segunda partida de Nim, llevado a cabo por el agente de Monte Carlo, ganando la partida.

Finalmente, en la Figura 23 se muestra el último movimiento de la partida, realizado por el agente de Monte Carlo. Este retira los 2 palos restantes y gana la partida, ya que la pila se queda a 0. La estrategia aplicada en sus anteriores movimientos le ha asegurado la victoria al agente.

3.1.2. Herramienta de depuración para Monte Carlo Tree Search

En este apartado, para comprender el funcionamiento de los agentes de MCTS, se analizan posiciones concretas de una partida del juego Nim gracias a la herramienta de depuración cuya implementación se puede encontrar en el anexo B.4. Esta herramienta utiliza `graphviz` para visualizar y analizar el comportamiento del método de MCTS al generar un diagrama de árbol con las posiciones siguientes solicitadas a partir de un determinado nodo inicial.

Cada nodo del árbol representa un estado del juego, el cual se alcanza mediante una secuencia de movimientos, y contiene información como el turno del jugador, el número

de visitas y la recompensa de cada jugador. Las aristas indican las acciones tomadas para pasar de un estado al siguiente. Se puede especificar la profundidad máxima del árbol generado. Además, los nodos en el árbol aparecen identificados mediante un ID. Para poder visualizar cuál es el estado de cada nodo, la función `generate_tree_MCTS` devuelve un diccionario que asocia cada ID al nodo correspondiente.

Veamos un ejemplo sencillo para comprobar y analizar el funcionamiento del agente de MCTS, particularmente usando el método de selección ϵ -greedy con $\epsilon = 0.2$ (se explora un 20 % de las veces, se explota un 80 % de las veces), comenzando la partida con 5 palos y permitiendo retirar un máximo de 3 palos por turno. Se genera un árbol con una profundidad máxima de 2:

```
# Crear estado inicial del juego
initial_state = NimGame(initial_sticks=5, max_sticks_per_turn=3, starting_player=1)

# Visualizar el árbol de MCTS
root_node = Node(initial_state, None)
mcts = MCTS(root_node, EpsilonGreedy, simulations=1000, epsilon = 0.2)
mcts.run()
id_to_node = generate_tree_MCTS(root_node, max_depth=2, filename="deputation_trees/
    arbol_nim", format="jpg")
```

El árbol resultante es:

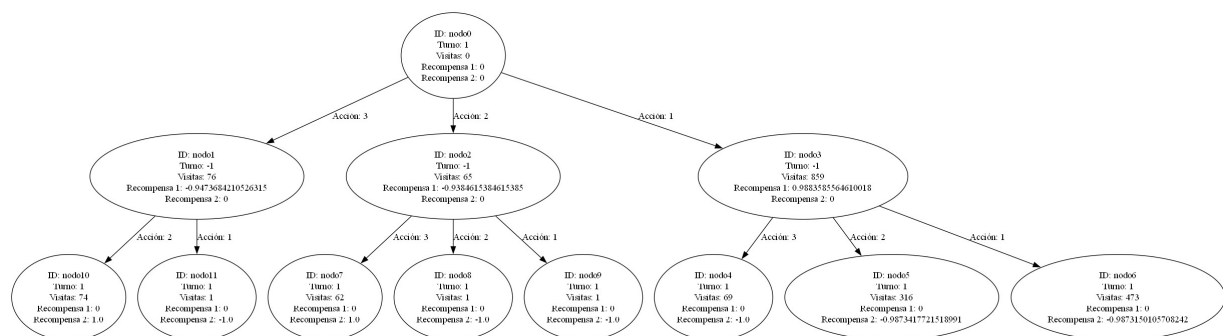


Figura 24: Diagrama de árbol de profundidad 2 de la partida de Nim inicializada con 5 palos y permitiendo retirar un máximo de 3 palos por turno, con MCTS con método de selección ϵ -greedy con $\epsilon = 0.2$.

En la Figura 24 se observa que, en el primer nivel, el nodo más visitado, con 859 visitas, es `nodo3`, el cual se alcanza cuando el jugador 1 realiza la acción de retirar 1 palo del montón, como se aprecia en la arista correspondiente, quedando 4 palos en el mismo. Esto se puede comprobar gracias al diccionario que asocia cada ID de nodo al nodo correspondiente, mediante:

```
node = id_to_node['nodo3']
```

```
node.state.draw()
```

En la Figura 25 se muestra el estado de dicho nodo:

Quedan 4 palos.

Figura 25: Estado del nodo con ID `nodo3`.

Este nodo muestra una recompensa para el jugador 1 muy cercana a 1, mientras que los otros dos nodos tienen recompensas muy cercanas a -1 . Esto es lógico, pues en `nodo3`, al quedar 4 nodos, se cumple la estrategia ganadora $N \equiv 0 \pmod{k+1}$, o equivalentemente $4 \equiv 0 \pmod{4}$, mientras que en los otros dos nodos no se cumple.

Por otro lado, en el segundo nivel, de nuevo se observa que los nodos con mayor recompensa para el jugador 2 son los más visitados. Por ejemplo, partiendo de `nodo3`, hay 3 nodos hijos, de los cuales los más visitado, con 316 y 473 visitas respectivamente, son `nodo5` y `nodo6` que tiene una recompensa muy similar entre sí, pero ligeramente superior a la de `nodo4`. Partiendo de `nodo2`, que tiene 3 nodos hijos, el más visitado es `nodo7`, que tiene una recompensa de 1, precisamente porque en este nodo el jugador 2 gana la partida al retirar 3 palos de un montón al que le quedaban 3 palos. Similarmente, en `nodo1`, con 2 nodos hijos, el más visitado es `nodo10`, que tiene una recompensa de 1 ya que en este nodo el jugador 2 gana la partida, al retirar 2 palos de un montón al que le quedaban 2 palos.

3.1.3. Simulaciones comparativas entre varios agentes

En esta sección, se van a comparar los diferentes agentes implementados, Monte Carlo y MCTS, realizando simulaciones en las que competirán entre ellos por pares en el juego Nim. Para ello, se necesitan las siguientes funciones:

```
def play_comparative_game(agent1, agent2, initial_sticks=21, max_sticks_per_turn=4,
    starting_player=1):
    game = NimGame(initial_sticks=initial_sticks, max_sticks_per_turn=max_sticks_per_turn,
        starting_player=starting_player)
    agents = {1: agent1, -1: agent2}

    while not game.terminal():
        game = agents[game.turn].move(game)

    return game.winner()

def run_simulations(agent_1, agent_2, initial_sticks=21, max_sticks_per_turn=4, n_games
    =100):
    results = []
```

```

for i in range(n_games):
    # Alternar jugador inicial
    starting_player = 1 if i % 2 == 0 else -1

    # Asignar agentes según quién empieza
    if starting_player == 1:
        winner = play_comparative_game(agent_1, agent_2, initial_sticks,
max_sticks_per_turn, starting_player)
    else:
        winner = play_comparative_game(agent_2, agent_1, initial_sticks,
max_sticks_per_turn, starting_player)
        # Invertir perspectiva
        winner *= -1

    results.append(winner)

df = pd.DataFrame(results, columns=["winner"])
win_rate = (df["winner"] == 1).mean()
print(f"% de partidas ganadas por el agente 1: {win_rate:.2%} ({df['winner'].
value_counts().to_dict()})")
return df

```

Los agentes que se van a usar para realizar los enfrentamientos se definen a continuación:

```

agent_mc = MCAGENT(1000)
agent_mcts_eps = MCTSAGENT(1000, EpsilonGreedy, epsilon = 0.2)
agent_mcts_UCB1 = MCTSAGENT(1000, UCB1, c=1)
agent_mcts_UCB2 = MCTSAGENT(1000, UCB2, alpha=0.5)
agent_mcts_soft = MCTSAGENT(1000, Softmax, tau = 1)
agent_mcts_adapsoft = MCTSAGENT(1000, AdaptiveSoftmax, tau_0 = 1, alpha = 0.5)

```

Por ejemplo, para enfrentar al agente de Monte Carlo frente al agente de MCTS con ϵ -greedy, se hace:

```

df_mc_mcts_eps = run_simulations(agent_mc, agent_mcts_eps, initial_sticks=21,
max_sticks_per_turn=4, n_games=100)

```

El resultado obtenido para este caso particular se puede observar en la Figura 26:

```
% de partidas ganadas por el agente 1: 44.00% ({-1: 56, 1: 44})
```

Figura 26: Resultado de enfrentar al agente de Monte Carlo frente al agente de MCTS con ϵ -greedy en 100 partidas de Nim

Los resultados de los enfrentamientos por pares se han recopilado en la tabla 1:

Agente A	Agente B	Ganó A	Ganó B	Runtimerow
MC	MCTS ϵ -greedy	44	56	3m 23.8s
MC	MCTS UCB1	32	68	3m 46.9s
MC	MCTS UCB2	84	16	4m 3.2s
MC	MCTS Softmax	71	29	4m 21.2s
MC	MCTS Softmax Adaptativo	66	34	4m 10.6s
MCTS ϵ -greedy	MCTS UCB1	27	73	56.6s
MCTS ϵ -greedy	MCTS UCB2	89	11	1m 23.5s
MCTS ϵ -greedy	MCTS Softmax	65	35	1m 36.9s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	66	34	1m 58.0s
MCTS UCB1	MCTS UCB2	89	11	1m 27.2s
MCTS UCB1	MCTS Softmax	83	17	1m 40.4s
MCTS UCB1	MCTS Softmax Adaptativo	87	13	2m 0.9s
MCTS UCB2	MCTS Softmax	30	70	2m 11.6s
MCTS UCB2	MCTS Softmax Adaptativo	17	83	2m 47.3s
MCTS Softmax	MCTS Softmax Adaptativo	50	50	2m 35.0s

Tabla 1: Resultados de enfrentamientos por pares entre agentes en el juego Nim (100 partidas por par).

3.2. 4 en raya

4 en raya es un juego de estrategia para dos jugadores que consiste en alinear cuatro fichas del mismo color en un tablero vertical. Sus reglas consisten en:

- El juego clásico se juega en un tablero de 7 columnas por 6 filas (aunque en esta implementación pueden tomar valores entre 4 y 10).
- Los jugadores se turnan para dejar caer una ficha en una de las columnas.
- La ficha cae hasta ocupar la posición más baja disponible en esa columna.
- El objetivo es alinear cuatro fichas consecutivas del mismo color (en línea horizontal, vertical o diagonal).
- Gana el primer jugador que consiga alinear cuatro fichas.

La implementación del juego 4 en raya se muestra en el anexo B.1.2. En dicho anexo se muestra la clase `Connect4Game`, encargada de modelar el juego.

Las importaciones necesarias para el código utilizado en esta sección son las siguientes:

```
import sys
import os
import pandas as pd
import random
import numpy as np

# Fijar la semilla
SEED = 123
random.seed(SEED)
np.random.seed(SEED)

# Agregar los directorios al path para poder importar los módulos
sys.path.append(os.path.abspath("../games"))
sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))
sys.path.append(os.path.abspath("../graphviz"))

# Importar la clase Connect4Game y las funciones de los métodos
from connect4 import Connect4Game
from monte_carlo import MCPlay, MCAgent
from monte_carlo_tree_search import MCTS, MCTSPlay, Node, MCTSAgent
from epsilon_greedy import EpsilonGreedy
from softmax import Softmax
from adaptive_softmax import AdaptiveSoftmax
from ucb1 import UCB1
from ucb2 import UCB2
from gradiente_preferencias import GradienteDePreferencias
from MCTS_graphviz import generate_tree_MCTS
```

3.2.1. Simulaciones humano vs Monte Carlo

En este apartado, se muestran simulaciones en las que el jugador humano se enfrenta a un agente de Monte Carlo. El objetivo principal de estas simulaciones es observar el desempeño de dicho agente frente a un jugador humano en el juego 4 en raya.

En cada turno del agente, se ejecutan un número determinado de simulaciones (en este caso, 1000) para estimar cuál de las jugadas disponibles tiene mayor probabilidad de llevar a una victoria. El juego se desarrolla en un tablero de 6 filas y 7 columnas. El jugador humano comienza la partida. En el siguiente fragmento de código se muestra la lógica utilizada en estas simulaciones:

```
# Crear una instancia del juego 4 en raya
game = Connect4Game(rows=6, cols=7, starting_player=1)

# Mostrar el estado inicial del juego
game.draw()

# Definir el número de simulaciones por movimiento
num_simulations = 1000
```



```

# Bucle del juego hasta que termine
while not game.terminal():
    if game.turn == 1: # Turno del jugador humano
        print("\nTu turno...")
        jugada = int(input(f"En qué columna quieres colocar tu ficha (0-{game.cols-1})?"))
    )

    if jugada in game.valid_actions():
        game = game.action(jugada)
    else:
        print("Jugada inválida. Inténtalo de nuevo.")
        continue # Volver a pedir jugada

    else: # Turno de la IA (Monte-Carlo)
        print("\nTurno de la computadora...")
        game = MCPlay(game, num_simulations)

    game.draw() # Mostrar el estado después del turno

# Anunciar el ganador
if game.winner() == 1:
    print("\nGanas la partida!")
elif game.winner() == -1:
    print("\nLa computadora gana. Inténtalo de nuevo!")
else:
    print("\nEs un empate!")

```

```

[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]

```

Figura 27: Estado inicial de la partida de 4 en raya con Humano vs Monte Carlo.

La partida comienza inicialmente con el tablero vacío (todas las celdas a 0), tal como se muestra en la Figura 27.

<pre> Tu turno... [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 1, 0, 0, 0] </pre>	<pre> Turno de la computadora... [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 2, 0, 0, 0] [0, 0, 0, 1, 0, 0, 0] </pre>	<pre> Tu turno... [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 2, 0, 0, 0] [0, 0, 0, 1, 1, 0, 0] </pre>	<pre> Turno de la computadora... [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 0, 0, 0, 0] [0, 0, 0, 2, 0, 0, 0] [0, 0, 2, 1, 1, 0, 0] </pre>
(a) 1 ^{er} movimiento	(b) 2 ^o movimiento	(c) 3 ^{er} movimiento	(d) 4 ^o movimiento

Figura 28: Movimientos del 1^o al 4^o de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 28 se muestran los 4 primeros movimientos de la partida. Las fichas del jugador humano vienen representadas por un 1, mientras que las fichas del agente de Monte Carlo vienen dadas por un 2. Comienza la partida el jugador humano, que coloca su ficha en la columna 3 (las columnas van enumeradas de 0 a 6), tal como se muestra en la Figura 28a. Esto es una ventaja inicial, pues tirar la primera ficha en la columna central le permite formar más combinaciones posibles en horizontal, vertical y diagonal. A continuación, tal como se muestra en la Figura 28b, el agente de Monte Carlo lanza también en la columna 3, que es la columna central, lo cual tiene sentido pues de esa forma puede neutralizar parcialmente el control del centro del jugador humano y reducir su ventaja. En la Figura 28c, el jugador humano lanza en la columna 4 para comenzar a alinear sus fichas en horizontal. Además, en el cuarto movimiento de la partida, en la Figura 28d se observa que el agente de Monte Carlo comienza a alinear sus fichas en diagonal, lanzando en la columna 2, lo cual a su vez le permite bloquear al jugador humano en horizontal por el lado izquierdo.

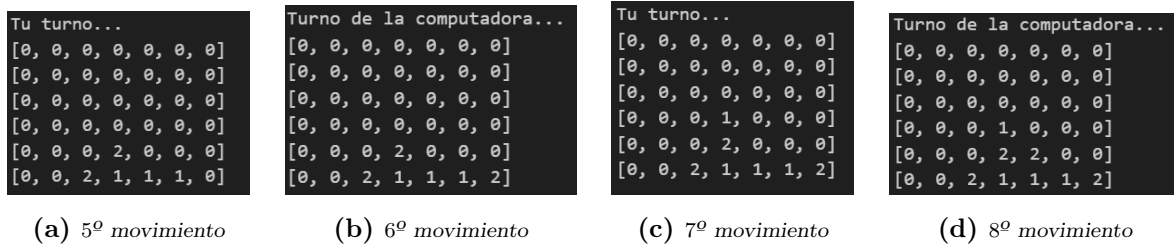


Figura 29: Movimientos del 5º al 8º de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 29 se muestran los 4 siguientes movimientos de la partida. En la Figura 29a, el jugador humano coloca una ficha en la columna 5, alineando 3 de sus fichas en horizontal. Sin embargo, en el siguiente turno, el agente de Monte Carlo le bloquea, lanzando en la columna 6, para que el jugador humano no pueda alinear una cuarta ficha y ganar, tal como se observa en la Figura 29b. A continuación, el jugador humano lanza en la columna 3 con la finalidad de alinear sus fichas en diagonal, como se muestra en la Figura 29c. Sin embargo, el agente de Monte Carlo bloquea dicha diagonal al lanzar en la columna 4, lo cual también le permite comenzar una hilera de sus fichas en horizontal, como se observa en la Figura 29d.

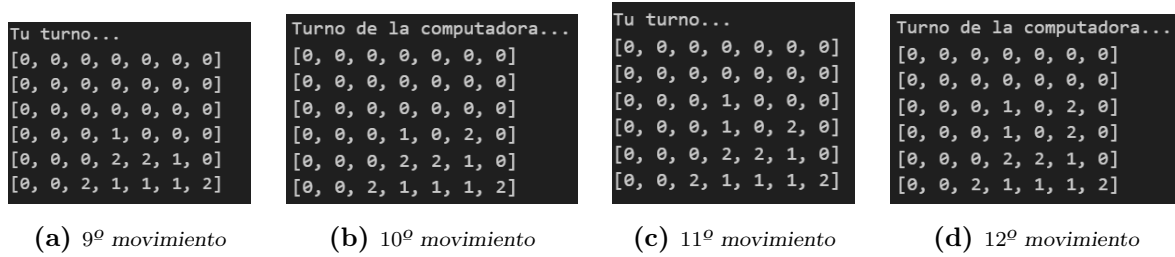


Figura 30: Movimientos del 9º al 12º de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 30 se muestran los 4 siguientes movimientos de la partida. En la Figura 30a, el jugador humano coloca una ficha en la columna 5, alineando dos de sus fichas en vertical y otras dos en diagonal, y bloqueando la hilera horizontal del agente de Monte Carlo. A continuación, en la Figura 30b, el agente de Monte Carlo lanza en la columna 5, bloqueando la hilera vertical del jugador humano, y alineando dos de sus fichas en diagonal. En la Figura 30c el jugador humano lanza en la columna 3, alineando dos de sus fichas en vertical. Seguidamente, en la Figura 30d, el agente de Monte Carlo lanza en la columna 5, lo cual le permite alinear 3 de sus fichas en una misma hilera diagonal, y dos de sus fichas en una hilera vertical.

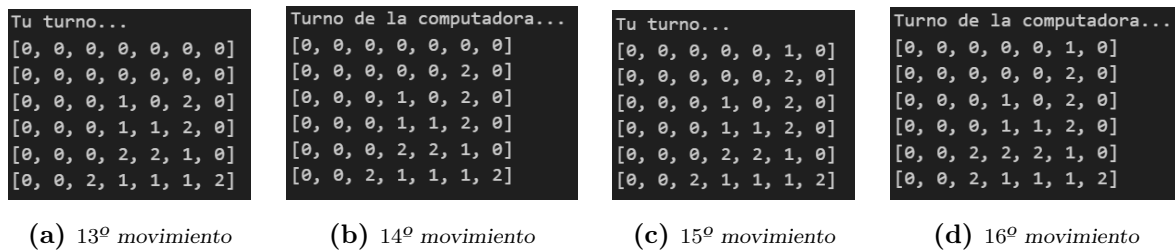


Figura 31: Movimientos del 13º al 16º de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 31 se muestran los 4 siguientes movimientos de la partida. En la Figura 31a, el jugador humano coloca una ficha en la columna 4 para bloquear la hilera diagonal formada por 3 fichas del agente de Monte Carlo. Además, esto le permite alinear 3 de sus fichas en diagonal. A continuación, el agente de Monte Carlo lanza en la columna 6, creando una fila vertical con 3 de sus fichas alineadas, como se observa en la Figura 31b. Esta fila es bloqueada por el jugador humano al lanzar en la columna 6, tal como se muestra en la Figura 31c. A continuación, en la Figura 31d el agente de Monte Carlo lanza en la columna 2, lo cual le permite construir una hilera de 3 fichas en horizontal, y una de 2 fichas en vertical.

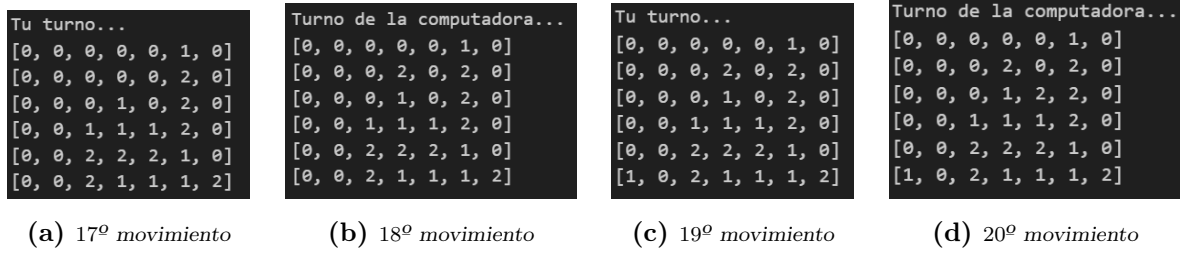


Figura 32: Movimientos del 17^o al 20^o de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 32 se muestran los 4 siguientes movimientos de la partida. En la Figura 32a, el jugador humano coloca una ficha en la columna 2, creando una fila horizontal y otra diagonal de 3 de sus fichas, y bloqueando la hilera vertical de 2 fichas del agente de Monte Carlo. Después, el agente de Monte Carlo lanza en la columna 3 para bloquear una fila vertical del jugador humano, como se muestra en la Figura 32b. En la Figura 32c, el jugador humano lanza en la columna 0, creando una hilera diagonal con 3 de sus fichas. A continuación, el agente de Monte Carlo lanza en la columna 4, lo cual le permite crear una hilera diagonal con 3 de sus fichas, y otra horizontal con 2, como se observa en la Figura 32d. Es interesante observar que el agente de Monte Carlo tiene una hilera de 3 fichas horizontales consecutivas, y para poder añadir una ficha más a esa fila, debería primero lanzar una ficha en la columna 1, y otra en el siguiente turno también en la ficha 1. Sin embargo, no hace eso porque, de hacerlo, el jugador humano podría lanzar en la columna 1 en su turno y ganar, pues este también tiene una hilera de 3 fichas consecutivas, en este caso diagonales.

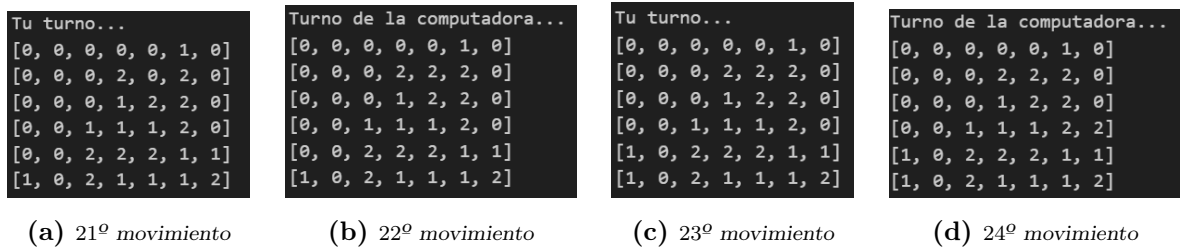


Figura 33: Movimientos del 21^o al 24^o de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 33 se muestran los 4 siguientes movimientos de la partida. En la Figura 33a, el jugador humano coloca una ficha en la columna 6 para bloquear la hilera diagonal de 3 fichas del agente de Monte Carlo. Seguidamente, el agente de Monte Carlo lanza en la columna 4, formando una hilera horizontal de 3 fichas y una diagonal de 2, como se aprecia en la Figura 33b. Después, en la Figura 33c el jugador humano lanza en la

columna 0 para alinear dos de sus fichas verticalmente. A continuación, en la Figura 33d, el agente de Monte Carlo lanza en la columna 6, lo cual le permite alinear 3 de sus fichas diagonalmente.

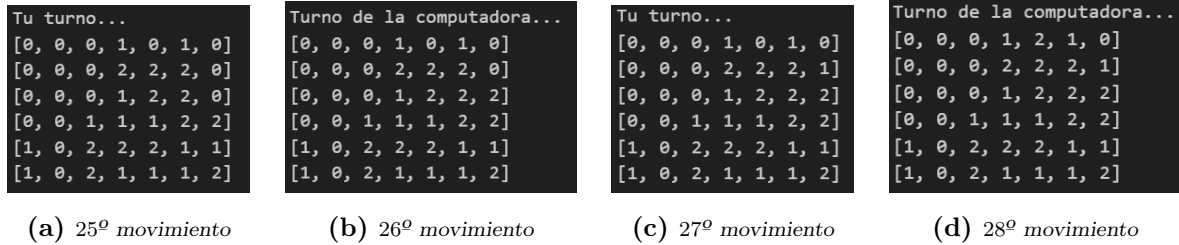


Figura 34: Movimientos del 25º al 28º de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 34 se muestran los 4 siguientes movimientos de la partida. En la Figura 34a, el jugador humano coloca una ficha en la columna 3 para bloquear la hilera diagonal de 3 fichas del agente. Después, en la Figura 34b, el agente de Monte Carlo lanza en la columna 6, lo cual le permite alinear dos de sus fichas verticalmente, además de rellenar una casilla más en dicha columna, permitiéndole en el siguiente turno lanzar en esa misma columna y ganar (si el oponente no le bloquea antes), pues a esa altura se encuentra una hilera horizontal con 3 de sus fichas. No obstante, en la Figura 34c, el jugador humano bloquea dicha casilla al lanzar en la columna 6. Después, en la Figura 34d, el agente de Monte Carlo lanza en la columna 4, alineando 2 de sus fichas de forma diagonal, y bloqueando una hilera vertical con 2 fichas del oponente.

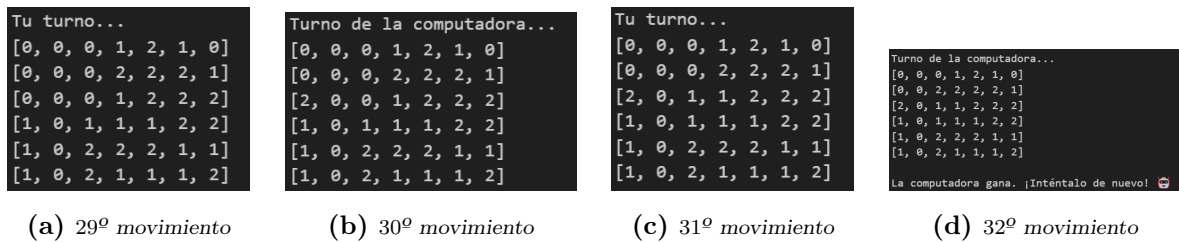


Figura 35: Movimientos del 29º al 32º de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 35 se muestran los 4 siguientes y últimos movimientos de la partida. En la Figura 35a, el jugador humano coloca una ficha en la columna 0, alineando 3 de sus fichas verticalmente. Sin embargo, en el siguiente turno, el agente de Monte Carlo también lanza en la columna 0, bloqueando dicha hilera, como se muestra en la Figura 35b. Para no alargar más la partida, y dado que ya se ha comprobado el correcto funcionamiento del agente de Monte Carlo en el juego 4 en raya, en la Figura 35c el jugador humano lanza

en la columna 2, lo cual deja al agente a un solo movimiento de ganar. Efectivamente, en la Figura 35d, el agente de Monte Carlo aprovecha dicha oportunidad y lanza en la columna 2, creando una hilera de 4 fichas horizontales consecutivas y, por tanto, ganando la partida.

3.2.2. Herramienta de depuración para Monte Carlo Tree Search

En este apartado, para comprender el funcionamiento de los agentes de MCTS, se analizan posiciones concretas de una partida del juego 4 en raya gracias a la herramienta de depuración cuya implementación se puede encontrar en el anexo B.4. Esta herramienta utiliza `graphviz` para visualizar y analizar el comportamiento del método de MCTS al generar un diagrama de árbol con las posiciones siguientes solicitadas a partir de un determinado nodo inicial.

Cada nodo del árbol representa un estado del juego, el cual se alcanza mediante una secuencia de movimientos, y contiene información como el turno del jugador, el número de visitas y la recompensa de cada jugador. Las aristas indican las acciones tomadas para pasar de un estado al siguiente. Se puede especificar la profundidad máxima del árbol generado. Además, los nodos en el árbol aparecen identificados mediante un ID. Para poder visualizar cuál es el estado de cada nodo, la función `generate_tree_MCTS` devuelve un diccionario que asocia cada ID al nodo correspondiente.

Veamos un ejemplo sencillo para comprobar y analizar el funcionamiento del agente de MCTS, en este caso usando el método de selección UCB1 con $c = 1$, con un tablero de 6×7 . Se genera un árbol con una profundidad máxima de 1:

```
# Crear estado inicial del juego
initial_state = Connect4Game(rows=6, cols=7, starting_player=1)

# Visualizar el árbol de MCTS
root_node = Node(initial_state, None)
mcts = MCTS(root_node, UCB1, simulations=1000, c=1)
mcts.run()
id_to_node = generate_tree_MCTS(root_node, max_depth=1, filename="depuration_trees/
    arbol_connect4", format="jpg")
```

El árbol resultante es:

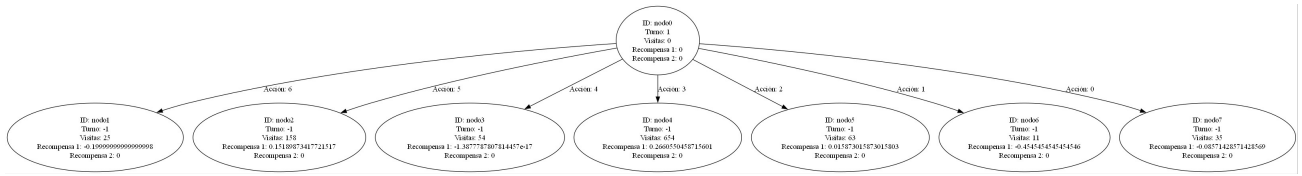


Figura 36: Diagrama de árbol de profundidad 1 de la partida de 4 en raya inicializada con un tablero vacío de 6×7 , con MCTS con método de selección UCB1 con $c = 1$.

En la Figura 36 se observa que el nodo más visitado, con 675 visitas, es **nodo4**, que además tiene la mayor recompensa para el jugador 1, de 0,266. Este nodo se alcanza cuando el jugador 1 realiza la acción de tirar una ficha en la columna 3 (la central) del tablero, como se aprecia en la arista correspondiente. La columna central da acceso a más combinaciones ganadoras posibles (verticales, horizontales y diagonales), y además permite al jugador expandirse hacia ambos lados con flexibilidad, por lo que tiene sentido que este sea el nodo más visitado, especialmente para el jugador que comienza la partida. Esto también se puede comprobar gracias al diccionario que asocia cada ID de nodo al nodo correspondiente, mediante:

```
node = id_to_node['nodo4']
node.state.draw()
```

En la Figura 37 se muestra el estado de dicho nodo:

```

[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0]
```

Figura 37: Estado del nodo con ID **nodo4**.

3.2.3. Simulaciones comparativas entre varios agentes

En esta sección, se van a comparar los diferentes agentes implementados, Monte Carlo y MCTS, realizando simulaciones en las que competirán entre ellos por pares en el juego 4 en raya con un tablero 5×5 . Para ello, se necesitan las siguientes funciones:

```
def play_comparative_game(agent1, agent2, rows=5, cols=5, starting_player=1):
    game = Connect4Game(rows=rows, cols=cols, starting_player=starting_player)
    agents = {1: agent1, -1: agent2}

    while not game.terminal():
```

```

        game = agents[game.turn].move(game)

    return game.winner()

def run_simulations(agent_1, agent_2, rows=5, cols=5, n_games=100):
    results = []

    for i in range(n_games):
        # Alternar jugador inicial
        starting_player = 1 if i % 2 == 0 else -1

        # Asignar agentes según quién empieza
        if starting_player == 1:
            winner = play_comparative_game(agent_1, agent_2, rows, cols, starting_player)
        else:
            winner = play_comparative_game(agent_2, agent_1, rows, cols, starting_player)
        # Invertir perspectiva
        winner *= -1

    results.append(winner)

df = pd.DataFrame(results, columns=["winner"])
win_rate = (df["winner"] == 1).mean()
print(f"% de partidas ganadas por el agente 1: {win_rate:.2%} ({df['winner'].value_counts().to_dict()})")
return df

```

De forma análoga a como se mostraba en la sección 3.1.3, se definen los agentes, esta vez usando un número de simulaciones de 300 para cada uno para reducir el coste computacional, y se realizan los enfrentamientos por pares de agentes. Los resultados de los enfrentamientos por pares se muestran en la tabla 2:

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	57	41	2	14m 33.8s
MC	MCTS UCB1	52	47	1	14m 16.4s
MC	MCTS UCB2	96	4	0	10m 35.4s
MC	MCTS Softmax	72	25	3	13m 21.0s
MC	MCTS Softmax Adaptativo	76	21	3	13m 51.8s
MCTS ϵ -greedy	MCTS UCB1	39	56	5	5m 19.8s
MCTS ϵ -greedy	MCTS UCB2	92	7	1	4m 45.2s
MCTS ϵ -greedy	MCTS Softmax	68	30	2	5m 12.6s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	70	26	4	5m 33.5s
MCTS UCB1	MCTS UCB2	96	3	1	4m 20.9s
MCTS UCB1	MCTS Softmax	74	19	7	5m 25.5s
MCTS UCB1	MCTS Softmax Adaptativo	75	16	9	5m 31.9s
MCTS UCB2	MCTS Softmax	26	72	2	5m 18.5s
MCTS UCB2	MCTS Softmax Adaptativo	22	76	2	5m 41.8s
MCTS Softmax	MCTS Softmax Adaptativo	53	40	7	6m 16.4s

Tabla 2: Resultados de enfrentamientos por pares entre agentes en el juego 4 en raya (100 partidas por par).

3.3. Othello

Othello es un juego de estrategia para dos jugadores en el que se colocan fichas en un tablero con el objetivo de tener la mayoría al final de la partida. Sus reglas son:

- El juego clásico se juega en un tablero de 8×8 casillas (aunque en esta implementación pueden tomar valores entre 4 y 10).
- Cada ficha tiene una cara blanca y una negra (en esta implementación se representan con 1 y 2).
- Comienza el jugador que lleva las fichas negras.
- Los jugadores se turnan para colocar una ficha con su color hacia arriba.
- Una jugada válida debe encerrar una o más fichas del oponente entre la nueva ficha y otra del mismo color.
- Las fichas del oponente que quedan encerradas se voltean al color del jugador que hizo la jugada.

- El juego termina cuando ningún jugador puede hacer una jugada válida.
- Gana quien tenga más fichas de su color en el tablero al final de la partida.

La implementación del juego Othello se muestra en el anexo B.1.3. En dicho anexo se muestra la clase `OthelloGame`, encargada de modelar el juego.

Las importaciones necesarias para el código utilizado en esta sección son las siguientes:

```
import sys
import os
import pandas as pd
import random
import numpy as np

# Fijar la semilla
SEED = 123
random.seed(SEED)
np.random.seed(SEED)

# Agregar los directorios al path para poder importar los módulos
sys.path.append(os.path.abspath("../games"))
sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))
sys.path.append(os.path.abspath("../graphviz"))

# Importar la clase OthelloGame y las funciones de los métodos
from othello import OthelloGame
from monte_carlo import MCPlay, MCAgent
from monte_carlo_tree_search import MCTS, MCTSPlay, Node, MCTSAgent
from epsilon_greedy import EpsilonGreedy
from softmax import Softmax
from adaptive_softmax import AdaptiveSoftmax
from ucb1 import UCB1
from ucb2 import UCB2
from gradiente_preferencias import GradienteDePreferencias
from MCTS_graphviz import generate_tree_MCTS
```

3.3.1. Simulaciones humano vs Monte Carlo

En este apartado, se muestran simulaciones en las que el jugador humano se enfrenta a un agente de Monte Carlo. El objetivo principal de estas simulaciones es observar el desempeño del agente frente a un jugador humano en el juego Othello.

En cada turno del agente, se ejecutan un número determinado de simulaciones (en este caso, 150) para estimar cuál de las jugadas disponibles tiene mayor probabilidad de llevar a una victoria. El juego se desarrolla en un tablero de 6 filas y 6 columnas. El agente de Monte Carlo comienza la partida, por lo que sus fichas, representadas por el número 2,

serán las fichas negras. En el siguiente fragmento de código se muestra la lógica utilizada en estas simulaciones:

```
# Crear una instancia del juego Othello
game = OthelloGame(rows=6, cols=6, starting_player=-1)

# Mostrar el estado inicial del juego
game.draw()

# Definir el número de simulaciones por movimiento
num_simulations = 150

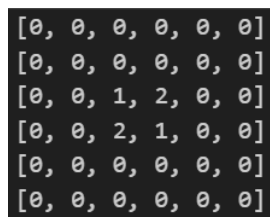
# Bucle del juego hasta que termine
while not game.terminal():
    if game.turn == 1: # Turno del jugador humano
        print("\nTu turno...")
        jugada = eval(input("En qué casilla quieres colocar tu ficha? (fila , columna): "))
    )

    if jugada in game.valid_actions():
        game = game.action(jugada)
    else:
        print("Jugada inválida. Inténtalo de nuevo.")
        continue # Volver a pedir jugada

    else: # Turno de la IA (Monte-Carlo)
        print("\nTurno de la computadora...")
        game = MCPlay(game, num_simulations)

    game.draw() # Mostrar el estado después del turno

# Anunciar el ganador
if game.winner() == 1:
    print("\nGanas la partida!")
elif game.winner() == -1:
    print("\nLa computadora gana. Inténtalo de nuevo!")
else:
    print("\nEs un empate!")
```



```
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 1, 2, 0, 0]
[0, 0, 2, 1, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
```

Figura 38: Estado inicial de la partida de Othello con Humano vs Monte Carlo.

La partida comienza inicialmente con el tablero vacío (celdas a 0) excepto por las 4 celdas centrales, tal como se muestra en la Figura 38. En dichas celdas, hay una ficha blanca en

la celda superior izquierda y en la inferior derecha, y una ficha negra en la celda superior derecha y en la inferior izquierda. Las fichas negras corresponden al jugador que comienza la partida, que en este caso es el agente de Monte Carlo, por lo que vienen dadas por un 2. Las fichas blancas corresponden al jugador humano y vienen dadas por un 1.

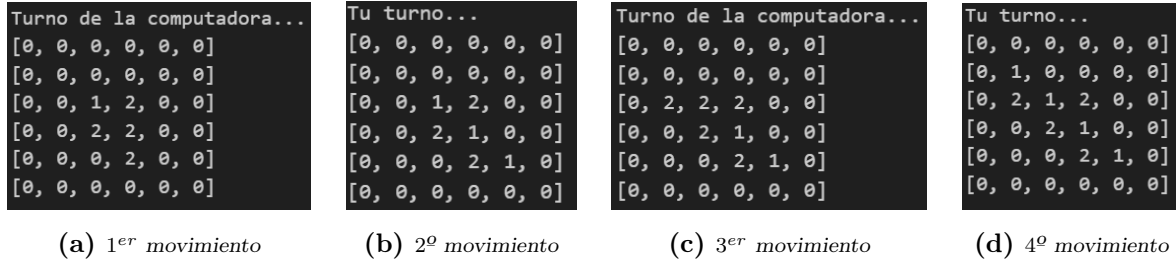


Figura 39: Movimientos del 1^o al 4^o de la partida de Othello con Humano vs Monte Carlo.

En la Figura 39 se muestran los 4 primeros movimientos de la partida. Comienza la partida el agente de Monte Carlo, que coloca su ficha en la posición (4, 3) (el primer valor representa la fila, enumeradas del 0 al 5, y el segundo representa la columna, enumeradas del 0 al 5), tal como se muestra en la Figura 39a. Al realizar dicho movimiento, una ficha blanca del jugador humano (1) queda encerrada verticalmente entre dos fichas negras del agente de Monte Carlo (2), por lo que dicha ficha blanca se voltea y pasa a ser negra (cambia de 1 a 2). A continuación, tal como se muestra en la Figura 39b, el jugador humano lanza en la posición (4, 4), encerrando diagonalmente una ficha negra (2) entre dos blancas (1), por lo que la ficha negra se voltea y pasa a ser blanca. Análogamente, en la Figura 39c, el agente de Monte Carlo lanza en la posición (2, 1), dejando encerrada horizontalmente una ficha blanca entre dos negras, por lo que se voltea. Después, en el cuarto movimiento de la partida, en la Figura 39d se observa que el jugador humano lanza en la posición (1, 1), y de nuevo deja encerrada diagonalmente una ficha negra entre dos blancas, por lo que la ficha negra se voltea.

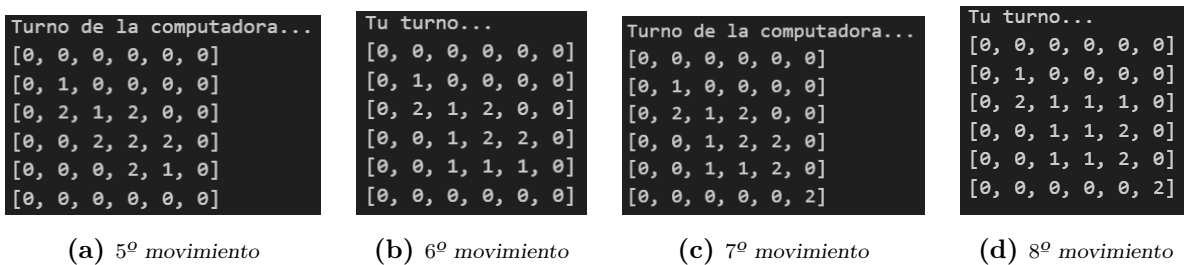


Figura 40: Movimientos del 5^o al 8^o de la partida de Othello con Humano vs Monte Carlo.

En la Figura 40 se muestran los 4 siguientes movimientos de la partida. Estos son análogos a los anteriores, con la particularidad de que en el 6º movimiento, representado en la Figura 40b, el jugador humano lanza en la casilla (4,2), dejando encerrada una ficha negra en vertical entre dos blancas, y otra en horizontal, también entre dos blancas. Ambas fichas negras se voltean y pasan a ser blancas.

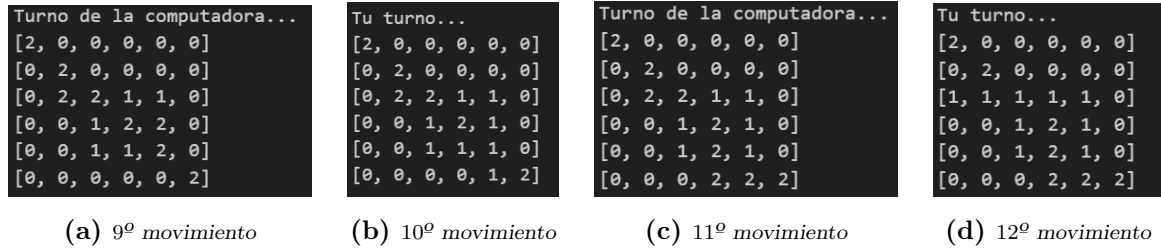


Figura 41: Movimientos del 9º al 12º de la partida de Othello con Humano vs Monte Carlo.

En la Figura 41 se muestran los 4 siguientes movimientos de la partida. Es interesante observar como, en la Figura 41a, el agente de Monte Carlo, al colocar una ficha en la posición (0,0), encierra a 3 fichas blancas consecutivas en diagonal entre dos negras. Las 3 fichas son volteadas y pasan a ser negras. Algo similar ocurre en la Figura 41b, donde el jugador humano lanza en la casilla (5,4), dejando encerradas 2 fichas negras entre dos blancas verticalmente. Las 2 fichas negras se voltean y pasan a ser blancas. En la Figura 41c el agente de Monte Carlo lanza en la posición (5,3), encerrando una ficha blanca entre dos negras verticalmente, y otra ficha blanca entre dos negras horizontalmente. Ambas se voltean y pasan a ser negras. Seguidamente, en la Figura 41d, el jugador humano lanza en la casilla (2,0) y encierra dos fichas negras entre dos blancas, por lo que se voltean.

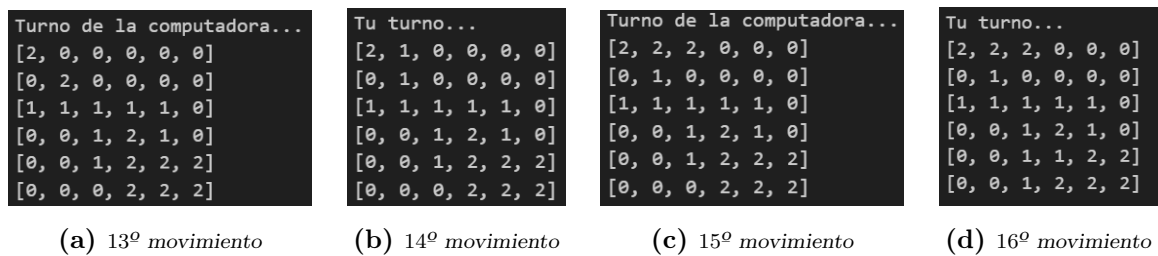


Figura 42: Movimientos del 13º al 16º de la partida de Othello con Humano vs Monte Carlo.

En la Figura 42 se muestran los 4 siguientes movimientos de la partida. La partida se sigue desarrollando de forma análoga a los movimientos anteriores.

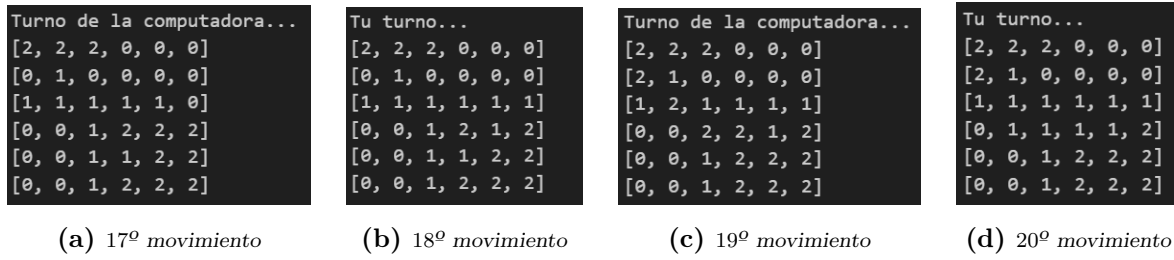


Figura 43: Movimientos del 17^o al 20^o de la partida de Othello con Humano vs Monte Carlo.

En la Figura 43 se muestran los 4 siguientes movimientos de la partida. De nuevo, estos son análogos a los movimientos vistos anteriormente.

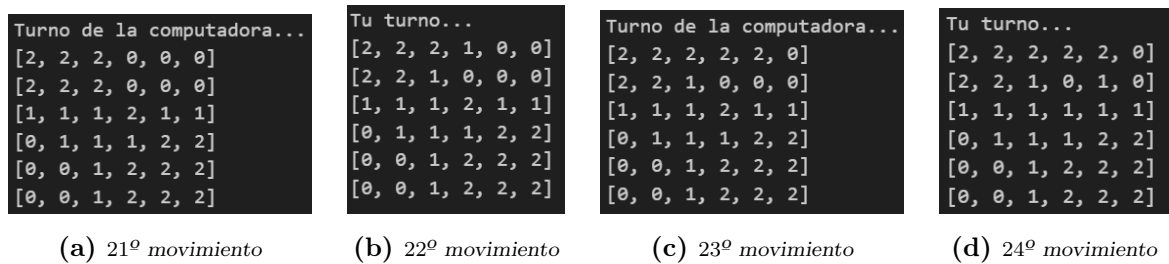


Figura 44: Movimientos del 21^o al 24^o de la partida de Othello con Humano vs Monte Carlo.

En la Figura 44 se muestran los 4 siguientes movimientos de la partida. La partida sigue desarrollándose análogamente a los movimientos anteriores.

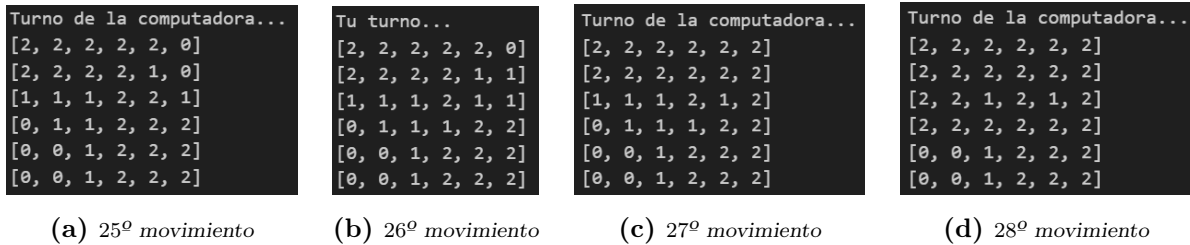


Figura 45: Movimientos del 25^o al 28^o de la partida de 4 en raya con Humano vs Monte Carlo.

En la Figura 45 se muestran los 4 siguientes movimientos de la partida. Es interesante observar como, en la Figura 45c, el agente de Monte Carlo termina su turno, pero en el tablero resultante, el jugador humano no tiene ningún movimiento válido (esto es, un movimiento que encierre una o más fichas negras del oponente entre la nueva ficha blanca lanzada por el jugador humano y otra blanca). Por esta razón, el jugador humano pierde el turno, de forma que en el siguiente turno vuelve a lanzar el agente de Monte Carlo, como se observa en la Figura 45d.

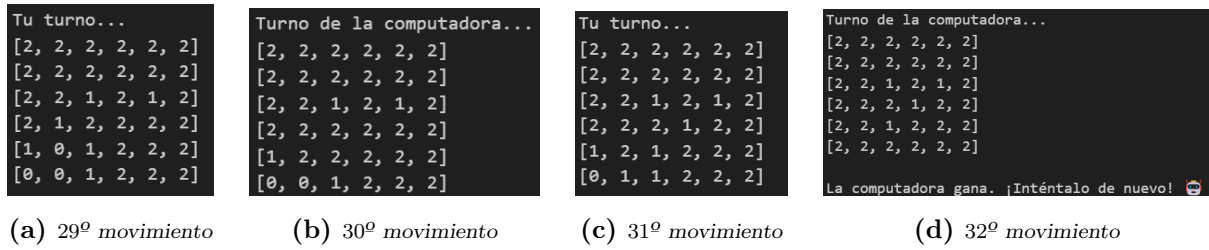


Figura 46: Movimientos del 29º al 32º de la partida de Othello con Humano vs Monte Carlo.

En la Figura 46 se muestran los 4 siguientes y últimos movimientos de la partida. En los últimos movimientos del jugador humano, representados en la Figura 46a y en la Figura 46c, este está prácticamente acorralado, pues solo tiene un movimiento válido en cada uno de esos turnos, así que no tiene más opción que elegir dicho movimiento. Finalmente, en la Figura 46d, el agente de Monte Carlo realiza el último movimiento posible, lanzando en la única casilla que queda vacía, la (5,0), y acaba la partida pues el tablero está completamente lleno y ya no quedan más jugadas válidas para ningún jugador. El agente de Monte Carlo gana la partida dado que en el tablero final, la mayoría de las fichas son de su color, negro, representado por el número 2.

3.3.2. Simulaciones comparativas entre varios agentes

En esta sección, se van a comparar los diferentes agentes implementados, Monte Carlo y MCTS, realizando simulaciones en las que competirán entre ellos por pares en el juego Othello con un tablero 4×4 . Para ello, se necesita la siguiente función junto con la función `run_simulations` definida en la sección 3.2.3:

```
def play_comparative_game(agent1, agent2, rows=4, cols=4, starting_player=1):
    game = OthelloGame(rows=rows, cols=cols, starting_player=starting_player)
    agents = {1: agent1, -1: agent2}

    while not game.terminal():
        game = agents[game.turn].move(game)

    return game.winner()
```

De forma análoga a como se mostraba en la sección 3.1.3, se definen los agentes, esta vez usando un número de simulaciones de 300 para cada uno para reducir el coste computacional, y se realizan los enfrentamientos por pares de agentes. Los resultados de los enfrentamientos por pares se muestran en la tabla 3:

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	1	99	0	10m 19.3s
MC	MCTS UCB1	0	100	0	9m 36.3s
MC	MCTS UCB2	23	77	0	9m 31.6s
MC	MCTS Softmax	0	100	0	9m 46.2s
MC	MCTS Softmax Adaptativo	0	100	0	10m 22.3s
MCTS ϵ -greedy	MCTS UCB1	0	100	0	3m 25.5s
MCTS ϵ -greedy	MCTS UCB2	46	54	0	3m 23.3s
MCTS ϵ -greedy	MCTS Softmax	24	76	0	3m 56.6s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	28	72	0	4m 20.3s
MCTS UCB1	MCTS UCB2	78	22	0	3m 32.4s
MCTS UCB1	MCTS Softmax	77	23	0	4m 5.3s
MCTS UCB1	MCTS Softmax Adaptativo	73	27	0	4m 21.9s
MCTS UCB2	MCTS Softmax	3	97	0	4m 4.2s
MCTS UCB2	MCTS Softmax Adaptativo	3	97	0	4m 50.5s
MCTS Softmax	MCTS Softmax Adaptativo	15	85	0	5m 9.5s

Tabla 3: Resultados de enfrentamientos por pares entre agentes en el juego Othello (100 partidas por par).

3.4. Go

Go es un juego de estrategia para dos jugadores en el que se colocan fichas en un tablero con el objetivo de controlar el mayor territorio al final de la partida. Sus reglas son:

- El juego se juega en un tablero cuadrado con un número impar de filas y columnas entre 5 y 19 (por defecto 9×9).
- Los jugadores se turnan para colocar fichas de su color (representadas por 1 o 2) en las intersecciones vacías del tablero.
- Una jugada válida debe respetar las reglas de no suicidio y Superko situacional:
 - Regla de no suicidio: Un jugador no puede colocar una ficha que haga que su propio grupo quede sin libertades inmediatamente, a menos que esa jugada capture fichas enemigas que devuelvan libertades al grupo.
 - Regla de Superko ¹ situacional: Un jugador no puede realizar una jugada que

¹Proveniente del coreano "ko", que significa bloqueo.

haga que el tablero y el turno actual coincidan exactamente con una situación previa en la misma partida (para evitar ciclos infinitos).

- Se llaman libertades a las casillas vacías adyacentes a una cierta ficha o grupo de fichas del mismo color.
- Si al colocar una ficha se rodean grupos enemigos sin libertades, estas fichas se capturan y se retiran del tablero.
- Un jugador puede pasar su turno, y si ambos jugadores pasan consecutivamente, el juego termina.
- El puntaje final de cada jugador se calcula sumando:
 - Las fichas en el tablero de dicho jugador.
 - Las fichas capturadas al oponente.
 - El territorio rodeado exclusivamente por fichas del color de dicho jugador.
 - La compensación Komi de 6.5 puntos para el jugador que no comenzó
- Gana el jugador con más puntos al final de la partida.

La implementación del juego Go se muestra en el anexo B.1.4. En dicho anexo se muestra la clase `GoGame`, encargada de modelar el juego.

Las importaciones necesarias para el código utilizado en esta sección son las siguientes:

```
import sys
import os
import pandas as pd
import random
import numpy as np

# Fijar la semilla
SEED = 123
random.seed(SEED)
np.random.seed(SEED)

# Agregar los directorios al path para poder importar los módulos
sys.path.append(os.path.abspath("../games"))
sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))
sys.path.append(os.path.abspath("../graphviz"))

# Importar la clase GoGame y las funciones de los métodos
from go import GoGame
from monte_carlo import MCPlay, MCAGENT
from monte_carlo_tree_search import MCTS, MCTSPlay, Node, MCTSAGENT
```

```
from epsilon_greedy import EpsilonGreedy
from softmax import Softmax
from adaptive_softmax import AdaptiveSoftmax
from ucb1 import UCB1
from ucb2 import UCB2
from gradiente_preferencias import GradienteDePreferencias
from MCTS_graphviz import generate_tree_MCTS
```

3.4.1. Simulaciones humano vs Monte Carlo

En este apartado, se muestran simulaciones en las que el jugador humano se enfrenta a un agente de Monte Carlo. El objetivo principal de estas simulaciones es observar el desempeño del agente frente a un jugador humano en el juego Go.

En cada turno del agente, se ejecutan un número determinado de simulaciones (en este caso, 20, debido a que por la alta complejidad del juego, el tiempo de ejecución se vuelve insostenible con un número más elevado de simulaciones) para estimar cuál de las jugadas disponibles tiene mayor probabilidad de llevar a una victoria. El juego se desarrolla en un tablero de 5 filas y 5 columnas. El agente de Monte Carlo comienza la partida. En el siguiente fragmento de código se muestra la lógica utilizada en estas simulaciones:

```
# Crear una instancia del juego Go
game = GoGame(rows=5, cols=5, starting_player=-1)

# Mostrar el estado inicial del juego
game.draw()

# Definir el número de simulaciones por movimiento
num_simulations = 20

# Bucle del juego hasta que termine
while not game.terminal():
    if game.turn == 1: # Turno del jugador humano
        print("\nTu turno...")
        jugada = eval(input("En qué casilla (fila, columna) quieres colocar tu ficha? (
        None para pasar de turno): "))

        if jugada in game.valid_actions():
            game = game.action(jugada)
        else:
            print("Jugada inválida. Inténtalo de nuevo.")
            continue # Volver a pedir jugada

    else: # Turno de la IA (Monte-Carlo)
        print("\nTurno de la computadora...")
        game = MCPlay(game, num_simulations)

game.draw() # Mostrar el estado después del turno
```

```
# Anunciar el ganador
if game.winner() == 1:
    print("\nGanas la partida!")
elif game.winner() == -1:
    print("\nLa computadora gana. Inténtalo de nuevo!")
else:
    print("\nEs un empate!")
```

```
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
```

Figura 47: Estado inicial de la partida de Go con Humano vs Monte Carlo.

La partida comienza inicialmente con el tablero vacío (todas las celdas a 0), tal como se muestra en la Figura 47.

Turno de la computadora... <pre>[0, 0, 0, 0, 0] [0, 0, 0, 2, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0]</pre>	Tu turno... <pre>[0, 0, 0, 0, 0] [0, 0, 1, 2, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0]</pre>	Turno de la computadora... <pre>[0, 0, 0, 0, 0] [2, 0, 1, 2, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0]</pre>	Tu turno... <pre>[0, 0, 0, 1, 0] [2, 0, 1, 2, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0] [0, 0, 0, 0, 0]</pre>
(a) 1 ^{er} movimiento	(b) 2 ^o movimiento	(c) 3 ^{er} movimiento	(d) 4 ^o movimiento

Figura 48: Movimientos del 1^o al 4^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 48 se muestran los 4 primeros movimientos de la partida. Las fichas del jugador humano vienen representadas por un 1, mientras que las fichas del agente de Monte Carlo vienen dadas por un 2. El agente de Monte Carlo realiza el primer movimiento, lanzando en la casilla (1, 3) (el primer elemento representa la fila, y el segundo la columna, ambas numeradas del 0 al 4), tal como se muestra en la Figura 48a. A continuación, el jugador humano lanza en la casilla (1, 2), como se aprecia en la Figura 48b. Después, el agente de Monte Carlo lanza en la posición (1, 0), como se observa en la Figura 48c. Este movimiento no parece tener mucho sentido, pues se encuentra lejos de la única casilla ocupada por el jugador humano, y no es una jugada urgente que responda a ninguna amenaza ni la plantee. Por ejemplo, podría haber lanzado en (1, 1) o en (2, 2) para empezar a rodear a la ficha del jugador humano que se encuentra en (1, 2). Esto refleja que, aunque en juegos simples, Monte Carlo funciona razonablemente bien, en juegos más complejos como Go se vuelve ineficiente. Además, el tomar un número tan bajo de simulaciones (20) debido

al elevado tiempo de ejecución, también podría estar perjudicando el funcionamiento de Monte Carlo. Este mal comportamiento del agente de Monte Carlo se repite a lo largo de toda la partida.

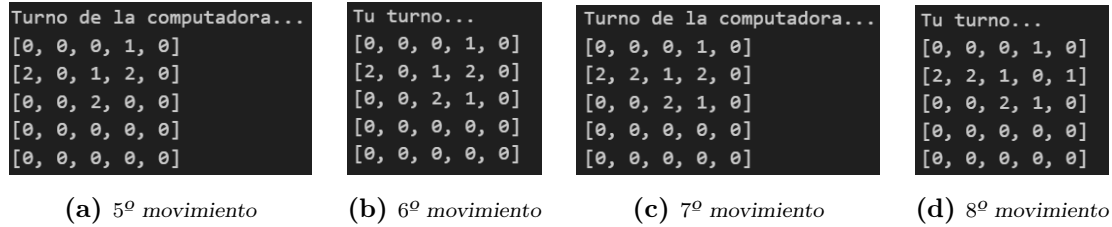


Figura 49: Movimientos del 5º al 8º de la partida de Go con Humano vs Monte Carlo.

En la Figura 49 se muestran los 4 siguientes movimientos de la partida. En la Figura 49a el agente de Monte Carlo coloca una ficha en (2, 2), justo debajo de la ficha enemiga en (1, 2). Esto se puede interpretar como un intento de empezar a rodear o presionar esa ficha. Además, está alineado con su ficha anterior en (1, 3). Por lo que este movimiento sí parece tener más sentido. Además, en la Figura 49c, el agente coloca una ficha en (1, 1), adyacente a la ficha enemiga (1, 2) y también a su propia piedra en (1, 0). Esto empieza a encerrar a la piedra del jugador humano por el lado izquierdo. Además, conecta sus propias piedras en la fila 1, lo cual mejora su estructura y reduce vulnerabilidades. Este movimiento también parece tener sentido. Esto sugiere que, aunque el agente tiene limitaciones por la complejidad del juego y el bajo número de simulaciones, ocasionalmente puede realizar jugadas razonables, especialmente cuando las opciones son más claras tácticamente. En la Figura 49d, el jugador humano lanza en la casilla (1, 4), con lo cual rodea completamente a la ficha del agente en (1, 3), que queda sin libertades, por lo que es capturada y se retira del tablero.

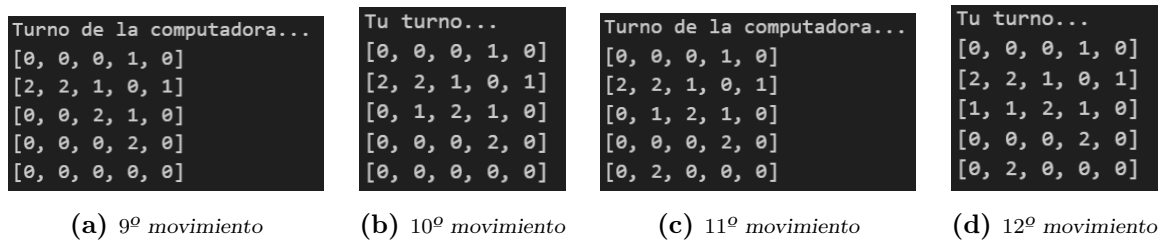


Figura 50: Movimientos del 9º al 12º de la partida de Go con Humano vs Monte Carlo.

En la Figura 50 se muestran los 4 siguientes movimientos de la partida. En la Figura 50a, el agente de Monte Carlo lanza en la posición (3, 3), la cual está algo aislada del

resto de sus fichas, aunque podría estar intentando rodear la ficha del jugador humano que se encuentra en (2,3). Sin embargo, su siguiente movimiento en (4,1), mostrado en la Figura 50c, no parece tener ningún sentido. Esta posición no conecta con ninguna de sus otras fichas, ni amenaza a ninguna ficha del oponente. El agente vuelve a demostrar un comportamiento ineficiente, reforzando la idea de que Monte Carlo puro no es adecuado para Go, incluso en tableros pequeños.

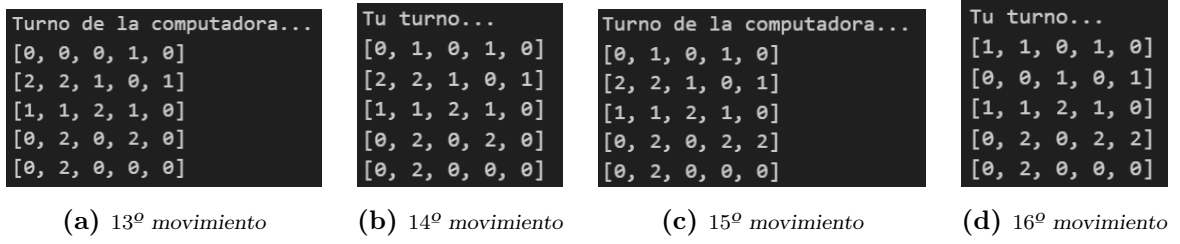


Figura 51: Movimientos del 13^o al 16^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 51 se muestran los 4 siguientes movimientos de la partida. Por ejemplo, en la Figura 51c, el agente de Monte Carlo podría haber lanzado en (3,0) para rodear y dejar sin libertades al grupo del jugador humano que se encuentra en las posiciones (2,0) y (2,1), pero en su lugar, lanza en (3,4), lo cual no parece tener mucho sentido. Además, en la Figura 51d, el jugador humano lanza en (0,0), con lo cual consigue rodear por completo un grupo del agente, que queda sin libertades y es retirado del tablero.

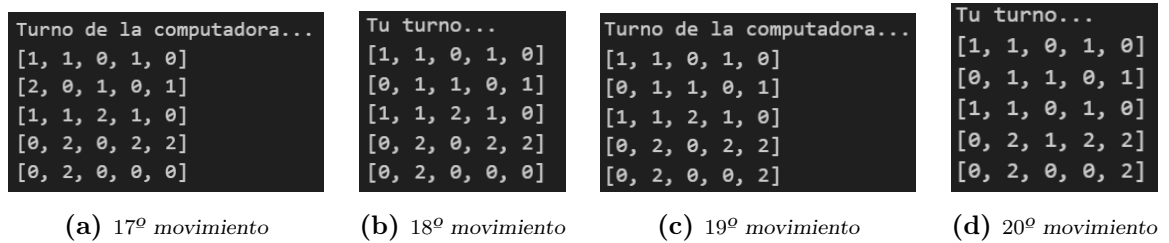


Figura 52: Movimientos del 17^o al 20^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 52 se muestran los 4 siguientes movimientos de la partida. La partida se sigue desarrollando sin grandes avances por parte del agente de Monte Carlo.

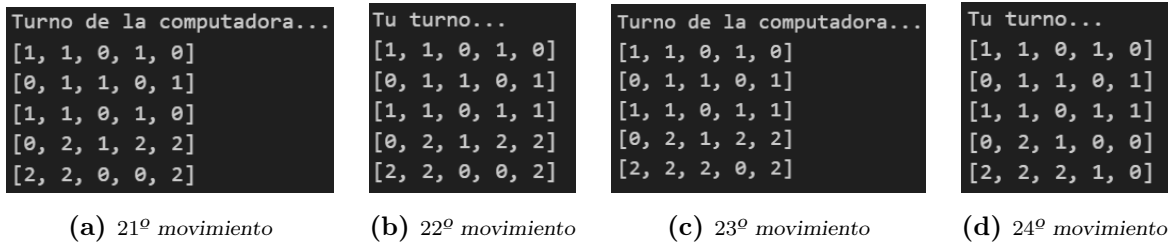


Figura 53: Movimientos del 21^o al 24^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 53 se muestran los 4 siguientes movimientos de la partida. En la Figura 53d el jugador humano lanza en (4,3), con lo cual deja sin libertades a un grupo del agente de Monte Carlo formado por 3 fichas, que son retiradas del tablero.

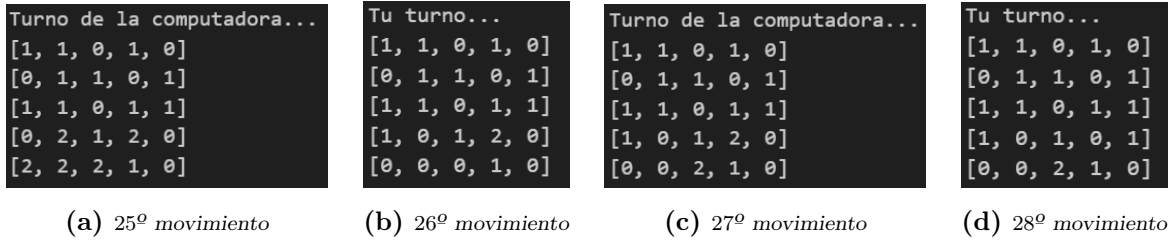


Figura 54: Movimientos del 25^o al 28^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 54 se muestran los 4 siguientes movimientos de la partida. El agente de Monte Carlo sigue sin remontar. De hecho, en la Figura 54b el jugador humano lanza en (3,0), con lo cual rodea un grupo del agente formado por 4 fichas que son eliminadas al quedar sin libertades, y en la Figura 54d el jugador humano lanza en (3,4) y rodea a otra ficha del agente que también es eliminada.

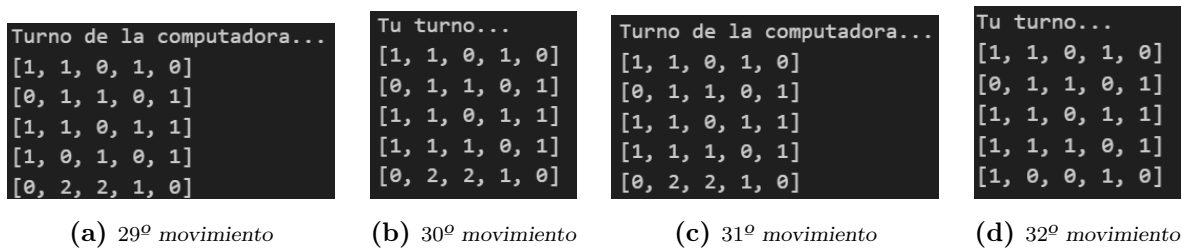


Figura 55: Movimientos del 29^o al 32^o de la partida de Go con Humano vs Monte Carlo.

En la Figura 55 se muestran los 4 siguientes movimientos de la partida. En la Figura 55c el agente de Monte Carlo pasa el turno, lo cual no tiene sentido, ya que en Go, pasar solo tiene sentido si no hay movimientos significativos restantes y el jugador está seguro

de estar ganando. Sin embargo, este no es el caso, y de hecho podría convenirle tirar en casillas como la (3,3) o la (1,3), ya que están cerca de grupos importantes.

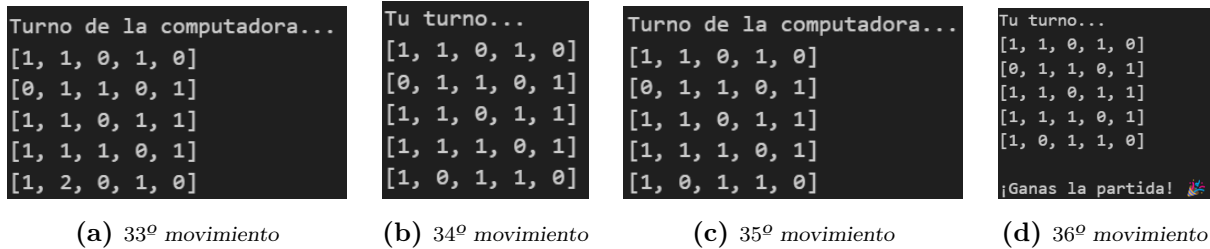


Figura 56: Movimientos del 33º al 36º de la partida de Go con Humano vs Monte Carlo.

En la Figura 56 se muestran los 4 siguientes y últimos movimientos de la partida. En la Figura 56c el agente de Monte Carlo pasa turno, lo cual no tiene sentido porque claramente está perdiendo la partida, y debería lanzar para intentar remontar. Después, el jugador humano vuelve a pasar turno, con lo que finaliza la partida, y al recomtar los puntos de cada jugador, el ganador es el jugador humano, como se muestra en la Figura 56d.

3.4.2. Simulaciones comparativas entre varios agentes

En esta sección, se van a comparar los diferentes agentes implementados, Monte Carlo y MCTS, realizando simulaciones en las que competirán entre ellos por pares en el juego Go con un tablero 5×5 . Para ello, se necesita la siguiente función junto con la función `run_simulations` definida en la sección 3.2.3:

```
def play_comparative_game(agent1, agent2, rows=5, cols=5, starting_player=1):
    game = GoGame(rows=rows, cols=cols, starting_player=starting_player)
    agents = {1: agent1, -1: agent2}

    while not game.terminal():
        game = agents[game.turn].move(game)

    return game.winner()
```

De forma análoga a como se mostraba en la sección 3.1.3, se definen los agentes, esta vez usando un número de simulaciones tan solo de 100 para cada agente, debido al muy elevado coste computacional en este caso. En particular, el agente de Monte Carlo presenta el mayor coste computacional de todos que, sumado a la gran complejidad del juego Go, se vuelve prácticamente inviable usar un número elevado de simulaciones. Esto muestra una clara limitación que presenta Monte Carlo básico cuando la complejidad del

juego aumenta, y muestra la necesidad de usar algoritmos más eficientes y complejos. De esta forma, se realizan los enfrentamientos por pares de agentes. En esta ocasión solo se realizarán 10 partidas por cada par de agentes, ya que, de nuevo, el coste computacional es demasiado elevado como para hacer 100 partidas por par como con los anteriores juegos. Es importante tener en cuenta que todas estas limitaciones pueden afectar negativamente a los resultados, que pueden no reflejar correctamente el funcionamiento de los agentes, por lo que los análisis realizados con los anteriores juegos pueden reflejar mejor su rendimiento. Los resultados de los enfrentamientos por pares se muestran en la tabla 4:

Agente A	Agente B	Ganó A	Ganó B	Empates	Runtime
MC	MCTS ϵ -greedy	10	0	0	87m 34.6s
MC	MCTS UCB1	10	0	0	115m 9.0s
MC	MCTS UCB2	10	0	0	99m 25.4s
MC	MCTS Softmax	10	0	0	101m 23.2s
MC	MCTS Softmax Adaptativo	10	0	0	98m 4.3s
MCTS ϵ -greedy	MCTS UCB1	8	2	0	13m 25.5s
MCTS ϵ -greedy	MCTS UCB2	7	3	0	13m 37.2s
MCTS ϵ -greedy	MCTS Softmax	7	3	0	14m 59.7s
MCTS ϵ -greedy	MCTS Softmax Adaptativo	6	4	0	14m 21.5s
MCTS UCB1	MCTS UCB2	7	3	0	12m 47.7s
MCTS UCB1	MCTS Softmax	7	3	0	14m 47.3s
MCTS UCB1	MCTS Softmax Adaptativo	8	2	0	12m 17.9s
MCTS UCB2	MCTS Softmax	8	2	0	13m 30.0s
MCTS UCB2	MCTS Softmax Adaptativo	7	3	0	15m 5.1s
MCTS Softmax	MCTS Softmax Adaptativo	5	5	0	12m 16.8s

Tabla 4: Resultados de enfrentamientos por pares entre agentes en el juego Go (10 partidas por par).

Los resultados mostrados en la tabla 4 están muy desequilibrados y muestran una aparente anomalía: Monte Carlo gana casi todas las partidas frente a todas las variantes de MCTS, lo cual no parece coherente con el comportamiento observado en juegos anteriores, ni con lo que se podría esperar teóricamente. Este comportamiento, aunque puede parecer contradictorio, puede explicarse por una combinación de factores relacionados con la configuración de los experimentos y la propia naturaleza del juego Go:

1. **Número bajo de simulaciones:** Todos los agentes han sido limitados a realizar

solo 100 simulaciones por movimiento, debido a restricciones computacionales. Esta cantidad es especialmente insuficiente para el algoritmo MCTS, ya que este depende de realizar múltiples simulaciones para construir un árbol de búsqueda significativo y ser capaz de identificar los nodos más prometedores del mismo. Por otro lado, Monte Carlo no construye ningún árbol, sino que simplemente realiza partidas aleatorias, lo cual puede explicar que su rendimiento no se degrade tanto cuando el número de simulaciones es muy bajo.

2. **Go con tablero muy reducido:** Aunque Go es un juego de gran complejidad, al usar un tablero tan pequeño, de solo 5×5 , se simplifica mucho el espacio de juego. Esto sumado a las pocas simulaciones realizadas puede llevar a que el comportamiento aleatorio de Monte Carlo genere jugadas razonables por pura casualidad, mientras que MCTS, al seguir rutas concretas del árbol con información insuficiente, puede cometer más errores.
3. **Parámetros de MCTS no óptimos:** Es posible que los parámetros seleccionados por defecto para las políticas de selección de MCTS no estén bien ajustados para este juego. Una posible mejora de los resultados podría lograrse probando con distintos valores de estos parámetros, para ver si se logran cambios significativos.

Sin embargo, los resultados cuando se enfrentan dos agentes de MCTS entre sí tienen algo más de sentido, pues no están tan desequilibrados como en los pares en los que Monte Carlo se enfrenta a un agente de MCTS. Por otro lado, el coste computacional es muy elevado a pesar de estar realizando únicamente 10 partidas por par, sobre todo en los enfrentamientos en los que uno de los agentes es el agente de Monte Carlo. Esto muestra una clara limitación del algoritmo de Monte Carlo en juegos de elevada complejidad, especialmente en comparación con MCTS, que muestra un tiempo de ejecución mucho menor.

4. Conclusiones

En este Trabajo Final de Máster se han estudiado diferentes métodos para la toma de decisiones en juegos de estrategia. En primer lugar, se ha comenzado con la exploración de los principales conceptos de la teoría de juegos, iniciando con las definiciones básicas, lo cual ha proporcionado una base teórica para comprender en qué consiste un comportamiento racional en situaciones competitivas y estratégicas.

Sin embargo, debido a las limitaciones de los enfoques tradicionales de la teoría de juegos en escenarios complejos, se ha avanzado a presentar técnicas más sofisticadas como Monte Carlo y Monte Carlo Tree Search. Estos algoritmos se han puesto a prueba con juegos de creciente dificultad y complejidad, empezando por el sencillo juego Nim, continuando con 4 en raya, Othello y finalmente Go, el más complejo de todos.

Para cada uno de estos juegos, se han realizado una serie de simulaciones en las que el agente de Monte Carlo jugaba contra el jugador humano, para así poder comprender el funcionamiento del juego de forma más visual. Se ha visto que Monte Carlo funciona decentemente en juegos medianamente sencillos como Nim y 4 en raya, e incluso en juegos ligeramente más complejos como Othello, ganando al jugador humano con relativa facilidad. Aunque para Nim se ha presentado una conocida estrategia ganadora, la cual, al ser aplicada por el jugador humano, Monte Carlo no ha sido capaz de vencerla, y además, cuando el jugador humano no la ha aplicado, Monte Carlo ha tardado varios turnos en descubrir dicha estrategia, aunque finalmente la ha acabado aplicando. A pesar de estas limitaciones, el desempeño general del agente de Monte Carlo en los juegos Nim, 4 en raya y Othello ha sido considerablemente bueno.

No obstante, en juegos más complejos como Go, el agente de Monte Carlo ha mostrado un rendimiento bastante deplorable frente al jugador humano. La mayoría de sus jugadas parecían aleatorias y sin sentido, no logrando aplicar estrategias lógicas y eficientes en muchos de sus movimientos. Esto finalmente le ha llevado a perder la partida frente al jugador humano, quien ha logrado ganar con bastante facilidad y rapidez.

Por otro lado, se ha probado una herramienta de depuración en varios de los juegos para MCTS, implementada con `graphviz`, una librería de Python que permite visualizar un diagrama de árbol que representa las posiciones siguientes del juego a partir de una posición inicial dada. Concretamente, esta herramienta se ha utilizado en Nim y en 4 en raya, ya que permiten usar versiones con pocas jugadas válidas, de forma que los árboles generados son sencillos y fáciles de entender visualmente. Sin embargo, con los juegos Othello y Go, al tener una mayor complejidad y más jugadas posibles por turno,

el árbol se habría vuelto demasiado extenso y difícil de visualizar. Gracias a esta herramienta de depuración se ha podido observar con mayor claridad el funcionamiento del algoritmo MCTS, y como prioriza explorar aquellos nodos más prometedores en términos de recompensa.

Finalmente, los distintos agentes implementados se han enfrentado por pares en cada uno de los juegos, lo cual permite analizar el funcionamiento de cada método en juegos de variable dificultad y complejidad computacional. En juegos simples como Nim, se ha podido apreciar un funcionamiento considerablemente bueno de Monte Carlo básico frente a MCTS. Dependiendo del método de selección ha habido algunas diferencias en los resultados, por ejemplo, con ϵ -greedy y con UCB1, el agente de MCTS ha logrado vencer a Monte Carlo básico, pero con otras políticas de selección, de hecho Monte Carlo ha mostrado un mejor desempeño que MCTS.

De forma similar, en el juego 4 en raya, el agente de Monte Carlo ha seguido mostrando un buen rendimiento, logrando superar a MCTS en todos los escenarios considerados. Aunque con las políticas de selección ϵ -greedy y UCB1 los resultados han estado más reñidos, con los otros métodos de selección Monte Carlo ha destacado considerablemente por encima de MCTS.

No obstante, al pasar a juegos más complejos como Othello, el rendimiento del agente de Monte Carlo básico se ha deteriorado considerablemente. En todos los casos, con todas las políticas de selección, los agentes de MCTS han superado con creces a Monte Carlo, ganando cerca del 100% de las partidas en la mayoría de los casos. Esto muestra una evidente limitación del método de Monte Carlo básico en juegos de mayor complejidad.

Además, en el juego Go, todavía de mayor complejidad, las limitaciones de Monte Carlo básico se han vuelto todavía más evidentes. En este caso, se ha debido tomar un tablero lo más pequeño posible (5×5), con tan solo 100 simulaciones por agente, y se han llevado a cabo únicamente 10 partidas por cada par de agentes enfrentados, debido a la muy elevada complejidad computacional del juego. Los resultados obtenidos no reflejan el rendimiento real de los agentes, debido a las pocas simulaciones y partidas consideradas. A pesar de que teóricamente el algoritmo MCTS debería superar al método de Monte Carlo básico en juegos complejos como Go, los resultados obtenidos en este trabajo muestran justo lo contrario. Esto puede explicarse por múltiples factores, como el número reducido de simulaciones por movimiento, lo que impide a MCTS construir árboles verdaderamente informativos. El agente de Monte Carlo, al no construir ningún árbol, puede en algunos casos operar de forma más estable en contextos con muy poca información disponible.

Sin embargo, lo que sí se ha podido observar es la gran diferencia en el tiempo de

ejecución entre los distintos pares enfrentados. En aquellos pares en los que se enfrentaba un agente de Monte Carlo frente a uno de MCTS, incluso tras todas las simplificaciones realizadas, el tiempo de ejecución ha sido en todos los casos en torno a 90 y 110 minutos. Sin embargo, en los pares en los que se enfrentaban dos agentes de MCTS con distinta política de selección, el tiempo de ejecución ha sido algo inferior a los 15 minutos.

Además, Go es un juego famoso por tener una combinatoria explosiva, de forma que, para tableros más grandes, puede llegar a ser más complejo incluso que el ajedrez. Esta es precisamente una de las razones por las que Go ha representado durante décadas un gran desafío para la inteligencia artificial. Un programa de IA desarrollado por DeepMind llamado *AlphaGo* es especialmente relevante en este contexto. Este programa combina redes neuronales profundas, búsqueda mediante Monte Carlo Tree Search y aprendizaje por refuerzo para jugar a Go competitivamente en tableros de tamaño profesional (19×19 celdas). En el año 2016, *AlphaGo* jugó contra Lee Sedol, uno de los mejores jugadores del mundo de Go, y le ganó, demostrando que la IA puede superar al ser humano en un juego de gran complejidad estratégica (MSMK The University of Science & Technology, 2024).

Por ello, los resultados obtenidos en este trabajo en el juego Go deben interpretarse en su contexto: el uso de una versión simplificada del juego, con una baja cantidad de simulaciones, y la ausencia de técnicas complementarias como redes neuronales que podrían mejorar considerablemente el rendimiento de los agentes, tal como se demostró con *AlphaGo*. Sin embargo, el estudio de estos algoritmos en entornos controlados y simplificados puede ser útil como primer paso hacia una mayor comprensión de los mecanismos de decisión en IA, y puede servir como base para futuras extensiones en las que se incorporen técnicas más sofisticadas.

En conclusión, si bien el método de Monte Carlo ha mostrado un rendimiento bastante razonable en juegos sencillos como Nim y 4 en raya, en juegos más complejos como Othello y Go, sus limitaciones se han vuelto evidentes. Por un lado, en Othello ha mostrado un rendimiento mucho peor que MCTS, mientras que en Go, el tiempo de ejecución con Monte Carlo se ha vuelto prácticamente inviable. Por ello, en juegos complejos resulta necesario recurrir a algoritmos más avanzados como MCTS, que gracias a su capacidad para guiar la exploración eficientemente por los caminos más prometedores, es capaz de adaptarse mejor a entornos con un gran número de posibles decisiones y una mayor profundidad estratégica, o incluso técnicas más avanzadas como el uso de redes neuronales. Esto pone de manifiesto la importancia de emplear métodos más sofisticados en la toma de decisiones cuando se abordan juegos con alta complejidad estratégica y computacional.

Referencias

- Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256.
- Browne, C., Powley, E., Whitehouse, D., Lucas, S., Cowling, P. I., Rohlfshagen, P., Tave-
ner, S., Perez, D., Samothrakis, S., and Colton, S. (2012). A survey of monte carlo tree
search methods. *IEEE Transactions on Computational Intelligence and AI in Games*,
4(1).
- Gasarch, W. and Purtilo, J. (2015). Nim with cash.
- MSMK The University of Science & Technology (2024). Alphago.
- Osborne, M. J. (2004). *An Introduction to Game Theory*. Oxford University Press, New
Delhi, India.
- Russell, S. J. and Norvig, P. (2022). *Artificial Intelligence: A Modern Approach*. Pearson
Education Limited, Harlow, United Kingdom, fourth, global edition.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT
Press, Cambridge, Massachusetts; London, England, second edition.
- Watson, J. (2013). *Strategy: An Introduction to Game Theory*. W. W. Norton Company,
New York, NY, third edition.

A. Abreviaturas

- **bit:** Binary Digit
- **CPU:** Central Processing Unit
- **GB:** Gigabyte
- **GHz:** Gigahertz
- **GPU:** Graphics Processing Unit
- **HP:** Hewlett-Packard
- **IA:** Inteligencia Artificial
- **ID:** Identificador
- **MC:** Monte Carlo
- **MCTS:** Monte Carlo Tree Search
- **NE:** Nash Equilibrium
- **SPE:** Subgame Perfect Equilibrium
- **SPNE:** Subgame Perfect Nash Equilibrium
- **UCB:** Upper Confidence Bounce

B. Detalles de la implementación en Python

Con el objetivo de estudiar y comparar los distintos métodos de simulación explicados en la sección 2.2 aplicados a juegos de estrategia, se ha desarrollado una implementación en Python que permite experimentar con dichos algoritmos. La estructura general de la implementación ya ha sido presentada en la sección 1.4.

B.1. Implementación de los juegos

En este apartado se muestra la implementación en Python de los juegos Nim, 4 en raya, Othello y Go, que se encuentran en el directorio `games/`.

B.1.1.1. Nim

Las reglas básicas del juego Nim han sido explicadas en la sección 3.1. La clase `NimGame` se encarga de modelar el juego. A continuación se presenta una breve descripción de cada uno de los métodos que la componen:

- `__init__(self, initial_sticks=21, max_sticks_per_turn=4, starting_player=1)`: Se utiliza para inicializar el juego. Sus parámetros incluyen:
 - `initial_sticks`: número de palos iniciales.
 - `max_sticks_per_turn`: número máximo de palos que un jugador puede retirar por turno.
 - `starting_player`: el jugador que empieza el juego (1 o -1).
- `valid_actions(self)`: Este método devuelve las acciones válidas que un jugador puede realizar, es decir, el número de palos que puede retirar.
- `action(self, amount)`: Este método simula un turno del juego, restando el número especificado de palos a los palos restantes y pasándole el turno al jugador siguiente, y devuelve el nuevo estado del juego. Además, realiza una comprobación para asegurarse de que la acción solicitada sea válida.
- `terminal(self)`: Verifica si el juego ha terminado, esto es, si no quedan palos.
- `winner(self)`: Devuelve quién es el ganador. Si el juego ha terminado, devuelve el número del jugador ganador (1 o -1), pero si no ha terminado devuelve 0.
- `draw(self)`: Este método dibuja el estado actual del juego, imprimiendo cuántos palos quedan.

A continuación, se muestra el código de la clase `NimGame` definida en el archivo `nim.py`:

```
class NimGame:
    def __init__(self, initial_sticks=21, max_sticks_per_turn=4, starting_player=1):
        """
        Inicializa el juego

        Parámetros:
        - initial_sticks: El número de palos al inicio del juego (por defecto 21)
        - max_sticks_per_turn: El número máximo de palos que se pueden retirar en un
        turno (por defecto 4)
        - starting_player: El jugador que inicia el juego, 1 o -1 (por defecto 1)
        """
        assert starting_player in (1, -1), "El jugador inicial debe ser 1 o -1."
```

```

self.sticks = initial_sticks
self.max_sticks_per_turn = max_sticks_per_turn
self.turn = starting_player
self.eval = [] # Evaluaciones de las simulaciones

def valid_actions(self):
    """
    Devuelve una lista con las acciones válidas
    """
    return [i for i in range(1, min(self.max_sticks_per_turn, self.sticks) + 1)] if self.sticks > 0 else []

def action(self, amount):
    """
    Realiza un turno del juego, restando el número especificado de palos
    """
    if amount not in self.valid_actions():
        raise ValueError(f"Acción inválida: solo puedes retirar entre 1 y {self.max_sticks_per_turn} palos, "
                        "y no más de los que quedan.")

    # Devolver un nuevo estado del juego con los valores actualizados
    new_game = NimGame(self.sticks - amount, self.max_sticks_per_turn, -self.turn)
    new_game.eval = self.eval[:] # Copiar el historial de evaluaciones
    return new_game

def terminal(self):
    """
    Devuelve True si el juego ha terminado (cuando no quedan palos)
    """
    return self.sticks == 0

def winner(self):
    """
    Devuelve el ganador: 1 para el jugador 1, -1 para el jugador 2
    """
    return -self.turn if self.terminal() else 0

def draw(self):
    """
    Dibuja el estado actual del juego
    """
    if self.sticks == 1:
        print(f"Queda {self.sticks} palo.")
    else:
        print(f"Quedan {self.sticks} palos.")

```

B.1.2. 4 en raya

Las reglas básicas del juego 4 en raya han sido explicadas en la sección 3.2. La clase `Connect4Game` se encarga de modelar el juego. A continuación se presenta una breve

descripción de cada uno de los métodos que la componen:

- `__init__(self, rows=6, cols=7, starting_player=1)`: Se utiliza para inicializar el juego. Inicializa el tablero de tamaño `rows` x `cols` con casillas vacías (0). Sus parámetros incluyen:
 - `rows`: número de filas del tablero (de 4 a 10).
 - `cols`: número de columnas del tablero (de 4 a 10).
 - `starting_player`: el jugador que empieza el juego (1 o -1).
- `valid_actions(self)`: Este método devuelve las acciones válidas que un jugador puede realizar, es decir, las columnas donde se pueden colocar fichas porque no están llenas.
- `action(self, col)`: Este método simula un turno del juego, encontrando la primera fila vacía en la columna seleccionada y colocando la ficha del jugador en esa posición, y devuelve el nuevo estado del juego. Además, realiza una comprobación para asegurarse de que la acción solicitada sea válida.
- `terminal(self)`: Verifica si el juego ha terminado, esto es, si un jugador ha alineado 4 fichas consecutivas, o si el tablero está lleno (lo que resulta en un empate).
- `winner(self)`: Devuelve quién es el ganador. Si el juego ha terminado, devuelve el número del jugador ganador (1 o -1) o un 0 si ha habido empate, pero si no ha terminado devuelve 0.
- `draw(self)`: Este método dibuja el estado actual del juego, imprimiendo el tablero en formato de matriz, donde cada celda representa una ficha (1, 2, o 0 si está vacía).

A continuación, se muestra el código de la clase `Connect4Game` definida en el archivo `connect4.py`:

```
class Connect4Game:
    def __init__(self, rows=6, cols=7, starting_player=1):
        """
        Inicializa el juego

        Parámetros:
        - rows: El número de filas del tablero del juego (por defecto 6, puede tomar valores entre 4 y 10)
        - cols: El número de columnas del tablero del juego (por defecto 7, puede tomar valores entre 4 y 10)
        - starting_player: El jugador que inicia el juego, 1 o -1 (por defecto 1)
```

```

"""
assert 4 <= rows <= 10, "El número de filas debe estar entre 4 y 10."
assert 4 <= cols <= 10, "El número de columnas debe estar entre 4 y 10."
assert starting_player in (1, -1), "El jugador inicial debe ser 1 o -1."

self.rows = rows
self.cols = cols
self.board = [[0 for _ in range(cols)] for _ in range(rows)] # Definir el
tablero
self.turn = starting_player
self.eval = [] # Evaluaciones de las simulaciones

def valid_actions(self):
    """
    Devuelve una lista con las columnas válidas donde se puede colocar una ficha
    """
    valid_moves = []

    for col in range(self.cols): # Iterar por cada columna
        # Encontrar la primera fila vacía en esa columna
        for row in range(self.rows-1, -1, -1): # Iterar por cada fila de abajo hacia
arriba
            if self.board[row][col] == 0: # Celda vacía
                valid_moves.append(col)
                break # No hace falta seguir buscando, ya se ha encontrado una fila
vacía en esa columna
    return valid_moves

def action(self, col):
    """
    Realiza un turno del juego
    """
    # Verificar si la columna es válida
    if col not in self.valid_actions():
        raise ValueError(f"Acción inválida: la columna {col + 1} está llena. Por
favor, elige una columna con espacio disponible.")

    # Crear una copia del tablero actual para el nuevo juego
    new_board = [row[:] for row in self.board]

    # Colocar la ficha en la primera fila vacía de la columna seleccionada
    for row in range(self.rows-1, -1, -1): # Iterar por cada fila de abajo hacia
arriba
        # Encontrar la primera celda vacía desde abajo y colocar la ficha
        if new_board[row][col] == 0:
            new_board[row][col] = self.turn
            break

    # Crear un nuevo juego con el tablero actualizado
    new_game = Connect4Game(self.rows, self.cols, -self.turn)
    new_game.board = new_board # Asignar el nuevo tablero
    new_game.eval = self.eval[:] # Copiar el historial de evaluaciones
    return new_game

```

```

def terminal(self):
    """
    Devuelve True si el juego ha terminado (cuando un jugador alinee 4 fichas
    consecutivas)
    """
    # Si hay un ganador
    if self.winner() != 0:
        return True

    # Si el tablero está lleno (empate)
    if all(self.board[0][col] != 0 for col in range(self.cols)):
        return True

    # Si el juego no ha terminado
    return False

def winner(self):
    """
    Devuelve el ganador: 1 para el jugador 1, -1 para el jugador 2
    """
    # Comprobar si hay 4 fichas consecutivas del mismo jugador
    for row in range(self.rows):
        for col in range(self.cols):

            # Comprobar si la celda actual contiene una ficha
            if self.board[row][col] in (1, -1):

                # Obtener el jugador de la ficha en la celda actual
                player = self.board[row][col]

                # En horizontal
                if col + 4 <= self.cols:      # Verificar si hay espacio suficiente a
la derecha
                    if self.board[row][col] == self.board[row][col+1] == self.board[
row][col+2] == self.board[row][col+3]:
                        return player

                # En vertical
                if row + 4 <= self.rows:      # Verificar si hay espacio suficiente
abajo
                    if self.board[row][col] == self.board[row+1][col] == self.board[
row+2][col] == self.board[row+3][col]:
                        return player

                # En diagonal descendente
                if row + 4 <= self.rows and col + 4 <= self.cols:      # Verificar si
hay espacio suficiente en la diagonal
                    if self.board[row][col] == self.board[row+1][col+1] == self.board
[row+2][col+2] == self.board[row+3][col+3]:
                        return player

                # En diagonal ascendente
                if row > 3 and col + 4 <= self.cols:      # Verificar si hay espacio
suficiente en la diagonal

```

```

        if self.board[row][col] == self.board[row-1][col+1] == self.board
[ row-2][ col+2] == self.board[ row-3][ col+3]:
            return player

    # Si no se encuentran 4 fichas consecutivas del mismo jugador
    return 0

def draw(self):
    """
    Dibuja el estado actual del juego
    """
    player = {0: 0, 1: 1, -1: 2}
    for row in self.board:
        player_row = []
        for i in row:
            player_row.append(player[i])
        print(player_row)

```

B.1.3. Othello

Las reglas básicas del juego Othello han sido explicadas en la sección 3.3. La clase `OthelloGame` se encarga de modelar el juego. A continuación se presenta una breve descripción de cada uno de los métodos que la componen:

- `__init__(self, rows=8, cols=8, starting_player=1)`: Se utiliza para inicializar el juego. Inicializa el tablero de tamaño `rows` x `cols` con casillas vacías (0). Sus parámetros incluyen:
 - `rows`: número de filas del tablero (de 4 a 10, debe ser par).
 - `cols`: número de columnas del tablero (de 4 a 10, debe ser par).
 - `starting_player`: el jugador que empieza el juego (1 o -1).
- `comprobar_limites(self, row, col)`: Este método verifica si la posición (`row`, `col`) está dentro de los límites del tablero.
- `fichas_a_voltrear_en_direccion(self, row, col, d_row, d_col)`: Determina qué fichas se deben voltear en una dirección específica (vertical, horizontal o diagonal) definida por `d_row` y `d_col`. Estas fichas se corresponden con las fichas del oponente que están entre la ficha colocada y otra ficha propia.
- `valid_actions(self)`: Este método devuelve las acciones válidas que un jugador puede realizar. Recorre el tablero y verifica para cada casilla vacía si colocar una ficha allí es una jugada válida, esto es, si hay al menos una ficha del oponente que

puede ser encerrada entre la ficha colocada y otra ficha del mismo jugador. Las direcciones posibles son: arriba, abajo, izquierda, derecha, y las 4 diagonales.

- **action(self, coordinates):** Este método simula un turno del juego en la posición dada por las coordenadas `coordinates`, de la forma `(row, col)`. Coloca la ficha en la casilla indicada y voltea las fichas del oponente que hayan sido encerradas. Finalmente, devuelve el nuevo estado del juego con el tablero actualizado y el turno del siguiente jugador. Además, realiza una comprobación para asegurarse de que la acción solicitada sea válida.
- **terminal(self):** Verifica si el juego ha terminado, esto es, si ninguno de los dos jugadores tiene jugadas válidas.
- **winner(self):** Devuelve quién es el ganador (el que tiene más fichas en el tablero al finalizar la partida). Si el juego ha terminado, devuelve el número del jugador ganador (1 o -1) o un 0 si ha habido empate, pero si no ha terminado devuelve 0.
- **draw(self):** Este método dibuja el estado actual del juego, imprimiendo el tablero en formato de matriz, donde cada celda representa una ficha (1, 2, o 0 si está vacía).

A continuación, se muestra el código de la clase `OthelloGame` definida en el archivo `othello.py`:

```
class OthelloGame:
    def __init__(self, rows=8, cols=8, starting_player=1):
        """
        Inicializa el juego

        Parámetros:
        - rows: El número de filas del tablero del juego (por defecto 8, puede tomar
valores entre 4 y 10)
        - cols: El número de columnas del tablero del juego (por defecto 8, puede tomar
valores entre 4 y 10)
        - starting_player: El jugador que inicia el juego, 1 o -1 (por defecto 1)
        """
        assert 4 <= rows <= 10, "El número de filas debe estar entre 4 y 10."
        assert 4 <= cols <= 10, "El número de columnas debe estar entre 4 y 10."
        assert rows % 2 == 0, "El número de filas debe ser par."
        assert cols % 2 == 0, "El número de columnas debe ser par."
        assert starting_player in (1, -1), "El jugador inicial debe ser 1 o -1."

        self.rows = rows
        self.cols = cols
        self.board = [[0 for _ in range(cols)] for _ in range(rows)] # Definir el
tablero

        # Fichas iniciales en el centro
```

```

self.board[rows // 2 - 1][cols // 2 - 1] = -starting_player # Blanca
self.board[rows // 2 - 1][cols // 2] = starting_player # Negra
self.board[rows // 2][cols // 2 - 1] = starting_player # Negra
self.board[rows // 2][cols // 2] = -starting_player # Blanca

self.turn = starting_player
self.eval = [] # Evaluaciones de las simulaciones

def comprobar_limites(self, row, col):
    """
    Verifica los límites del tablero
    """
    return 0 <= row < self.rows and 0 <= col < self.cols

def fichas_a_voltar_en_direccion(self, row, col, d_row, d_col):
    """
    Devuelve una lista de fichas que se deben voltear en una dirección dada
    """
    fichas_a_voltar = []

    # Coordenadas de la siguiente casilla en esa dirección
    new_row = row + d_row
    new_col = col + d_col

    # Seguir en esa dirección mientras las casillas sigan ocupadas por el oponente
    while self.comprobar_limites(new_row, new_col) and self.board[new_row][new_col] == - self.turn:
        fichas_a_voltar.append((new_row, new_col))

        # Coordenadas de la siguiente casilla en esa dirección
        new_row += d_row
        new_col += d_col

    # Comprobar si hay una ficha del propio jugador que cierra la secuencia
    if self.comprobar_limites(new_row, new_col) and self.board[new_row][new_col] == self.turn:
        return fichas_a_voltar
    else:
        return [] # No se puede voltear nada en esta dirección

def valid_actions(self):
    """
    Devuelve una lista con las acciones válidas
    """
    valid_moves = []

    # Se recorren las casillas del tablero
    for row in range(self.rows):
        for col in range(self.cols):

            # Para que una casilla sea válida debe estar vacía
            if self.board[row][col] == 0:

                # Direcciones posibles

```

```

        directions = [(-1,0), (1,0), (0,-1), (0,1), (-1,-1), (1,-1), (-1,1),
(1,1)]

        for (d_row, d_col) in directions:

            # Coordenadas de la siguiente casilla en esa dirección
            new_row = row + d_row
            new_col = col + d_col

            # Verificar los límites del tablero
            if self.comprobar_lmites(new_row, new_col):

                # Comprobar que la primera casilla en esa dirección está
ocupada por el oponente
                if self.board[new_row][new_col] == - self.turn:

                    # Avanzar a la siguiente casilla en esa dirección
                    new_row += d_row
                    new_col += d_col

                    # Seguir en esa dirección mientras las casillas sigan
ocupadas por el oponente
                    while self.comprobar_lmites(new_row, new_col) and self.
board[new_row][new_col] == - self.turn:
                        new_row += d_row
                        new_col += d_col

                        # Comprobar que la siguiente casilla en esa dirección est
é ocupada por el propio jugador
                        if self.comprobar_lmites(new_row, new_col) and self.
board[new_row][new_col] == self.turn:
                            valid_moves.append((row, col))
                            break

        return valid_moves

def action(self, coordinates):
    """
    Realiza un turno del juego
    """
    if coordinates not in self.valid_actions():
        raise ValueError(f"Acción inválida: la ficha debe colocarse en una casilla
vacía de modo que capture "
                        "al menos una ficha del oponente")

    row, col = coordinates

    # Crear una copia del tablero actual para el nuevo juego
    new_board = [row[:] for row in self.board]

    # Colocar la ficha en la casilla seleccionada
    new_board[row][col] = self.turn

    # Direcciones posibles
    directions = [(-1,0), (1,0), (0,-1), (0,1), (-1,-1), (1,-1), (-1,1), (1,1)]

```

```
for (d_row, d_col) in directions:
    fichas_a_voltar = self.fichas_a_voltar_en_direccion(row, col, d_row, d_col)
    for f_row, f_col in fichas_a_voltar:
        new_board[f_row][f_col] = self.turn # Voltar la ficha

# Devolver un nuevo estado del juego con los valores actualizados
new_game = OthelloGame(self.rows, self.cols, -self.turn)
new_game.board = new_board # Asignar el nuevo tablero
new_game.eval = self.eval[:] # Copiar el historial de evaluaciones

# Comprobar si el nuevo jugador puede jugar
if not new_game.valid_actions():

    # Si no puede, comprobar si el jugador actual sí podía seguir jugando
    if self.valid_actions():
        new_game.turn = self.turn # El turno no cambia

    # Si el jugador actual tampoco tiene jugadas válidas
    else:
        new_game.turn = -self.turn # Pasar el turno al oponente

return new_game

def terminal(self):
    """
    Devuelve True si el juego ha terminado
    """
    if self.valid_actions():
        return False # El jugador actual puede jugar

    # Simular el turno del oponente y ver si él puede jugar
    temp_game = OthelloGame(self.rows, self.cols, -self.turn)
    temp_game.board = [row[:] for row in self.board]

    # Si el oponente tampoco tiene jugadas válidas, termina el juego
    if not temp_game.valid_actions():
        return True
    else:
        return False

def winner(self):
    """
    Devuelve el ganador: 1 para el jugador 1, -1 para el jugador 2
    """
    # Si no ha acabado el juego, no hay ganador
    if not self.terminal():
        return 0

    count_1 = 0
    count_2 = 0

    for row in range(self.rows):
        for col in range(self.cols):
```



```

        # Casillas ocupadas por el jugador 1
        if self.board[row][col] == 1:
            count_1 += 1

        # Casillas ocupadas por el jugador 2
        elif self.board[row][col] == -1:
            count_2 += 1

    # Determinar el ganador
    if count_1 > count_2:
        return 1 # Gana el jugador 1
    elif count_1 < count_2:
        return -1 # Gana el jugador 2
    elif count_1 == count_2:
        return 0 # Empate

def draw(self):
    """
    Dibuja el estado actual del juego
    """
    player = {0: 0, 1: 1, -1: 2}
    for row in self.board:
        player_row = []
        for i in row:
            player_row.append(player[i])
        print(player_row)

```

B.1.4. Go

Las reglas básicas del juego Go han sido explicadas en la sección 3.4. La clase `GoGame` se encarga de modelar el juego. A continuación se presenta una breve descripción de cada uno de los métodos que la componen:

- `__init__(self, rows=9, cols=9, starting_player=1)`: Se utiliza para inicializar el juego. Inicializa el tablero de tamaño `rows` x `cols` con casillas vacías (0), además de un historial de movimientos, un contador de pases y los contadores de puntuación para cada jugador. Además, establece el jugador inicial (1 o -1) y asigna la compensación Komi al jugador que no inicia la partida para equilibrar la ventaja del otro jugador. Sus parámetros incluyen:
 - `rows`: número de filas del tablero (de 5 a 19, debe ser impar).
 - `cols`: número de columnas del tablero (de 5 a 19, debe ser impar).
 - `starting_player`: el jugador que empieza el juego (1 o -1).
- `comprobar_limites(self, row, col)`: Este método verifica si la posición (`row`, `col`) está dentro de los límites del tablero.

- **find_group(self, board, row, col, turn):** Determina qué fichas forman parte del grupo conectado a la ficha en (row, col) del jugador turn. Este grupo se obtiene explorando en 4 direcciones (arriba, abajo, derecha, izquierda).
- **has_liberties(self, board, group):** Comprueba si un grupo de fichas tiene libertades, es decir, si existe al menos una casilla vacía adyacente a alguna ficha del grupo. Si no tiene libertades, entonces el grupo está capturado.
- **eliminate_enemy_groups(self, board, row, col):** Identifica los grupos enemigos adyacentes que no tengan libertades después de colocar una ficha en (row, col) y los elimina. Las fichas eliminadas se suman a la puntuación del jugador que realiza la captura.
- **valid_actions(self):** Este método devuelve las acciones válidas en el estado actual del tablero. Una jugada es válida si la casilla está vacía, no incumple la regla del suicidio (esto es, que el grupo resultante al realizar la jugada tenga al menos una libertad y, por tanto, no quede capturado), y cumple la regla de Situational Superko (el tablero resultante no repite un estado anterior).
- **action(self, coordinates):** Este método simula un turno del juego en la posición dada por las coordenadas coordinates, de la forma (row, col). Si se indica una posición válida, coloca una ficha del jugador actual, elimina los grupos enemigos que se queden sin libertades, y genera un nuevo estado del juego cambiando el turno al jugador contrario. Si no se indican unas coordenadas, se pasa el turno al siguiente jugador.
- **terminal(self):** Verifica si el juego ha terminado, lo cual sucede cuando ambos jugadores pasan turno de forma consecutiva.
- **winner(self):** Devuelve quién es el ganador de la partida. Si el juego ha terminado, devuelve el número del jugador ganador (1 o -1) o un 0 si ha habido empate, pero si no ha terminado devuelve 0. La puntuación final de cada jugador se calcula sumando:
 - El número de fichas en el tablero de dicho jugador.
 - Las fichas capturadas al oponente.
 - El territorio rodeado exclusivamente por fichas del color de dicho jugador.
 - La compensación Komi de 6.5 puntos para el jugador que no comenzó.

- **draw(self):** Este método dibuja el estado actual del juego, imprimiendo el tablero en formato de matriz, donde cada celda representa una ficha (1, 2, o 0 si está vacía).

A continuación, se muestra el código de la clase **GoGame** definida en el archivo **go.py**:

```
from copy import deepcopy

class GoGame:
    def __init__(self, rows=9, cols=9, starting_player=1):
        """
        Inicializa el juego

        Parámetros:
        - rows: El número de filas del tablero del juego (por defecto 9, puede tomar
        valores entre 5 y 19)
        - cols: El número de columnas del tablero del juego (por defecto 9, puede tomar
        valores entre 5 y 19)
        - starting_player: El jugador que inicia el juego, 1 o -1 (por defecto 1)
        """
        assert 5 <= rows <= 19, "El número de filas debe estar entre 5 y 19."
        assert 5 <= cols <= 19, "El número de columnas debe estar entre 5 y 19."
        assert rows % 2 == 1, "El número de filas debe ser impar."
        assert cols % 2 == 1, "El número de columnas debe ser impar."
        assert rows == cols, "El número de filas y el de columnas deben ser iguales."
        assert starting_player in (1, -1), "El jugador inicial debe ser 1 o -1."

        self.rows = rows
        self.cols = cols
        self.board = [[0 for _ in range(cols)] for _ in range(rows)] # Definir el
        tablero
        self.turn = starting_player
        self.eval = [] # Evaluaciones de las simulaciones
        self.pass_count = 0 # Contador de pases consecutivos
        self.history = [(deepcopy(self.board), self.turn)] # Historial con información
        de los turnos anteriores

        # Compensación Komi para equilibrar que un jugador juegue primero que el otro
        if starting_player == 1:
            self.count_1 = 0 # Contador de puntos para el jugador 1 (1)
            self.count_2 = 6.5 # Contador de puntos para el jugador 2 (-1)
        elif starting_player == -1:
            self.count_1 = 6.5 # Contador de puntos para el jugador 1 (1)
            self.count_2 = 0 # Contador de puntos para el jugador 2 (-1)

    def comprobar_limites(self, row, col):
        """
        Verifica los límites del tablero
        """
        return 0 <= row < self.rows and 0 <= col < self.cols

    def find_group(self, board, row, col, turn):
        """
        Encuentra todas las fichas conectadas al grupo de la ficha colocada en (row, col)
        """
```

```

group = []
visited = set()
stack = [(row, col)] # Comenzamos desde la casilla donde se colocó la ficha

while stack:
    current_row, current_col = stack.pop()

    if (current_row, current_col) not in visited:
        visited.add((current_row, current_col))
        group.append((current_row, current_col))

    # Revisar las casillas adyacentes
    for d_row, d_col in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        new_row = current_row + d_row
        new_col = current_col + d_col

        if self.comprobar_limites(new_row, new_col) and board[new_row][
new_col] == turn:
            stack.append((new_row, new_col))

    return group

def has_liberties(self, board, group):
    """
    Verifica si el grupo tiene libertades (casillas vacías adyacentes)
    """
    for row, col in group:

        # Direcciones posibles
        directions = [(-1,0), (1,0), (0,-1), (0,1)]
        for d_row, d_col in directions:

            # Coordenadas de la casilla adyacente en esa dirección
            new_row = row + d_row
            new_col = col + d_col

            if self.comprobar_limites(new_row, new_col) and board[new_row][new_col]
== 0:
                return True # Si hay una casilla vacía adyacente, el grupo tiene
libertades

        return False # Si no hay casillas vacías adyacentes, el grupo no tiene
libertades

def eliminate_enemy_groups(self, board, row, col, update_score=True):
    """
    Identifica los grupos enemigos adyacentes sin libertades y los elimina del
    tablero
    """
    # Direcciones posibles
    directions = [(-1,0), (1,0), (0,-1), (0,1)]
    for d_row, d_col in directions:

        # Coordenadas de la casilla adyacente en esa dirección

```

```

        new_row = row + d_row
        new_col = col + d_col

        # Buscar grupos formados por fichas del oponente
        if self.comprobar_limites(new_row, new_col) and board[new_row][new_col] == -
self.turn:
            enemy_group = self.find_group(board, new_row, new_col, -self.turn)

            # Si el grupo enemigo no tiene libertades, se elimina
            if not self.has_liberties(board, enemy_group):
                for enemy_row, enemy_col in enemy_group:
                    board[enemy_row][enemy_col] = 0

            # Se suman las puntuaciones por fichas enemigas capturadas
            if update_score:
                if self.turn == 1:
                    self.count_1 += 1
                elif self.turn == -1:
                    self.count_2 += 1

        return board

def valid_actions(self):
    """
    Devuelve una lista con las acciones válidas
    """
    valid_moves = []

    # Se recorren las casillas del tablero
    for row in range(self.rows):
        for col in range(self.cols):

            # Para que una casilla sea válida debe estar vacía
            if self.board[row][col] == 0:

                # Simular la jugada: colocar la ficha en la casilla seleccionada
                temp_board = deepcopy(self.board)
                temp_board[row][col] = self.turn

                # Eliminar grupos enemigos sin libertades
                temp_board = self.eliminate_enemy_groups(temp_board, row, col,
update_score=False)

                # Encontrar todas las fichas del grupo asociado a la ficha
seleccionada
                group = self.find_group(temp_board, row, col, self.turn)

                # Regla del suicidio: Comprobar si el grupo tiene libertades (si no
las tiene, el movimiento no es válido)
                if self.has_liberties(temp_board, group):

                    # Regla de Situational Superko
                    superko = False
                    for (prev_board, prev_turn) in self.history:

```

```

        if temp_board == prev_board and self.turn == prev_turn:
            superko = True
            break

        # Si no se infrinje la regla de Situational Superko
        if not superko:
            valid_moves.append((row, col))

    # En Go siempre se puede pasar, así que se añade como acción válida
    valid_moves.append(None)

    return valid_moves

def action(self, coordinates):
    """
    Realiza un turno del juego
    """
    if coordinates is None:
        # Pase: no se coloca ficha ni se modifica el tablero
        new_game = GoGame(self.rows, self.cols, -self.turn)
        new_game.board = deepcopy(self.board)
        new_game.eval = self.eval[:]
        new_game.pass_count = self.pass_count + 1
        new_game.count_1 = self.count_1
        new_game.count_2 = self.count_2

        # Actualizar el historial
        if (new_game.board, -self.turn) not in self.history:
            new_game.history = self.history[:] + [(deepcopy(new_game.board), -self.
turn)]
        else:
            new_game.history = self.history[:]

        return new_game

    if coordinates not in self.valid_actions():
        raise ValueError("Acción inválida: la jugada no es legal según las reglas de
Go "

                            "(puede ser casilla ocupada, suicidio o Superko).")

    row, col = coordinates

    # Crear una copia del tablero actual para el nuevo juego
    new_board = deepcopy(self.board)

    # Colocar la ficha en la casilla seleccionada
    new_board[row][col] = self.turn

    # Eliminar grupos enemigos sin libertades
    new_board = self.eliminate_enemy_groups(new_board, row, col)

    # Devolver un nuevo estado del juego con los valores actualizados
    new_game = GoGame(self.rows, self.cols, -self.turn)
    new_game.board = new_board    # Asignar el nuevo tablero

```

```

new_game.eval = self.eval[:] # Copiar el historial de evaluaciones
new_game.pass_count = 0      # Reiniciar contador de pases
new_game.count_1 = self.count_1
new_game.count_2 = self.count_2

# Actualizar el historial
if (new_board, -self.turn) not in self.history:
    new_game.history = self.history[:] + [(deepcopy(new_board), -self.turn)]
else:
    new_game.history = self.history[:]

return new_game

def terminal(self):
    """
    Devuelve True si el juego ha terminado: cuando ambos jugadores han pasado de
    forma consecutiva
    """
    return self.pass_count >= 2

def evaluate_territory(self, row, col, visited):
    """
    Explora la región vacía conectada a (row, col) y determina si pertenece a un solo
    jugador
    """
    stack = [(row, col)]
    territory = []
    border_players = set()

    while stack:
        row, col = stack.pop()
        if (row, col) in visited:
            continue
        visited.add((row, col))
        territory.append((row, col))

        # Revisar vecinos
        directions = [(-1,0), (1,0), (0,-1), (0,1)]
        for d_row, d_col in directions:
            n_row = row + d_row
            n_col = col + d_col

            if not self.comprobar_limites(n_row, n_col):
                # El territorio toca el borde del tablero: neutral
                border_players.add(0)
                continue

            if self.board[n_row][n_col] == 0:
                # Casilla vacía, agregar para explorar
                if (n_row, n_col) not in visited:
                    stack.append((n_row, n_col))
            else:
                # Casilla con piedra negra (1) o blanca (-1)
                border_players.add(self.board[n_row][n_col])

```

```
# Si el territorio está rodeado solo por un color (no 0 ni ambos)
if len(border_players) == 1:
    owner = border_players.pop()
else:
    owner = 0 # Neutral o compartido

return territory, owner

def winner(self):
    """
    Devuelve el ganador: 1 para el jugador 1, -1 para el jugador 2
    """
    # Si no ha acabado el juego, no hay ganador
    if not self.terminal():
        return 0

    visited = set()

    count_1_territory = 0
    count_2_territory = 0

    for row in range(self.rows):
        for col in range(self.cols):
            if self.board[row][col] == 1:
                count_1_territory += 1
            elif self.board[row][col] == -1:
                count_2_territory += 1
            elif self.board[row][col] == 0 and (row, col) not in visited:
                territory, owner = self.evaluate_territory(row, col, visited)
                if owner == 1:
                    count_1_territory += len(territory)
                elif owner == -1:
                    count_2_territory += len(territory)

    final_score_1 = self.count_1 + count_1_territory
    final_score_2 = self.count_2 + count_2_territory

    # Determinar el ganador
    if final_score_1 > final_score_2:
        return 1 # Gana el jugador 1
    elif final_score_1 < final_score_2:
        return -1 # Gana el jugador 2
    elif final_score_1 == final_score_2:
        return 0 # Empate

def draw(self):
    """
    Dibuja el estado actual del juego
    """
    player = {0: 0, 1: 1, -1: 2}
    for row in self.board:
        player_row = []
        for i in row:
```



```

        player_row.append(player[i])
    print(player_row)

```

B.2. Implementación de los métodos para la toma de decisiones

En este apartado se muestra la implementación en Python de los métodos de Monte Carlo y de Monte Carlo Tree Search, que se encuentran en el directorio `methods/`.

B.2.1. Monte Carlo

El método de Monte Carlo, explicado en la sección 2.2.1, ha sido implementado en el archivo `monte_carlo.py` como se muestra a continuación:

```

import numpy as np

def rollout(root, n):
    """
    Simulación de Rollout (partida completa moviendo al azar)

    Parámetros:
    - root: Estado inicial del juego
    - n: Número de simulaciones
    """
    # Contadores de partidas ganadas por cada jugador
    P = 0
    N = 0
    for _ in range(n):
        g = root
        while not g.terminal(): # El juego no ha terminado
            # Obtener las acciones válidas
            valid_actions = g.valid_actions()

            # Seleccionar un índice aleatorio
            index = np.random.choice(range(len(valid_actions)))

            # Seleccionar la acción correspondiente al índice
            action = valid_actions[index]

            # Ejecutar la acción
            g = g.action(action)

        # Comprobar qué jugador ganó el juego
        if g.winner() > 0:
            P += 1
        if g.winner() < 0:
            N += 1
    return P, N, n

```

```

def MCPlay(g, n):
    """
    Algoritmo de Monte-Carlo para elegir el mejor movimiento en un estado

    Parámetros:
    - g: Estado actual del juego
    - n: Número de simulaciones para cada posible movimiento
    """
    acts = g.valid_actions()      # Acciones posibles
    pn = 0 if g.turn == 1 else 1  # Seleccionar al jugador

    # Simular los juegos para cada acción y almacenar las probabilidades de ganar
    probs = [rollout(g.action(a), n)[pn] for a in acts]
    #print(probs)

    # Seleccionar la acción con mayor probabilidad de éxito
    selected = acts[np.argmax(np.array(probs))]
    print(f'La computadora juega {selected}')

    # Ejecutar el movimiento seleccionado
    g = g.action(selected)

    # Evaluar el movimiento
    g.eval.append((-g.turn, max(probs)/n))
    return g

class MCAgent:
    def __init__(self, sims):
        self.sims = sims
    def move(self, game):
        return MCPlay(game, self.sims)

```

B.2.2. Monte Carlo Tree Search

El método de Monte Carlo Tree Search, explicado en la sección 2.2.2, ha sido implementado en el archivo `monte_carlo_tree_search.py` como se muestra a continuación:

```

import random

class Node:
    """
    Clase que representa un nodo en el árbol de búsqueda para un juego.
    Guarda el estado del juego, su relación con otros nodos y estadísticas
    útiles para MCTS.
    """
    def __init__(self, state, parent, action_from_parent=None):
        """
        Inicializa un nodo del árbol de búsqueda.

        Parámetros:
        state: estado actual del juego en este nodo.

```

```

parent: nodo padre (puede ser None para la raíz).
action_from_parent: acción que llevó desde el nodo padre a este nodo.
"""

# Estado del juego en este nodo
self.state = state

# Nodo padre en el árbol
self.parent = parent

# Acción tomada desde el nodo padre para llegar a este nodo
self.action_from_parent = action_from_parent

# Lista de nodos hijos (se llenará cuando se expanda el nodo)
self.children = []

# Acciones que pueden tomarse en este estado (si no es terminal)
if not self.state.terminal():
    self.remaining = state.valid_actions()
else:
    self.remaining = []

# Número de simulaciones que han pasado por este nodo
self.N = 0

# Recompensa acumulada para el jugador 1
self.Q1 = 0

# Recompensa acumulada para el jugador 2
self.Q2 = 0

class MCIS:
    def __init__(self, root_node: Node, selection_algorithm_class: type, simulations: int
    = 1000, *selection_algorithm_args, **selection_algorithm_kwargs):
        """
        Inicializa el algoritmo Monte-Carlo Tree Search.

        :param root_node: Nodo raíz del árbol.
        :param selection_algorithm_class: Clase del algoritmo de selección.
        :param simulations: Número de simulaciones.
        :param selection_algorithm_args: Argumentos adicionales para inicializar el
        algoritmo de selección.
        :param selection_algorithm_kwargs: Argumentos con nombre para inicializar el
        algoritmo de selección.
        """
        self.root_node = root_node
        self.simulations = simulations

        # Inicializar el algoritmo de selección
        self.selection_algorithm = selection_algorithm_class(self.root_node, *
        selection_algorithm_args, **selection_algorithm_kwargs)

    def selection(self):
        """

```

```

    Realiza la fase de selección de MCTS: se parte del nodo raíz y se seleccionan
    nodos hijos sucesivamente hasta llegar a un nodo hoja (nodo que tiene al menos un
    hijo
    potencial al que no se le ha aplicado ninguna simulación todavía).

    :return: El nodo hoja encontrado.
    """
    # Partir del nodo raíz
    current_node = self.root_node

    # Repetir mientras el nodo actual no sea un nodo hoja
    while current_node.children and not current_node.remaining:

        # Actualizar el nodo en el algoritmo de selección
        self.selection_algorithm.set_node(current_node)

        # Usar el algoritmo de selección para elegir un nodo hijo
        current_node = self.selection_algorithm.select_node()

    # Devolver el nodo hoja encontrado
    return current_node

def expansion(self, leaf_node):
    """
    Realiza la fase de expansión de MCTS: A partir del nodo hoja seleccionado, crear
    un nodo hijo
    (movimiento válido desde el nodo hoja).

    :param leaf_node: Nodo hoja seleccionado en la fase de selección.
    :return: El nodo hijo creado.
    """
    # Si el nodo es terminal, devolverlo tal cual
    if leaf_node.state.terminal():
        return leaf_node

    else:
        # Tomar una acción de las restantes en el nodo
        action = leaf_node.remaining.pop()

        # Aplicar la acción sobre el estado actual para obtener el nuevo estado
        state = leaf_node.state.action(action)

        # Crear un nodo hijo a partir de dicha acción
        child_node = Node(state, parent = leaf_node, action_from_parent=action)

        # Añadir el nodo hijo a la lista de hijos
        leaf_node.children.append(child_node)

        # Devolver el nodo hijo
        return child_node

def simulation(self, child_node):
    """

```

```

        Realiza la fase de simulación de MCTS: Completar una jugada aleatoria desde el
        nodo hijo creado.

        :param child_node: Nodo hijo creado en la fase de expansión.
        :return: El ganador del juego.
        """
        # Estado inicial
        state = child_node.state

        # Realizar la simulación hasta alcanzar un estado terminal
        while not state.terminal():
            # Seleccionar una acción aleatoria de entre las acciones válidas para el
            estado actual
            random_index = random.randint(0, len(state.valid_actions()) - 1)    # Índice
            aleatorio
            action = state.valid_actions()[random_index]    # Acción seleccionada
            aleatoriamente

            # Aplicar la acción seleccionada al estado para obtener un nuevo estado
            state = state.action(action)

        # Devolver el resultado del juego
        return state.winner()    # Devuelve el ganador: 1 para el jugador 1, -1 para el
        jugador 2

def backpropagation(self, child_node, winner):
    """
    Realiza la fase de retropropagación de MCTS: Usar los resultados de la simulación
    para actualizar
    información en los nodos, recorriéndolos desde el nodo expandido hasta el nodo raíz.

    :param child_node: Nodo hijo creado en la fase de expansión.
    :param winner: Ganador del juego en la simulación.
    """
    # Se comienza en el nodo expandido
    current_node = child_node

    # Se recorren los nodos desde el nodo expandido hasta el nodo raíz
    while current_node.parent is not None:
        self.selection_algorithm.set_node(current_node.parent)

        # Recompensa: +1 si ganó ese jugador, -1 si perdió
        player_that_moved = -current_node.state.turn    # Jugador que hizo el
        movimiento para llegar a este estado
        reward = 1 if winner == player_that_moved else -1

        # Actualizar la recompensa y las visitas del jugador que movía en ese nodo
        self.selection_algorithm.update(current_node, reward)

        # Subir al nodo padre
        current_node = current_node.parent

def run(self):

```

```

    """
    Ejecuta MCTS por el número de simulaciones especificado.
    """
    for _ in range(self.simulations):
        leaf_node = self.selection()          # Selección
        child_node = self.expansion(leaf_node) # Expansión
        winner = self.simulation(child_node)  # Simulación
        self.backpropagation(child_node, winner) # Retropropagación

def MCTSPlay(game_state, num_simulations, selection_algorithm_class, *
selection_algorithm_args, **selection_algorithm_kwargs):
    """
    Algoritmo de Monte-Carlo Tree Search para elegir el mejor movimiento en un estado.

    Parámetros:
    - game_state: Estado actual del juego.
    - num_simulations: Número de simulaciones.
    - selection_algorithm_class: Clase del algoritmo de selección.
    - selection_algorithm_args: Argumentos adicionales para inicializar el algoritmo de
    selección.
    - selection_algorithm_kwargs: Argumentos con nombre para inicializar el algoritmo de
    selección.
    """
    # Elegir directamente la jugada ganadora si está disponible
    for action in game_state.valid_actions():
        next_state = game_state.action(action)
        if next_state.terminal() and next_state.winner() == game_state.turn:
            return next_state

    # Crear el nodo raíz a partir del estado actual del juego
    root_node = Node(game_state, parent=None)

    # Inicializar una clase MCTS
    mcts = MCTS(root_node, selection_algorithm_class, num_simulations, *
selection_algorithm_args, **selection_algorithm_kwargs)

    # Ejecutar el algoritmo MCTS
    mcts.run()

    # Si no hay jugada ganadora inmediata, elegir la correspondiente al hijo más visitado
    best_child = max(root_node.children, key=lambda c: c.N)
    return best_child.state

class MCTSAgent:
    """
    Agente que elige jugadas usando MCTS.
    """
    def __init__(self, sims, selection_class, *selection_args, **selection_kwargs):
        self.sims = sims
        self.selection_class = selection_class
        self.selection_args = selection_args
        self.selection_kwargs = selection_kwargs

```

```
def move(self, game):
    return MCTSPlay(game, self.sims, self.selection_class, *self.selection_args, **
self.selection_kwargs)
```

Es interesante observar que en esta implementación, en la función `MCTSPlay`, se ha añadido una comprobación manual para detectar jugadas ganadoras inmediatas, y en caso de detectarse alguna, se elige directamente dicha jugada ganadora, en lugar de usar las estadísticas obtenidas por MCTS para elegir el mejor movimiento. Aunque esta estrategia no forma parte del algoritmo MCTS teórico, se ha incluido por cuestiones prácticas.

MCTS es un algoritmo basado en simulaciones, por lo que no garantiza explorar todas las acciones posibles en cada ejecución, especialmente si el número de simulaciones es bajo. Como consecuencia, una jugada ganadora inmediata puede no ser detectada si no es seleccionada aleatoriamente durante la ejecución del algoritmo, lo cual conlleva un riesgo de perder la oportunidad de ganar la partida. De esta manera, se logra una mejora práctica capaz de asegurar el mejor resultado posible en situaciones evidentes y directas.

B.3. Implementación de las políticas de selección para MCTS

En el directorio `selection/` se encuentra la implementación de las distintas políticas de selección para MCTS, explicadas en la sección 2.2.2.1. En el archivo `algorithm.py` se encuentra una clase genérica `Algorithm` para las políticas de selección:

```
import sys
import os
from abc import ABC, abstractmethod

sys.path.append(os.path.abspath("../methods"))

from monte_carlo_tree_search import Node

class Algorithm(ABC):
    def __init__(self, node: Node):
        """
        Inicializa el algoritmo con un nodo.

        :param node: Nodo actual.
        """
        # Nodo actual
        self.node = node

        # Número de nodos hijos = hijos ya expandidos + acciones aún no exploradas
        self.k = len(node.remaining) + len(node.children)

    def set_node(self, node: Node):
        """
```

```

    Cambia de nodo.

    :param node: Nuevo nodo.
    """
    self.node = node

@abstractmethod
def select_node(self) -> Node:
    """
    Selecciona un nodo hijo basado en la política de selección.

    :return: El nodo hijo seleccionado (acción a tomar).
    """
    raise NotImplementedError("Este método debe ser implementado por la subclase.")

def update(self, chosen_node: Node, reward: float):
    """
    Actualiza las recompensas y visitas para el nodo hijo seleccionado.

    :param chosen_node: El nodo hijo seleccionado.
    :param reward: La recompensa obtenida para ese nodo.
    """
    chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

    # Actualización incremental de la recompensa promedio
    # value = value + (reward - value) / n

    chosen_node.Q += (reward - chosen_node.Q) / chosen_node.N # Actualizar
    recompensa promedio

def reset(self):
    """
    Reinicia el estado del algoritmo.
    """
    for child in self.node.children:
        child.N = 0
        child.Q = 0

```

B.3.1. ϵ -greedy

La política de selección ϵ -greedy, explicada en la sección 2.2.2.1.1, ha sido implementada en el archivo `epsilon_greedy.py` como se muestra a continuación:

```

import sys
import os
import numpy as np

sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))

from algorithm import Algorithm
from monte_carlo_tree_search import Node

```



```

class EpsilonGreedy(Algorithm):
    def __init__(self, node: Node, epsilon: float = 0.1):
        """
        Inicializa el algoritmo epsilon-greedy.

        :param node: Nodo actual en el árbol de búsqueda.
        :param epsilon: Probabilidad de exploración (seleccionar un nodo al azar).
        :raises ValueError: Si epsilon no está en [0, 1].
        """
        assert 0 <= epsilon <= 1, "El parámetro epsilon debe estar entre 0 y 1."

        super().__init__(node)
        self.epsilon = epsilon
        self.node_exploration_map = {}

        self.ensure_node_initialized(self.node)

    def ensure_node_initialized(self, node: Node):
        """
        Asegura que el nodo tenga un conjunto de hijos por explorar registrado.
        """
        node_id = id(node)
        if node_id not in self.node_exploration_map:
            self.node_exploration_map[node_id] = list(range(len(node.children)))

    def set_node(self, node: Node):
        """
        Método para cambiar de nodo y reiniciar el conjunto de exploración.
        """
        self.node = node
        self.ensure_node_initialized(node)

    def select_node(self) -> Node:
        """
        Selecciona un nodo hijo basado en la política epsilon-greedy, asegurando que cada
        nodo hijo
        sea seleccionado al menos una vez antes de iniciar la exploración/explotación
        normal.

        :return: El nodo hijo seleccionado.
        """
        if not self.node.children:
            raise ValueError("EpsilonGreedy: no hay nodos hijos disponibles para
            seleccionar.")

        # Comprobar si hay una jugada ganadora inmediata
        for child in self.node.children:
            if child.state.terminal() and child.state.winner() == self.node.state.turn:
                return child # Ejecutar la jugada ganadora directamente

        # Inicialización: asegurar que cada hijo se explore al menos una vez
        node_id = id(self.node)
        unexplored = self.node_exploration_map[node_id]

```

```

    if unexplored:
        # Extraer y eliminar un nodo hijo del conjunto de nodos no explorados
        index = unexplored.pop()
        return self.node.children[index]

    # Exploración: seleccionar un nodo hijo aleatorio
    if np.random.random() < self.epsilon:
        # Selecciona un brazo al azar
        return np.random.choice(self.node.children)

    # Explotación: seleccionar el nodo hijo con el valor Q más alto
    else:
        # Selecciona el brazo con la mayor recompensa acumulada Q
        player_in_turn = self.node.state.turn
        Q_values = [child.Q1 if player_in_turn == 1 else child.Q2 for child in self.
node.children]
        return self.node.children[np.argmax(Q_values)]

def update(self, chosen_node: Node, reward: float):
    """
    Actualiza las recompensas y visitas para el nodo hijo seleccionado.

    :param chosen_node: El nodo hijo seleccionado.
    :param reward: La recompensa obtenida para ese nodo.
    """
    chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

    # Actualizar recompensa promedio
    player_that_moved = -chosen_node.state.turn
    if player_that_moved == 1:
        chosen_node.Q1 += (reward - chosen_node.Q1) / chosen_node.N
    elif player_that_moved == -1:
        chosen_node.Q2 += (reward - chosen_node.Q2) / chosen_node.N

```

B.3.2. UCB1

La política de selección UCB1, explicada en la sección 2.2.2.1.2, ha sido implementada en el archivo `ucb1.py` como se muestra a continuación:

```

import sys
import os
import numpy as np

sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))

from algorithm import Algorithm
from monte_carlo_tree_search import Node

class UCB1(Algorithm):
    def __init__(self, node: Node, c = 1):
        """

```

```

    Inicializa el algoritmo UCB1.

:param node: Nodo actual en el árbol.
:param c: Parámetro de ajuste que controla el grado de exploración.
"""
super().__init__(node)
self.c = c # Parámetro de ajuste que controla el grado de exploración

def set_node(self, node: Node):
    """
    Método para cambiar de nodo.
    """
    self.node = node

def select_node(self) -> Node:
    """
    Selecciona el nodo hijo con el mayor valor UCB1.

    :return: El nodo hijo seleccionado.
    """
    if not self.node.children:
        raise ValueError("UCB1: no hay nodos hijos disponibles para seleccionar.")

    # Comprobar si hay una jugada ganadora inmediata
    for child in self.node.children:
        if child.state.terminal() and child.state.winner() == self.node.state.turn:
            return child # Ejecutar la jugada ganadora directamente

    total_pulls = sum(child.N for child in self.node.children) # Total de
    simulaciones realizadas
    ucb_values = np.zeros(len(self.node.children))

    for i, child in enumerate(self.node.children):
        if child.N == 0: # Si nunca ha sido seleccionado, asignar un valor muy alto
            para exploración
            ucb_values[i] = float('inf')
        else:
            player_in_turn = self.node.state.turn
            Q = child.Q1 if player_in_turn == 1 else child.Q2

            # Fórmula de UCB1:  $Q + c * \sqrt{\log(\text{total}) / N}$ 
            ucb_values[i] = Q + self.c * np.sqrt((np.log(max(total_pulls, 1))) /
            child.N)

    # Seleccionar el nodo con el mayor UCB1
    selected_index = np.argmax(ucb_values)
    return self.node.children[selected_index]

def update(self, chosen_node: Node, reward: float):
    """
    Actualiza las recompensas y visitas para el nodo hijo seleccionado.

    :param chosen_node: El nodo hijo seleccionado.
    :param reward: La recompensa obtenida para ese nodo.

```

```

"""
chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

# Actualizar recompensa promedio
player_that_moved = -chosen_node.state.turn
if player_that_moved == 1:
    chosen_node.Q1 += (reward - chosen_node.Q1) / chosen_node.N
elif player_that_moved == -1:
    chosen_node.Q2 += (reward - chosen_node.Q2) / chosen_node.N

```

B.3.3. UCB2

La política de selección UCB2, explicada en la sección 2.2.2.1.2, ha sido implementada en el archivo `ucb2.py` como se muestra a continuación:

```

import sys
import os
import numpy as np

sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))

from algorithm import Algorithm
from monte_carlo_tree_search import Node

class UCB2(Algorithm):
    def __init__(self, node: Node, alpha: float = 0.5):
        """
        Inicializa el algoritmo UCB2.

        :param node: Nodo actual en el árbol.
        :param alpha: Parámetro para regular la exploración. Valores más grandes aumentan
        la exploración.
        :raises ValueError: Si alpha no está en (0, 1).
        """
        assert 0 < alpha < 1, "El parámetro alpha debe estar entre 0 y 1."

        super().__init__(node)
        self.alpha = alpha # Parámetro que regula la exploración
        self.d_ka = {} # Número de épocas para cada nodo
        self.d_remaining_plays = {} # Jugadas restantes por nodo

    def set_node(self, node: Node):
        """
        Método para cambiar de nodo.
        """
        self.node = node
        self.d_ka = {}
        self.d_remaining_plays = {}

    def select_node(self) -> Node:
        """

```

```

        Selecciona el nodo hijo con el mayor valor UCB2.

        :return: El nodo hijo seleccionado.
        """
        if not self.node.children:
            raise ValueError("UCB2: no hay nodos hijos disponibles para seleccionar.")

        # Comprobar si hay una jugada ganadora inmediata
        for child in self.node.children:
            if child.state.terminal() and child.state.winner() == self.node.state.turn:
                return child # Ejecutar la jugada ganadora directamente

        # Inicializar los nuevos hijos en los diccionarios
        for child in self.node.children:
            if child not in self.d_ka:
                self.d_ka[child] = 0
                self.d_remaining_plays[child] = 0

        # Revisar si algún hijo está en medio de una época
        for child in self.node.children:
            if self.d_remaining_plays[child] > 0:
                self.d_remaining_plays[child] -= 1
                return child

        total_pulls = sum(child.N for child in self.node.children) # Total de
        simulaciones realizadas
        ucb_values = np.zeros(len(self.node.children))

        for i, child in enumerate(self.node.children):
            if child.N == 0: # Si nunca ha sido seleccionado, asignar un valor muy alto
                para exploración
                ucb_values[i] = float('inf')
            else:
                player_in_turn = self.node.state.turn
                Q = child.Q1 if player_in_turn == 1 else child.Q2

                # Fórmula de UCB2:  $Q + \sqrt{((1 + \alpha) * \log(e * \text{total} / \text{tau\_ka})) / (2 * \text{tau\_ka})}$ 
                ka = self.d_ka[child]
                tau_ka = np.ceil((1 + self.alpha) ** ka)
                ratio = np.exp(1) * total_pulls / max(tau_ka, 1)
                log_term = np.log(ratio) if ratio > 0 else 0
                sqrt_term = np.sqrt(((1 + self.alpha) * log_term) / (2 * tau_ka)) if
                log_term > 0 else 0
                ucb_values[i] = Q + sqrt_term

        # Seleccionar el nodo con el mayor UCB2
        selected_index = np.argmax(ucb_values)
        selected_node = self.node.children[selected_index]

        # Nueva época
        ka = self.d_ka[selected_node]
        tau_ka = np.ceil((1 + self.alpha) ** ka)
        tau_ka_plus_1 = np.ceil((1 + self.alpha) ** (ka + 1))

```

```

# Número de veces que el nodo será seleccionado en la nueva época
time_block = int(tau_ka_plus_1 - tau_ka)

self.d_remaining_plays[selected_node] = time_block - 1
self.d_ka[selected_node] += 1

return selected_node

def update(self, chosen_node: Node, reward: float):
    """
    Actualiza las recompensas y visitas para el nodo hijo seleccionado.

    :param chosen_node: El nodo hijo seleccionado.
    :param reward: La recompensa obtenida para ese nodo.
    """
    chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

    # Actualizar recompensa promedio
    player_that_moved = -chosen_node.state.turn
    if player_that_moved == 1:
        chosen_node.Q1 += (reward - chosen_node.Q1) / chosen_node.N
    elif player_that_moved == -1:
        chosen_node.Q2 += (reward - chosen_node.Q2) / chosen_node.N

```

B.3.4. Softmax

La política de selección Softmax, explicada en la sección 2.2.2.1.3, ha sido implementada en el archivo `softmax.py` como se muestra a continuación:

```

import sys
import os
import numpy as np

sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))

from algorithm import Algorithm
from monte_carlo_tree_search import Node

class Softmax(Algorithm):
    def __init__(self, node: Node, tau = 1):
        """
        Inicializa el algoritmo Softmax.

        :param node: Nodo actual en el árbol de búsqueda.
        :param tau: Parámetro que controla el equilibrio entre exploración y explotación.
        :raises ValueError: Si tau no es mayor que 0.
        """
        assert tau > 0, "El parámetro tau debe ser mayor que 0."

        super().__init__(node)

```

```

        self.tau = tau

    def set_node(self, node: Node):
        """
        Método para cambiar de nodo.
        """
        self.node = node

    def select_node(self) -> Node:
        """
        Selecciona un nodo hijo basado en la política Softmax.
        :return: El nodo hijo seleccionado.
        """
        if not self.node.children:
            raise ValueError("Softmax: no hay nodos hijos disponibles para seleccionar.")

        # Comprobar si hay una jugada ganadora inmediata
        for child in self.node.children:
            if child.state.terminal() and child.state.winner() == self.node.state.turn:
                return child # Ejecutar la jugada ganadora directamente

        # Calcular las probabilidades de selección para cada nodo hijo
        player_in_turn = self.node.state.turn
        Q_values = [child.Q1 if player_in_turn == 1 else child.Q2 for child in self.node.children]
        Q_values = np.array(Q_values)
        Q_stable = Q_values - np.max(Q_values) # Estabilizar los valores
        exp_values = np.exp(Q_stable / self.tau)
        prob = exp_values / np.sum(exp_values)

        # Seleccionar un nodo hijo de acuerdo con la distribución de probabilidad calculada
        selected_index = np.random.choice(len(self.node.children), p=prob)

        return self.node.children[selected_index]

    def update(self, chosen_node: Node, reward: float):
        """
        Actualiza las recompensas y visitas para el nodo hijo seleccionado.

        :param chosen_node: El nodo hijo seleccionado.
        :param reward: La recompensa obtenida para ese nodo.
        """
        chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

        # Actualizar recompensa promedio
        player_that_moved = -chosen_node.state.turn
        if player_that_moved == 1:
            chosen_node.Q1 += (reward - chosen_node.Q1) / chosen_node.N
        elif player_that_moved == -1:
            chosen_node.Q2 += (reward - chosen_node.Q2) / chosen_node.N

```

B.3.5. Softmax Adaptativo

La política de selección Softmax Adaptativo, explicada en la sección 2.2.2.1.3, ha sido implementada en el archivo `adaptive_softmax.py` como se muestra a continuación:

```
import sys
import os
import numpy as np

sys.path.append(os.path.abspath("../methods"))
sys.path.append(os.path.abspath("../selection"))

from algorithm import Algorithm
from monte_carlo_tree_search import Node

class AdaptiveSoftmax(Algorithm):
    def __init__(self, node: Node, tau_0 = 1, alpha = 0.5):
        """
        Inicializa el algoritmo AdaptiveSoftmax con ajuste dinámico de tau.

        :param node: Nodo actual en el árbol de búsqueda.
        :param tau_0: Valor inicial del parámetro tau.
        :param alpha: Parámetro de ajuste para la disminución de tau (0.001 - 1 |
Exploración - Explotación).
        """
        assert tau_0 > 0, "El parámetro tau_0 debe ser mayor que 0."
        assert 0 < alpha <= 1, "El parámetro alpha debe estar entre 0 y 1."

        super().__init__(node)
        self.tau_0 = tau_0
        self.alpha = alpha
        self.t = 0 # Contador de iteraciones

    def set_node(self, node: Node):
        """
        Método para cambiar de nodo.
        """
        self.node = node
        self.t = 0

    def select_node(self) -> Node:
        """
        Selecciona un nodo hijo basado en la política Softmax con tau adaptativo.
        :return: Nodo hijo seleccionado.
        """
        if not self.node.children:
            raise ValueError("AdaptiveSoftmax: no hay nodos hijos disponibles para seleccionar.")

        # Comprobar si hay una jugada ganadora inmediata
        for child in self.node.children:
            if child.state.terminal() and child.state.winner() == self.node.state.turn:
                return child # Ejecutar la jugada ganadora directamente
```



```

# Ajuste dinámico de tau
tau = max(self.tau_0 / (1 + self.alpha * self.t), 1e-2) # Evitar tau demasiado
pequeño

# Obtener valores Q de los nodos hijos
player_in_turn = self.node.state.turn
Q_values = [child.Q1 if player_in_turn == 1 else child.Q2 for child in self.node.
children]
Q_values = np.array(Q_values)
Q_stable = Q_values - np.max(Q_values) # Estabilizar los valores
exp_values = np.exp(np.clip(Q_stable / tau, -500, 500)) # Evita desbordamientos
prob = exp_values / np.sum(exp_values) # Normalización de Softmax

# Seleccionar nodo de acuerdo con la distribución de probabilidad
selected_index = np.random.choice(len(self.node.children), p=prob)

return self.node.children[selected_index]

def update(self, chosen_node: Node, reward: float):
    """
    Actualiza la recompensa y las visitas para el nodo hijo seleccionado, y el
    contador de iteraciones.

    :param chosen_node: El nodo hijo seleccionado.
    :param reward: La recompensa obtenida para ese nodo.
    """
    self.t += 1 # Actualizar contador

    chosen_node.N += 1 # Incrementar el número de visitas del nodo hijo

    # Actualizar recompensa promedio
    player_that_moved = -chosen_node.state.turn
    if player_that_moved == 1:
        chosen_node.Q1 += (reward - chosen_node.Q1) / chosen_node.N
    elif player_that_moved == -1:
        chosen_node.Q2 += (reward - chosen_node.Q2) / chosen_node.N

```

B.4. Implementación de la herramienta de depuración para MCTS con graphviz

Con el fin de poder visualizar y analizar el comportamiento del método de MCTS, se ha implementado una herramienta de depuración capaz de generar un diagrama de árbol con las posiciones siguientes solicitadas a partir de un determinado nodo inicial. Para ello, se ha utilizado la biblioteca `graphviz`, que permite dibujar grafos desde Python de forma sencilla. A continuación, se muestra la función encargada de generar el grafo del árbol de búsqueda generado por MCTS, y guardarlo en un archivo para poder visualizarlo con posterioridad.

```

from graphviz import Digraph

def generate_tree_MCTS(root_node, max_depth=1, filename='arbol_mcts', format='pdf'):
    """
    Genera un árbol a partir de un nodo raíz de MCTS y lo guarda.

    Parámetros:
    - root_node: Nodo raíz del árbol.
    - max_depth: Profundidad máxima a recorrer.
    - filename: Nombre del archivo generado.
    - format: Formato del archivo generado.

    Devuelve:
    - Un diccionario que mapea IDs de nodo (str) a objetos nodo (Node).
    """
    # Inicializar el grafo
    dot = Digraph(comment='Árbol MCTS')

    counter = 0 # Contador de nodos
    id_to_node = {} # Diccionario ID -> Nodo

    # Crear ID y etiqueta para el nodo raíz
    node_id = f"nodo{counter}"
    counter += 1
    label = f"ID: {node_id}\\nTurno: {root_node.state.turn}\\nVisitas: {root_node.N}\\nRecompensa 1: {root_node.Q1}\\nRecompensa 2: {root_node.Q2}"

    # # Añadir nodo al grafo
    dot.node(node_id, label)

    # Guardar en el diccionario y añadir al stack
    id_to_node[node_id] = root_node
    stack = [(root_node, node_id, 0)]

    # Recorrer el árbol en profundidad hasta max_depth
    while stack:
        node, node_id, depth = stack.pop() # Extraer un nodo del stack
        if depth < max_depth:

            # Recorrer los nodos hijos
            for child_node in node.children:

                # Crear ID y etiqueta para el nodo hijo
                child_id = f"nodo{counter}"
                counter += 1
                label = (f'ID: {child_id}\\nTurno: {child_node.state.turn}\\nVisitas: {
child_node.N}'
                        f'\\nRecompensa 1: {child_node.Q1}\\nRecompensa 2: {child_node.
Q2}')
                edge_label = f"Acción: {child_node.action_from_parent}" # Acción desde
el nodo padre

                # Añadir nodo y arista al grafo
                dot.node(child_id, label)

```

```
dot.edge(node_id, child_id, label=edge_label)

# Guardar en el diccionario y añadir al stack
id_to_node[child_id] = child_node
stack.append((child_node, child_id, depth + 1))

# Renderizar el grafo
dot.render(filename, format=format, view=True)

return id_to_node
```